

Lecture Notes
Iterative Methods in Matrix Computations

Davoud Mirzaei

September 30, 2026

Contents

1	Large and sparse linear systems	4
1.1	Sparse representation formats	6
1.2	Sparse matrices in Python and MATLAB	9
1.3	Sparse matrix operations and fill-in	10
2	Classical iterative methods	12
2.1	Computational costs	15
2.2	Convergence analysis	15
2.3	Relaxation	18
2.4	Preconditioning and splitting	21
2.5	Polynomial representation	23
3	Krylov subspace methods	24
3.1	Classification of methods	25
3.2	Arnoldi algorithm	25
3.3	Lanczos algorithm	28
3.4	Galerkin methods	29
3.5	Conjugate Gradient method	35
3.6	CG connection to Lanczos process	39
3.7	Convergence of the CG method	40
3.8	Minimum residual methods	43
3.9	GMRES methods	44
3.10	MINRES method	50
3.11	Convergence of GMRES and MINRES methods	54
3.12	Petrov–Galerkin methods	57
3.13	BiCG method	58
3.14	Quasi-MINRES (QMR) algorithm	64
3.15	Transpose-free methods	66
4	Preconditioned iterative methods	66
4.1	Preconditioned CG algorithm	67

4.2	Preconditioned GMRES algorithm	68
4.3	Preconditioned MINRES algorithm	71
4.4	Preconditioned BiCG algorithm	72
4.5	Different types of preconditioners	73
4.5.1	Algebraic preconditioners	73
4.5.2	Problem-dependent preconditioners	76
5	Numerical examples	77
5.1	Symmetric (in)definite systems	78
5.2	Nonsymmetric systems	81
5.3	Which method?	84
6	Eigenvalue computations for large sparse matrices	84
6.1	Rayleigh–Ritz projection	85
6.2	Arnoldi method for the eigenvalue problem	86
6.3	Lanczos method for symmetric matrices	87
6.4	Polynomial interpretation of the Arnoldi method	88
6.5	Approximation of extreme eigenvalues by Lanczos	92
6.6	Numerical experiments with Lanczos method	92
6.7	Reorthogonalization in the Lanczos method	95
6.8	Computational cost	96
6.9	Arnoldi lemniscates	97
6.10	Shift-and-invert transformation	99
6.11	Restarting	101
7	Additional Exercises	101
A	Appendix	107
A.1	Implementation of Lanczos-Galerkin (LanGal) method	107
A.2	Householder and Givens transformations	111

Iterative Methods in Matrix Computations

In these notes, we introduce iterative methods for solving large and sparse systems of linear equations and eigenvalue problems. We start with a brief review of sparse matrices and common sparse matrix storage formats. We then discuss classical iterative methods, including the Richardson, Jacobi, Gauss–Seidel, and successive over-relaxation (SOR) methods. Then we study the more sophisticated Krylov subspace methods. In this part, we first introduce the Arnoldi and Lanczos processes, which provide the basic tools for constructing suitable Krylov subspaces. We then develop projection methods based on the Galerkin condition, including the conjugate gradient (CG) method for symmetric positive definite systems, as well as minimum-residual methods such as GMRES and MINRES. The notes continue with an introduction to Petrov–Galerkin methods, including BiCG and QMR. Finally, we discuss preconditioned iterative methods and introduce some commonly used classes of preconditioners. The last section is devoted to Krylov subspace methods for solving large eigenvalue problems. We introduce the Arnoldi and Lanczos methods, shift and invert transformation, their advantages and limitations.

Throughout the notes, particular attention is given to computational aspects of the algorithms, their convergence properties, and their behavior in finite-precision arithmetic.

These notes are intended as course material rather than as a comprehensive study of iterative methods or sources for research studies. For further details and additional theoretical and computational results the reader is encouraged to refer to [2, 6, 13, 20].

In preparing these notes, AI tools have been used to assist with the presentation and refinement of some parts of the text, in particular, for language editing, improving the exposition of selected topics, preparation of TikZ plots, some examples and exercises. The mathematical content, concepts, numerical experiments, organization of the material, and accuracy of the notes remain the responsibility of the author.

1 Large and sparse linear systems

Many applications in scientific computing lead to linear systems

$$Ax = b,$$

where the dimension n of the matrix $A \in \mathbb{R}^{n \times n}$ can be very large. Typical examples arise from the discretization of partial differential equations (PDEs), optimization problems, network and AI models, and many other applications. The meaning of a “large” matrix has been changed with time and with the available computer hardware. A matrix that was considered large several decades ago may now fit easily into the memory of a personal computer. From the other hand, the feasibility of a computation depends not only on the dimension n , but also on the *structure* of the matrix.

If a matrix is stored as a dense array, all its n^2 entries must be stored. For example, using double-precision floating-point arithmetic, each entry requires 8 bytes, and hence storing a dense matrix requires approximately $8n^2$ bytes. Thus, the storage requirement grows quadratically with n :

n	n^2 entries	storage
10^3	10^6	8 MB
10^4	10^8	0.8 GB
10^5	10^{10}	80 GB
10^6	10^{12}	8 TB

These numbers show that even before solving the system, storing a large matrix densely may be impossible. Similarly, standard direct methods for dense linear systems require $\mathcal{O}(n^3)$ arithmetic operations.

In many large-scale applications, however, most entries of A are zero. This leads to the important concept of a *sparse matrix*. A matrix is called sparse if only a relatively small fraction of its entries are nonzero. We denote the number of nonzero entries of A by

$$\text{nnz}(A).$$

For a sparse matrix,

$$\text{nnz}(A) \ll n^2.$$

There is no strict threshold that separates sparse matrices from dense ones. Usually, if

$$\text{nnz}(A) = \mathcal{O}(\log n) \cdot n$$

the matrix is called a sparse matrix. However, in most realistic applications $\text{nnz}(A) = \eta \cdot n$ where η is reasonable constant. The important point is that the number and distribution of the nonzero entries make it advantageous to store and operate on only the nonzero entries. As a simple example, consider the tridiagonal matrix

$$A = \begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{bmatrix} \in \mathbb{R}^{n \times n}.$$

Although A has n^2 entries, only

$$\text{nnz}(A) = 3n - 2$$

of them are nonzero. Hence, for large n , storing all n^2 entries would be extremely wasteful.

Sparse matrices can have many different patterns. Some common examples are *diagonal*, *banded*, or *block sparse*. For example, matrices resulting from finite difference or finite element discretizations of PDES often have a sparse structure because each unknown interacts only with a small number of nearby unknowns. For example, the standard five-point finite difference discretization of a two-dimensional elliptic problem produces a matrix in which each row contains at most five nonzero entries. See Figure 1. In this case, $\text{nnz}(A) \approx 5n$, even though A contains n^2 entries in total.

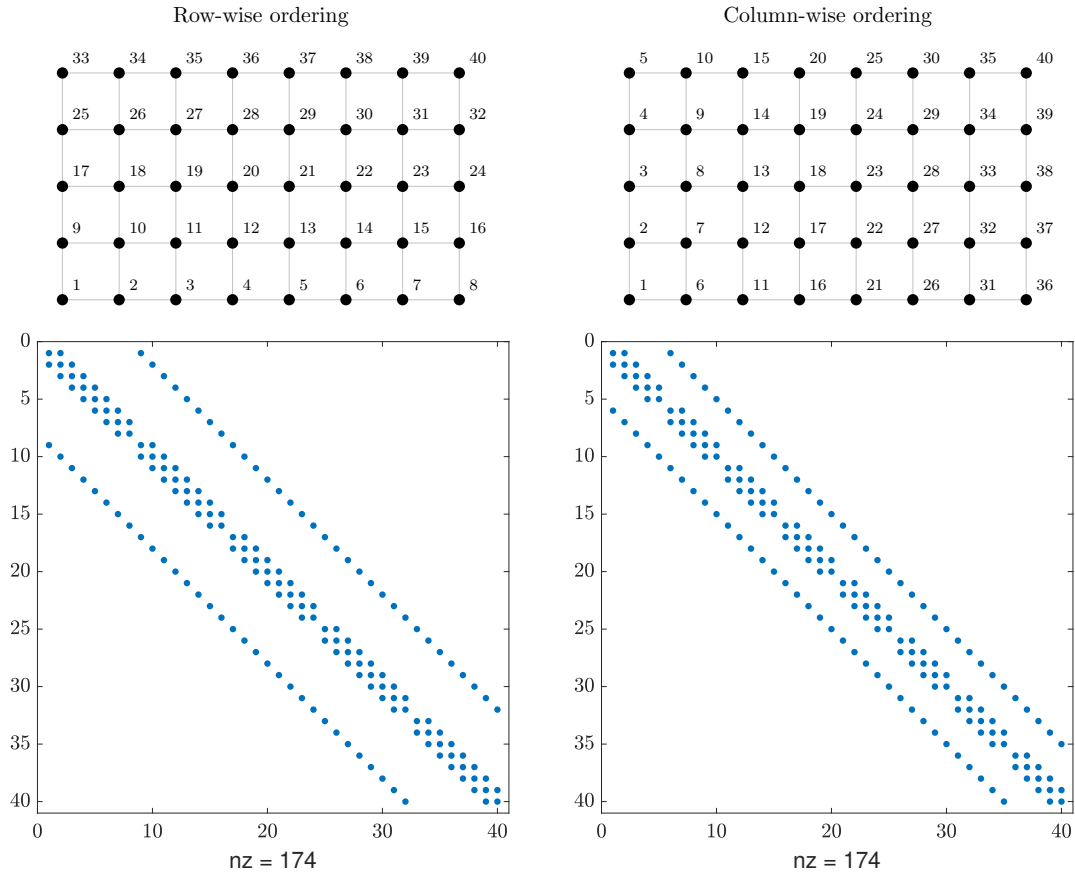


Figure 1: Finite difference discretization of Laplacian operator $\Delta u = u_{xx} + u_{yy}$ on a Cartesian grid of size 8×5 . Two different orderings of the unknowns on the grid with their corresponding Laplacian matrix.

The sparsity pattern of a matrix is the set

$$\mathcal{S}(A) = \{(i, j) : a_{ij} \neq 0\}.$$

When studying a large sparse matrix, it is often more informative to visualize its sparsity pattern than its numerical entries. MATLAB's `spy(A)` command, for example, displays a point at every location (i, j) for which $a_{ij} \neq 0$. See Figure 1. The corresponding command

in Python, using the `matplotlib` library, is `plt.spy(A)`, where `plt` is imported by `import matplotlib.pyplot as plt`.

1.1 Sparse representation formats

Now, consider a matrix $A \in \mathbb{R}^{n \times n}$ having only a small number of nonzero entries in each row. Storing all n^2 entries would mean that most of the memory is used to store zeros. Note that storing a zero in standard floating-point format requires the same amount of memory as storing any other floating-point number. A sparse representation therefore stores only the nonzero values, together with enough information to identify their locations. As an illustrative example, consider

$$A = \begin{bmatrix} 10 & 0 & 0 & 2 & 0 \\ 3 & 9 & 0 & 0 & 0 \\ 0 & 7 & 8 & 0 & 0 \\ 0 & 0 & 1 & 5 & 0 \\ 0 & 0 & 0 & 6 & 4 \end{bmatrix}. \quad (1.1)$$

This matrix has $n^2 = 25$ entries and $\text{nnz}(A) = 10$ nonzero entries. A dense representation stores all 25 entries, whereas a sparse representation needs to store only the 10 nonzero values and information about their positions.

There are several ways of organizing this information. The most common formats include coordinate storage (COO), compressed sparse row (CSR), and compressed sparse column (CSC).

Coordinate format

Perhaps the simplest sparse representation is the *coordinate format*, usually abbreviated as COO. Each nonzero entry is represented by the triple

$$(i, j, a_{ij}),$$

where i is the row index, j is the column index, and a_{ij} is the value of the entry. For the matrix in (1.1), ordering the entries row by row gives

value	10	2	3	9	7	8	1	5	6	4
row	1	1	2	2	3	3	4	4	5	5
column	1	4	1	2	2	3	3	4	4	5

Thus, instead of storing the entire matrix, we store three arrays, which may be written schematically as

$$\begin{aligned} \text{data} &= [10, 2, 3, 9, 7, 8, 1, 5, 6, 4], \\ \text{row} &= [1, 1, 2, 2, 3, 3, 4, 4, 5, 5], \\ \text{col} &= [1, 4, 1, 2, 2, 3, 3, 4, 4, 5]. \end{aligned}$$

The storage requirement is therefore $3 \cdot \text{nnz}(A)$ rather than n^2 . COO is conceptually simple and is particularly convenient when a matrix is being assembled. For example, contributions to a matrix can be generated one at a time as triples (i, j, a_{ij}) . Once the matrix has been assembled, however, another representation is usually more efficient for repeated matrix operations.

Compressed sparse row format

The *compressed sparse row* format, abbreviated CSR or sometimes CRS, is designed to provide efficient access to the rows of a sparse matrix. Let us again consider the matrix (1.1). Reading the nonzero entries row by row gives

$$\underbrace{10, 2}_{\text{row 1}}, \quad \underbrace{3, 9}_{\text{row 2}}, \quad \underbrace{7, 8}_{\text{row 3}}, \quad \underbrace{1, 5}_{\text{row 4}}, \quad \underbrace{6, 4}_{\text{row 5}}.$$

CSR stores these entries consecutively:

$$\mathbf{data} = [10, 2, 3, 9, 7, 8, 1, 5, 6, 4].$$

The corresponding column indices are

$$\mathbf{col} = [1, 4, 1, 2, 2, 3, 3, 4, 4, 5].$$

At this point we know the values and their column locations, but we still need to know which entries belong to each row. This is the purpose of the *row pointer* array. Using one-based indexing, the first row starts at position 1 of **data**, the second row at position 3, the third at position 5, and so forth. We therefore store

$$\mathbf{rowptr} = [1, 3, 5, 7, 9, 11]. \tag{1.2}$$

The last entry, 11, is one position beyond the last stored nonzero. These arrays completely determine the matrix. The interpretation of **rowptr** is particularly important. The nonzero entries of row i occupy the positions

$$\mathbf{rowptr}(i), \dots, \mathbf{rowptr}(i+1) - 1$$

in the arrays **data** and **col**. For example, $\mathbf{rowptr}(3) = 5$ and $\mathbf{rowptr}(4) = 7$. Hence, the nonzero entries in row 3 occupy positions 5 and 6, i.e. $\mathbf{data}(5:6) = [7, 8]$ and their column indices are $\mathbf{col}(5:6) = [2, 3]$.

Notice that **rowptr** contains $n + 1$ entries. The extra entry is needed to indicate where the last row ends. Another useful interpretation is

$$\mathbf{rowptr}(i+1) - \mathbf{rowptr}(i) = \text{number of nonzeros in row } i.$$

For example, $\mathbf{rowptr}(3) - \mathbf{rowptr}(2) = 5 - 3 = 2$, so row 2 has two nonzero entries.

Note that the indices in **col** and **rowptr** are written here using one-based indexing, as in MATLAB. In actual implementations, however, zero-based indexing is also common. For example, SciPy in Python uses zero-based indexing. Thus, for the example above, the corresponding pointer array is $\mathbf{rowptr} = [0, 2, 4, 6, 8, 10]$.

Sparse matrix–vector multiplication

One of the most important operations in iterative methods is the matrix–vector product

$$y = Ax.$$

Its i th component is

$$y_i = \sum_{j=1}^n a_{ij}x_j = \sum_{\substack{j \\ a_{ij} \neq 0}} a_{ij}x_j.$$

There is clearly no need to perform operations involving zero entries. CSR is particularly well suited to this operation because the nonzero entries of each row are stored consecutively. In one-based pseudocode, sparse matrix–vector multiplication can be written as

Algorithm 1: Sparse matrix-vector multiplication in CSR format

Input: CSR representation `data`, `col`, `rowptr` of $A \in \mathbb{R}^{n \times n}$; vector $x \in \mathbb{R}^n$

Output: $y = Ax$

```

for  $i = 1$  to  $n$  do
  Set  $y_i = 0$ 
  for  $k = \text{rowptr}(i)$  to  $\text{rowptr}(i+1) - 1$  do
     $j = \text{col}(k)$ 
     $y_i = y_i + \text{data}(k)x_j$ 
  end for
end for

```

Assuming one multiplication and one addition for every nonzero entry, the cost of sparse matrix–vector multiplication is approximately $2 \text{nnz}(A)$ flops. Thus, if $\text{nnz}(A) = \mathcal{O}(n)$, the matrix–vector product requires only $\mathcal{O}(n)$ operations, in contrast to the $\mathcal{O}(n^2)$ cost for a general dense matrix. This observation will be fundamental when we study iterative methods. Many such methods are organized around repeated sparse matrix–vector products.

Compressed sparse column format

The *compressed sparse column* format (CSC) is the column analogue of CSR. Instead of storing the nonzero entries row by row, CSC stores them column by column. A CSC representation therefore consists of

`data`, `row`, `colptr`,

where `colptr` identifies the beginning and end of each column.

CSR and CSC contain essentially the same information, but their organization makes different operations convenient. CSR provides efficient access to rows while CSC provides efficient access to columns. Consequently, CSR is a natural format for repeated computation of Ax , while CSC is often convenient when an algorithm needs direct access to columns. Both formats can, of course, be used to perform either operation.

Other sparse storage formats

There are many sparse storage formats, each designed for particular types of matrices or operations. Some common examples are summarized below:

Format	Main idea	Typical use
COO	(i, j, a_{ij}) triples	matrix construction
CSR	compressed rows	Ax , row access
CSC	compressed columns	column access
DIA	store selected diagonals	banded/structured matrices
LIL	list of entries in each row	incremental construction

For example, if a matrix has only a few nonzero diagonals, as in many finite difference discretizations, a diagonal format can avoid storing an index for every individual entry. There is therefore no universally optimal sparse representation. The appropriate format depends on the matrix structure and on the operations that will be performed. It is also common to construct a matrix in one format and then convert it to another format for the actual numerical computation.

1.2 Sparse matrices in Python and MATLAB

The `scipy.sparse` module provides several sparse matrix representations in Python. For example, the matrix in (1.1) can first be constructed in COO format:

```
import numpy as np
from scipy.sparse import coo_matrix

row = np.array([0,0,1,1,2,2,3,3,4,4])
col = np.array([0,3,0,1,1,2,2,3,3,4])
val = np.array([10,2,3,9,7,8,1,5,6,4])

A = coo_matrix((val, (row, col)), shape=(5,5))
```

Python uses zero-based indexing, which explains why the first row and column are numbered 0 rather than 1. After construction, the matrix can be converted to CSR:

```
A = A.tocsr()
```

and a matrix–vector product is simply

```
y = A @ x
```

The three principal arrays of the CSR representation are accessible as

```
A.data      # nonzero values
A.indices    # column indices
A.indptr     # row pointer array
```

In additions, SciPy provides several sparse formats, including COO, CSR, CSC, DIA, and LIL. A common strategy is to use a format convenient for construction and then convert to CSR or CSC before carrying out the main computation.

MATLAB also provides built-in support for sparse matrices. A sparse matrix can be constructed by specifying the row indices, column indices, and values of its nonzero entries. For matrix (1.1), we may write

```
i = [1 1 2 2 3 3 4 4 5 5];
j = [1 4 1 2 2 3 3 4 4 5];
v = [10 2 3 9 7 8 1 5 6 4];

A = sparse(i,j,v,5,5);
```

MATLAB uses a compressed sparse column representation internally. Once the matrix has been constructed, many operations use the same syntax as for dense matrices:

```
y = A*x;
z = A\b;
B = A';
```

Some useful MATLAB commands are

```
nnz(A)      % number of nonzero entries
spy(A)      % display the sparsity pattern
sparse(A)   % convert to sparse representation
full(A)     % convert to dense representation
```

The distinction between mathematical notation and storage format is important. The mathematical matrix A is the same whether it is stored densely, in CSR, or in CSC format. The storage format merely determines how the entries are organized in computer memory and, consequently, which operations can be performed most efficiently.

1.3 Sparse matrix operations and fill-in

Sparse storage is useful only if the numerical algorithms also exploit the sparsity. Some operations preserve sparsity naturally, whereas others may create new nonzero entries. For example, if A is sparse, as we saw the product

$$y = Ax$$

can be computed without changing the sparsity structure of A . Similarly, operations such as

$$r = b - Ax$$

can be carried out in $\mathcal{O}(\text{nnz}(A))$ operations. Matrix factorization is more complicated. Even if A is sparse, the factors in

$$A = LU$$

need not have the same sparsity pattern. Entries that are zero in A may become nonzero in L or U . This phenomenon is known as *fill-in*. The amount of fill-in depends strongly on the

sparsity pattern and on the ordering of the unknowns. As an example, consider the sparse arrow matrix

$$A = \begin{bmatrix} n & 1 & 1 & \cdots & 1 \\ 1 & n & 0 & \cdots & 0 \\ 1 & 0 & n & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ 1 & 0 & \cdots & 0 & n \end{bmatrix}.$$

The matrix has only $3n - 2$ nonzero entries. Nevertheless, Gaussian elimination (LU factorization) creates new nonzero entries in positions that are zero in the original matrix. See the sparsity pattern in Figure (2).

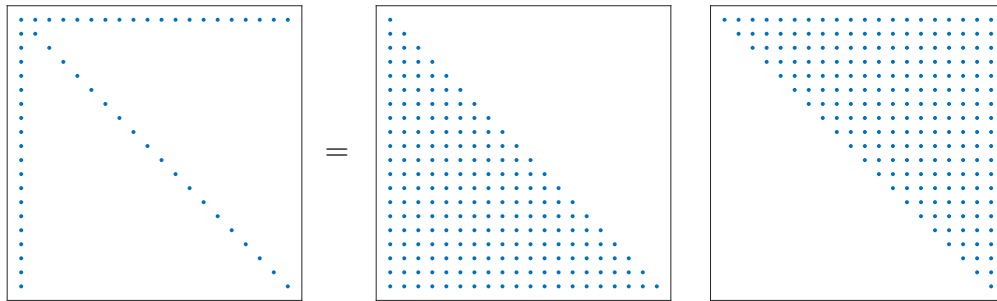


Figure 2: Fill-in in the LU factorization of the sparse arrow matrix.

Sparse direct methods therefore use sophisticated reordering strategies to reduce fill-in. Nevertheless, for very large problems, the storage and computational cost of the factors can become much greater than the cost of storing the original sparse matrix. This is one of the main motivations for iterative methods. For a large sparse linear system

$$Ax = b, \tag{1.3}$$

we would ideally like a numerical method that

1. stores only $\mathcal{O}(\text{nnz}(A))$ data,
2. exploits sparse matrix–vector products,
3. avoids excessive fill-in,
4. requires substantially less than the cost of a dense $\mathcal{O}(n^3)$ factorization,
5. and can take advantage of special properties of A , such as symmetry or positive definiteness.

Iterative methods attempt to achieve these goals by starting from an initial approximation x_0 and constructing a sequence

$$x_0, x_1, x_2, \dots$$

that approaches the solution of (1.3). The simplest iterative schemes are the classical stationary methods which are the topic of the next section. These methods also provide the basic ideas needed to understand preconditioning and the more powerful Krylov subspace methods that will be studied later.

2 Classical iterative methods

We start by introducing the idea of classical iterative methods for solving the linear system (1.3). A common approach is to split the matrix A into two parts,

$$A = M - R, \quad (2.1)$$

where M is a nonsingular matrix that approximates A and R is the remaining part of the matrix. Such a decomposition is called a *matrix splitting*, and the matrix M is referred to as the *splitting matrix*. Different choices of M lead to different iterative methods. Using the splitting (2.1), the system (1.3) can be rewritten as $Mx = Rx + b$. Since M is assumed to be nonsingular, we obtain

$$x = M^{-1}Rx + M^{-1}b.$$

This naturally suggests the fixed-point iteration

$$Mx_{m+1} = Rx_m + b, \quad \text{or equivalently} \quad x_{m+1} = M^{-1}Rx_m + M^{-1}b. \quad (2.2)$$

Another equivalent form is obtained using the fact that $Rx_m + b = Mx_m - Ax_m + b$ to get

$$x_{m+1} = x_m + M^{-1}(b - Ax_m), \quad (2.3)$$

where $b - Ax_m =: r_m$ is the *residual* at iteration m . Thus, every matrix splitting gives rise to an iterative method. The quality of the method depends on how well M approximates A and how much cheap and efficiently systems involving M can be solved.

The simplest possible choice is $M = I$, the identity matrix. In this case, $R = I - A$, and the iteration becomes

$$x_{m+1} = (I - A)x_m + b. \quad (2.4)$$

This method is known as the *Richardson iteration* or *simple iteration*. It can also be written in the form

$$x_{m+1} = x_m + (b - Ax_m). \quad (2.5)$$

Subtracting the exact solution x from both sides and replacing b by Ax , we obtain

$$x_{m+1} - x = (I - A)(x_m - x).$$

Taking norms (operator norms) on both sides then yields

$$\|x_{m+1} - x\| \leq \|I - A\| \|x_m - x\|.$$

Therefore, a sufficient condition for the error to decrease at each iteration is $\|I - A\| < 1$. Indeed, under this assumption the error is reduced by at least the factor $\|I - A\|$ at every step. Later, we will derive a necessary and sufficient condition for convergence. Unfortunately, such condition is often too restrictive in practical applications. A natural improvement is to choose M as the diagonal part of A . Let

$$A = D - E - F, \quad (2.6)$$

$$A = \begin{array}{c|ccccc} & \text{light green} & \text{light red} & \text{light red} & \text{light red} & \text{light red} \\ \hline & a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ \hline & a_{21} & \text{light green} & \text{light red} & \text{light red} & \text{light red} \\ \hline & a_{31} & a_{32} & \text{light green} & \text{light red} & \text{light red} \\ \hline & \vdots & \vdots & \vdots & \text{light green} & \vdots \\ \hline & a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{array}$$

Figure 3: Splitting $A = D - E - F$, where D is the diagonal (light green), $-E$ is the strictly lower triangular part (light blue), and $-F$ is the strictly upper triangular part (light red).

where D is the diagonal of A , E is the strictly lower triangular part, and F is the strictly upper triangular part.

Taking $M = D$ gives $R = E + F$, and the iteration becomes

$$x_{m+1} = D^{-1}(E + F)x_m + D^{-1}b, \quad (2.7)$$

provided that the diagonal entries of A are nonzero. This method is known as the *Jacobi method*. The main advantage of the Jacobi method is that applying D^{-1} is extremely inexpensive. Compared with solving the original system, this represents a substantial reduction in computational cost. The price to pay is that convergence is guaranteed only under certain assumptions on the matrix A , which we will discuss later.

The Jacobi method uses only the diagonal of A . Since solving triangular systems is also relatively inexpensive, it is natural to ask whether a better approximation can be obtained by including the lower triangular part of the matrix as well. Using the splitting (2.6), let $M = D - E$ and $R = F$. The resulting iteration is

$$x_{m+1} = (D - E)^{-1}Fx_m + (D - E)^{-1}b. \quad (2.8)$$

From a computational point of view, it is more relevant to write the above iteration as

$$(D - E)x_{m+1} = Fx_m + b \quad (2.9)$$

which shows that each step requires the solution of a lower triangular system. Such systems can be solved efficiently by forward substitution, which makes the method computationally attractive. This iteration is known as the *Gauss-Seidel method*. It was introduced by Gauss in 1823 and later analyzed systematically by Seidel. In practice, Gauss-Seidel often converges more rapidly than the Jacobi method because it incorporates newly computed components of the solution immediately in each iteration. However, one can find matrices for which Jacobi converges but Gauss-Seidel does not, and also cases where Gauss-Seidel converges but Jacobi does not.

As we observed in (2.3), both Jacobi and Gauss-Seidel methods can be written in the form

$$x_{m+1} = x_m + M^{-1}(b - Ax_m)$$

where $M = D$ for Jacobi method and $M = D - E$ for Gauss-Seidel method. In this form, at each iteration we compute residual r_m and solve the linear system

$$Mz_m = r_m$$

for correction term z_m . Finally we set

$$x_{m+1} = x_m + z_m$$

and proceed to the next iteration.

Example 2.1. Consider the system

$$\begin{bmatrix} 8 & 0 & 1 \\ 1 & 10 & 2 \\ 1 & \frac{1}{2} & 7 \end{bmatrix} \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ x^{(3)} \end{bmatrix} = \begin{bmatrix} 9 \\ 23 \\ 9 \end{bmatrix},$$

which has the exact solution $x = [1, 2, 1]^T$. We denote the components of vector x by $x^{(i)}$, $i = 1, 2, 3$. After computing a few iterations, we observe that Richardson and Jacobi methods diverge while Gauss-Seidel converges quickly to the exact solution. The first three iterations are shown in Table 1.

iteration	1	2	3
$x^{(1)}$	1.12500	1.00391	1.00012
$x^{(2)}$	2.18750	2.00586	2.00018
$x^{(3)}$	0.96875	0.99902	0.99997

Table 1: Three iterations of the Gauss-Seidel method with initial guess $x_0 = [0, 0, 0]^T$.

Ideally, we would continue the iteration until the relative *forward error* is smaller than a prescribed tolerance ε , i.e.

$$\frac{\|x - x_m\|}{\|x\|} \leq \varepsilon,$$

where x is the exact solution. In practice, however, x is not available, so the forward error cannot be computed. Since the residual $r_m = b - Ax_m$ is readily available during the iteration, a natural stopping criterion is based on the relative *residual*

$$\frac{\|r_m\|}{\|b\|} \leq \varepsilon.$$

This criterion is simple and widely used, but it measures the residual only relative to the right-hand side and does not take into account the sizes of A and x_m . A more meaningful stopping criterion is based on the relative *backward error*

$$\frac{\|r_m\|_2}{\|A\|_2 \|x_m\|_2 + \|b\|_2} \leq \varepsilon$$

which is scaled with respect to both the matrix and the right-hand side.

2.1 Computational costs

The computational cost per iteration is dominated by the matrix–vector product Ax_m , which requires approximately $2n^2$ floating-point operations (flops) for *dense* matrices. In addition, solving a linear system of equations with coefficient matrix M is required. It is supposed to be much cheaper than solving the original system with matrix A . For example solving the lower triangular system in the Gauss–Seidel method requires about n^2 flops, while for the Jacobi method the cost is only n flops. Thus, the total cost per iteration is approximately $3n^2$ flops for the Gauss–Seidel method and $2n^2$ flops for the Jacobi method. If a satisfactory accuracy is achieved after m iterations, the overall computational complexity is

$$m \cdot \mathcal{O}(n^2).$$

As long as the number of iterations m is significantly smaller than n , these iterative methods are computationally more efficient than direct methods such as Gaussian elimination, which requires about $\frac{2}{3}n^3$ flops.

However, this is not the whole picture. In practice, large matrices are often *sparse*. Assume that the number of nonzero entries of A is denoted by $\text{nnz}(A)$. Also assume that A has η number of nonzero entries per row in average. Then the matrix–vector product costs only $2 \text{nnz}(A) \approx 2\eta n$ flops. Also, solving the sparse triangular system requires approximately $\eta \mathcal{O}(n)$ flops. Consequently, the total computational cost becomes

$$m \cdot \mathcal{O}(n)$$

which is a substantial gain compared to direct methods.

2.2 Convergence analysis

Considering the splitting (2.1), the exact solution x of the system $Ax = b$ satisfies $Mx = Rx + b$. The iterative method generates approximations x_m satisfying $Mx_m = Rx_{m-1} + b$. Subtracting these two relations and defining $e_m = x_m - x$ together with the iteration matrix $B = M^{-1}R$, we obtain

$$e_m = Be_{m-1}.$$

Applying the same recursion to e_{m-1} gives $e_m = B^2e_{m-2}$. Continuing in this way down to e_0 yields

$$e_m = B^m e_0, \tag{2.10}$$

where B^m denotes the m -th power of B , and $e_0 = x_0 - x$ is the error corresponding to the initial guess x_0 . For the iteration to converge, we must have $e_m \rightarrow 0$ as $m \rightarrow \infty$. Assuming that $e_0 \neq 0$ (that is, x_0 is not already the exact solution), it follows from (2.10) that convergence is equivalent to requiring B^m to approach the zero matrix as m increases. Therefore, the condition

$$B^m \rightarrow 0 \quad \text{as} \quad m \rightarrow \infty$$

is necessary and sufficient for the convergence of the iterative method (2.2). An equivalent criterion is given below.

Definition 2.1. Let $B \in \mathbb{C}^{n \times n}$, and let $\lambda_1, \dots, \lambda_n$ denote its eigenvalues. The *spectral radius* of B is defined by

$$\rho(B) := \max_{1 \leq k \leq n} |\lambda_k|.$$

Theorem 2.1. Let $B \in \mathbb{C}^{n \times n}$. Then $B^m \rightarrow 0$ as $m \rightarrow \infty$ if and only if $\rho(B) < 1$.

Proof. We prove that if $B^m \rightarrow 0$ then $\rho(B) < 1$. Let λ be an eigenvalue of B with corresponding eigenvector $v \neq 0$. Then $Bv = \lambda v$, and hence $B^m v = \lambda^m v$. Since $B^m \rightarrow 0$, it follows that $B^m v \rightarrow 0$. Therefore $\lambda^m v \rightarrow 0$. Since $v \neq 0$, this implies $\lambda^m \rightarrow 0$, which is possible only if $|\lambda| < 1$. Since this holds for every eigenvalue λ of B , we conclude that $\rho(B) < 1$. Conversely, let

$$S^{-1}BS = \begin{bmatrix} J_1 & & & \\ & J_2 & & \\ & & \ddots & \\ & & & J_p \end{bmatrix} =: J$$

be the Jordan canonical form of B , where J is block diagonal with Jordan blocks

$$J_k = \lambda_k I + N_k = \begin{bmatrix} \lambda_k & 1 & & \\ & \lambda_k & \ddots & \\ & & \ddots & 1 \\ & & & \lambda_k \end{bmatrix} \in \mathbb{C}^{\ell_k \times \ell_k}$$

where ℓ_k is the multiplicity of eigenvalue λ_k , so that $\ell_1 + \dots + \ell_p = n$. Then

$$B^m = SJ^m S^{-1},$$

so it is enough to show that $J^m \rightarrow 0$. For each Jordan block we have

$$J_k^m = (\lambda_k I + N_k)^m = \sum_{j=0}^m \binom{m}{j} \lambda_k^{m-j} N_k^j = \sum_{j=0}^{\ell_k-1} \binom{m}{j} \lambda_k^{m-j} N_k^j$$

since $N_k^j = 0$ for $j \geq \ell_k$. Since $|\lambda_k| < 1$, each term

$$\binom{m}{j} \lambda_k^{m-j}, \quad j \leq \ell_k - 1,$$

tends to zero as $m \rightarrow \infty$ because the exponential decay of λ_k^{m-j} dominates the polynomial growth of $\binom{m}{j}$. This shows that $J_k^m \rightarrow 0$ for each block k , and thus $J^m \rightarrow 0$. □

Corollary 2.2. The classical iteration $x_{m+1} = M^{-1}R x_m + M^{-1}b$, converges for every

initial vector x_0 to solution x of system $Ax = b$ if and only if

$$\rho(M^{-1}R) < 1.$$

Consequently, the Richardson method converges if and only if $\rho(I - A) < 1$, the Jacobi method converges if and only if $\rho(D^{-1}(E + F)) < 1$, and the Gauss–Seidel method converges if and only if $\rho((D - E)^{-1}F) < 1$.

Exercise 2.1. Prove that if $\|B\| < 1$ for at least one operator norm $\|\cdot\|$, then $\rho(B) < 1$ and the corresponding iterative method converges. *Hint:* Show that $\rho(B) \leq \|B\|$ for all operator norms $\|\cdot\|$.

Definition 2.2. A matrix $A = (a_{ij})$ is called row-wise strictly diagonally dominant if

$$|a_{ii}| > \sum_{j=1, j \neq i}^n |a_{ij}|, \quad i = 1, 2, \dots, n.$$

The matrix A is called columns-wise strictly diagonally dominant if A^T is row-wise strictly diagonally dominant.

Exercise 2.2. By applying the the result of Exercise 2.1, show that if the matrix A is either row-wise or column-wise strictly diagonally dominant then the Jacobi method converges for any starting vector x_0 . *Hint:* Show that in the row-wise case $\|B\|_\infty < 1$ and in the column-wise case $\|B\|_1 < 1$.

Exercise 2.3. Show that if the matrix A is either row-wise or column-wise strictly diagonally dominant then the Gauss–Seidel method converges for any starting vector x_0 .

Exercise 2.4. Test the Jacobi method for the following system and observe the divergence of the method:

$$\begin{bmatrix} 29 & 2 & 1 \\ 2 & 6 & 1 \\ 1 & 1 & \frac{1}{5} \end{bmatrix} \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ x^{(3)} \end{bmatrix} = \begin{bmatrix} 7 \\ 11 \\ 2 \end{bmatrix}.$$

Note that the exact solution to this system is $x = [0, 1, 5]^T$. Compute the eigenvalues of the matrix (e.g. using `eig` command in Matlab) and show that the matrix is symmetric positive definite. Also compute the eigenvalues of $B = D^{-1}(E + F)$ and show that the spectral radius of B is greater than 1.

The above example shows that the Jacobi method does not necessarily converge for a symmetric positive definite matrix. The following theorem has a fully positive result for the Gauss–Seidel method.

Theorem 2.3. If A is symmetric positive definite then the Gauss–Seidel method converges to the solution of $Ax = b$ for every starting vector x_0 .

Proof. The matrix B of Gauss–Seidel method is $B = (D - E)^{-1}F$. It is enough to show that $\rho(B) < 1$. Let λ be an eigenvalue of B with eigenvector $v \neq 0$. Then $Bv = \lambda v$ or $Fv = \lambda(D - E)v$. If $\lambda = 0$, then clearly $|\lambda| < 1$. Assume therefore that $\lambda \neq 0$. Multiplying the eigenvalue relation from the left by v^* gives

$$v^*Fv = \lambda v^*Dv - \lambda v^*Ev.$$

Since A is symmetric positive definite, we have $F = E^*$ and $v^*Dv > 0$ (i.e. it is real and positive, because the diagonal of a symmetric positive definite matrix is positive). Define $\alpha = v^*Dv$ and $\beta = v^*Ev$. Then $v^*Fv = \bar{\beta}$, and therefore $\bar{\beta} = \lambda(\alpha - \beta)$. Taking absolute values yields

$$|\beta| = |\lambda| |\alpha - \beta|.$$

On the other hand,

$$|\alpha - \beta|^2 - |\beta|^2 = \alpha^2 - \alpha(\beta + \bar{\beta}) = \alpha(\alpha - \beta - \bar{\beta}).$$

But $\alpha - \beta - \bar{\beta} = v^*Dv - v^*Ev - v^*Fv = v^*Av > 0$, because A is positive definite. Hence $|\alpha - \beta|^2 - |\beta|^2 > 0$, which implies $|\beta| < |\alpha - \beta|$. Combining this with $|\beta| = |\lambda| |\alpha - \beta|$, we obtain $|\lambda| < 1$. Thus every eigenvalue λ of B satisfies $|\lambda| < 1$, and hence $\rho(B) < 1$. $\square$

2.3 Relaxation

The convergence of classical iterative methods can often be accelerated by introducing a *relaxation parameter* and choosing it appropriately. For example, the relaxed Richardson iteration is given by

$$x_{m+1} = x_m + \omega(b - Ax_m),$$

where ω is the relaxation parameter. The choice $\omega = 1$ corresponds to the standard Richardson iteration (2.5). In many cases, a suitable choice of ω leads to faster convergence and may even render the method convergent when the standard iteration diverges.

For example, suppose that all eigenvalues of A are real and satisfy

$$0 < a \leq \lambda_k \leq b, \quad k = 1, 2, \dots, n.$$

The iteration matrix of the relaxed Richardson method is $B = I - \omega A$, whose eigenvalues are $\mu_k = 1 - \omega \lambda_k$. By assumption, we have $1 - \omega b \leq \mu_k \leq 1 - \omega a$. The iteration converges if and only if $\rho(B) < 1$, which is equivalent to requiring

$$-1 < 1 - \omega b \quad \text{and} \quad 1 - \omega a < 1.$$

Because $a > 0$, the second inequality holds whenever $\omega > 0$, while the first is equivalent to $\omega < 2/b$. Hence the relaxed Richardson iteration converges if and only if

$$0 < \omega < \frac{2}{b}.$$

Now assume that A is symmetric positive definite, and let $a = \lambda_{\min}(A)$, and $b = \lambda_{\max}(A)$. Since all eigenvalues are real, the spectral radius of the iteration matrix is

$$\rho(B) = \max\{|1 - \omega a|, |1 - \omega b|\}.$$

We seek the value of ω that minimizes $\rho(B)$. Since the two quantities $|1 - \omega a|$ and $|1 - \omega b|$ vary monotonically in opposite directions on the interval $(0, 2/b)$, the minimum is attained when $|1 - \omega a| = |1 - \omega b|$, which yields

$$\omega_{\text{opt}} = \frac{2}{a+b}, \quad \rho_{\text{opt}}(B) = \frac{b-a}{b+a}.$$

Recalling that the condition number of A is $\kappa(A) = b/a$, we obtain

$$\rho_{\text{opt}}(B) = \frac{\kappa(A) - 1}{\kappa(A) + 1} = 1 - \frac{2}{\kappa(A) + 1}.$$

If $\kappa(A) = 1$, then A is a scalar multiple of the identity and $\rho_{\text{opt}}(B) = 0$. This gives convergence in one iteration. As the condition number increases, the optimal spectral radius approaches 1, and the convergence becomes increasingly slow. This illustrates a general principle in iterative linear algebra: ill-conditioned systems typically require more iterations to achieve a prescribed accuracy.

For the Gauss–Seidel method, introducing a relaxation parameter leads to the well-known SOR (Successive Over-Relaxation) method, originally proposed by Young in [21, 1954]. To derive it, we first rewrite the Gauss–Seidel iteration (2.9) as

$$Dx_{m+1} = b + Ex_{m+1} + Fx_m.$$

Adding and subtracting Dx_m on the right-hand side and multiplying by D^{-1} yields

$$x_{m+1} = x_m + D^{-1}(b + Ex_{m+1} - (D - F)x_m).$$

The SOR iteration is then defined by

$$x_{m+1} = x_m + \omega D^{-1}(b + Ex_{m+1} - (D - F)x_m),$$

where ω is the relaxation parameter. Rearranging terms gives

$$x_{m+1} = Bx_m + d, \quad B = (D - \omega E)^{-1}[(1 - \omega)D + \omega F], \quad d = \omega(D - \omega E)^{-1}b.$$

Clearly, the choice $\omega = 1$ reduces the SOR method to the Gauss–Seidel iteration. The following theorem provides a necessary condition for convergence.

Theorem 2.4. Assume that B is the iteration matrix of the SOR method. Then

$$\rho(B) \geq |1 - \omega|,$$

with equality if all eigenvalues of B have modulus $|1 - \omega|$. Consequently, the SOR method can converge only if $\omega \in (0, 2)$.

Proof. Since the inverse of a triangular matrix is again triangular with reversed diagonals, and the determinant of a triangular matrix equals the product of its diagonal entries, we obtain

$$\det(B) = \det(D - \omega E)^{-1} \det[(1 - \omega)D + \omega F] = \left(\prod_i a_{ii}\right)^{-1} \left(\prod_i (1 - \omega)a_{ii}\right) = (1 - \omega)^n.$$

On the other hand, $\det(B) = \lambda_1 \lambda_2 \cdots \lambda_n$, where λ_k are the eigenvalues of B . Therefore, $|1 - \omega|^n = |\lambda_1 \lambda_2 \cdots \lambda_n| \leq \rho(B)^n$. Taking the n -th root yields $\rho(B) \geq |1 - \omega|$. Equality holds if and only if $|\lambda_k| = \rho(B) = |1 - \omega|$ for all $k = 1, \dots, n$. Finally, convergence requires $\rho(B) < 1$. Since $\rho(B) \geq |1 - \omega|$, a necessary condition for convergence is

$$|1 - \omega| < 1,$$

which is equivalent to $\omega \in (0, 2)$. □

Theorem 2.4 says that ω has to be chosen in open interval $(0, 2)$. However, this condition is only necessary and not sufficient for convergence. If A is symmetric positive definite then this condition becomes sufficient.

Theorem 2.5. If A is symmetric positive definite and $\omega \in (0, 2)$ then the SOR method converges for all starting vectors x_0 .

Proof. The proof is similar to that of Theorem 2.3 for Gauss–Seidel method and is left as an exercise. □

The convergence rate of the SOR method depends strongly on the choice of the relaxation parameter ω . Values of ω in the interval $(0, 1)$ correspond to *under-relaxation*, while values in $(1, 2)$ correspond to *over-relaxation*. The latter is often used in practice because it can accelerate convergence. However, choosing ω too close to 2 may lead to slow convergence or even divergence when the assumptions of the convergence theorem are not satisfied. In general, determining the optimal value of ω is a difficult problem because it depends on the spectrum of the iteration matrix. A remarkable exception occurs for *consistently ordered* matrices, which include many matrices arising from finite difference discretizations of elliptic partial differential equations. In this case, Young [21, 1954] proved that if $\mu = \rho(B_J)$ is the spectral radius of the Jacobi iteration matrix, then the optimal relaxation parameter for SOR is given by

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \mu^2}},$$

and the corresponding spectral radius of the iteration matrix is

$$\rho(B_{\text{SOR}}) = \omega_{\text{opt}} - 1 = \frac{1 - \sqrt{1 - \mu^2}}{1 + \sqrt{1 - \mu^2}}.$$

For many discretized elliptic problems, μ is close to 1. Writing $\mu = 1 - \varepsilon$ with $\varepsilon \ll 1$, we obtain the approximation

$$\rho(B_{\text{SOR}}) \approx 1 - 2\sqrt{2\varepsilon},$$

whereas the Jacobi method has spectral radius

$$\rho(B_J) = 1 - \varepsilon.$$

Thus the error reduction factor of SOR depends on $\sqrt{\varepsilon}$ rather than ε , explaining the substantial acceleration achieved by over-relaxation.

In practice, the optimal value of ω is rarely known in advance. Typical values used for elliptic problems lie between 1.5 and 1.95. The performance of the method is often quite sensitive to the choice of ω . Nevertheless, for many years after its introduction in [21, 1954], the SOR method remained one of the most important ‘workhorses’ of scientific computing.

2.4 Preconditioning and splitting

Iterative methods typically converge rapidly when the matrix A is close to the identity matrix. In practice, however, A is often far from the identity, which may result in slow convergence and require many iterations to achieve a desired accuracy. In such cases, the computational advantage of iterative methods can be significantly reduced. To improve convergence, one may replace the original system $Ax = b$ with the modified system

$$M^{-1}Ax = M^{-1}b, \tag{2.11}$$

where M is a nonsingular matrix chosen so that the new matrix $\tilde{A} = M^{-1}A$ is closer to the identity matrix, or at least has spectral properties that lead to faster convergence. The matrix M is called a *preconditioner*, and the modified system (2.11) is called the *preconditioned system*.

We deliberately denote the preconditioner by M , and now we return to the general iteration (2.3),

$$x_{m+1} = x_m + M^{-1}(b - Ax_m),$$

for solving non-preconditioned system $Ax = b$. We see that this iteration can be interpreted as the Richardson iteration (2.5) applied to the preconditioned system (2.11). Classical methods such as Jacobi and Gauss–Seidel can be viewed in this framework, with specific choices of the preconditioner M . These methods often show improved convergence properties compared to Richardson method because the preconditioned matrix $M^{-1}A$ gives better spectral properties for convergence.

Consequently, a preconditioner M is an splitting matrix, so that

$$A = M - R.$$

From practical point of view, M should satisfy two competing requirements. First, systems involving M should be inexpensive to solve. Second, M should approximate A sufficiently well so that iteration converges rapidly, say $M \approx A$ and $R \approx 0$. Ideally, one would choose $M = A$, in which case convergence is achieved in one iteration. However, solving systems with this M would be as difficult as solving the original problem. The goal of preconditioning is therefore to construct a matrix M that approximates A well while it is computationally cheap to invert or solve.

Exercise 2.5. Another Gauss–Seidel method can be derived in a backward manner. The standard (forward) Gauss–Seidel method is written as

$$x_{m+1} = (D - E)^{-1}Fx_m + (D - E)^{-1}b.$$

Now assume that the approximation part of A (the splitting matrix) is chosen as the upper triangular part $D - F$, while the error matrix is $-E$. The iteration then becomes

$$x_{m+1} = (D - F)^{-1}Ex_m + (D - F)^{-1}b$$

which requires backward substitution to invert $(D - F)$.

The *symmetric Gauss–Seidel* method is obtained by first applying the forward iteration to move from x_m to $x_{m+1/2}$, followed by a backward iteration to move from $x_{m+1/2}$ to x_{m+1} . Show that the symmetric Gauss–Seidel iteration can be written as

$$(D - F)x_{m+1} = E(D - E)^{-1}Fx_m + D(D - E)^{-1}b.$$

Reformulate this expression and write the method in the form $Mx_{m+1} = Rx_m + b$ where

$$M = (D - E)D^{-1}(D - F), \quad R = (D - E)D^{-1}E(D - E)^{-1}F = ED^{-1}F.$$

Show that M is symmetric if A is. Show that M and R satisfy $A = M - R$ and conclude that the symmetric Gauss–Seidel method has the standard form (2.3).

The matrix $M = (D - E)D^{-1}(D - F)$ is also called the *symmetric Gauss–Seidel preconditioner*. It can be used as a preconditioner for other iterative methods designed for symmetric systems. Similarly, the *Gauss–Seidel preconditioner* is $M = D - E$, and the *Jacobi preconditioner* is $M = D$.

Exercise 2.6. Derive the backward SOR iteration from the backward Gauss–Seidel method. Then use the forward and backward SOR methods in the same way discussed in Exercise 2.5 to obtain the Symmetric SOR (SSOR) method. *Hint:* the SSOR method has the form $Mx_{m+1} = Rx_m + b$ where

$$M = \frac{1}{w(2 - \omega)}(D - \omega E)D^{-1}(D - \omega F), \quad R = \frac{1}{w(2 - \omega)}((1 - \omega)D + \omega E)D^{-1}((1 - \omega)D + \omega F).$$

If A is symmetric then M is also symmetric.

The matrix M of the SSOR iteration is also called *SSOR preconditioner*. The case $\omega = 1$ reduces to symmetric Gauss–Seidel preconditioner. We will come back to preconditioned systems in Section 4.

2.5 Polynomial representation

Since other classical iterative methods are obtained by applying the Richardson method on a preconditioned form of the system, here we consider the Richardson iteration

$$x_{m+1} = x_m + (b - Ax_m) = x_m + r_m.$$

Applying $-A$ and adding b to both sides, we obtain $b - Ax_{m+1} = b - Ax_m - Ar_m$ which simplifies to

$$\begin{aligned} r_{m+1} &= (I - A)r_m \\ &= (I - A)^{m+1}r_0 \\ &= p_{m+1}(A)r_0, \end{aligned} \tag{2.12}$$

where p_{m+1} is a polynomial of degree $m + 1$ satisfying $p_{m+1}(0) = 1$, given explicitly by

$$p_{m+1}(t) = (1 - t)^{m+1}.$$

In terms of the error $e_m = x - x_m$, this relation can be written as

$$e_{m+1} = p_{m+1}(A)e_m.$$

The second identity in (2.12) shows that the residuals can be expressed as powers of A times the initial residual. This observation will be fundamental for the development of Krylov subspace methods later in this text.

The third identity in (2.12) gives further insight into the convergence behavior of the iterative method by highlighting the role of the eigenvalues of A and the polynomial p_m . Assume that A has n linearly independent eigenvectors $v_1, v_2, \dots, v_n$ with corresponding eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$. Then the initial residual r_0 can be expanded in this eigenvector basis as

$$r_0 = \sum_{j=1}^n c_j v_j.$$

Using (2.12), we obtain

$$r_m = p_m(A)r_0 = \sum_{j=1}^n p_m(\lambda_j) c_j v_j.$$

This expression shows that each eigencomponent of the residual is scaled by the factor $p_m(\lambda_j)$. Hence, the convergence depends on how well the polynomial p_m damps the components of the initial error r_0 . For example, if all eigenvalues λ_j of A are close to 1 (that is, if A is close to the identity matrix), then $p_m(\lambda_j) = (1 - \lambda_j)^m \approx 0$, so the residual decays rapidly and the iteration converges quickly.

Jacobi and Gauss–Seidel methods often improve the convergence compared to Richardson method because the polynomial values $p_m(\lambda_j(M^{-1}A))$ annihilate the components of the initial residual more effectively.

3 Krylov subspace methods

A conclusion from Subsection 2.5 is that efficient iterative methods are those for which the associated polynomial p_m provides strong damping over the spectrum of the (preconditioned) matrix. Another idea is that instead of using only the latest iterate, one can store previous approximations x_m and attempt to recombine them to obtain a better solution. To formalize this idea, we first identify the subspace in which the successive approximations lie.

Definition 3.1. The subspace

$$\mathcal{K}_m(A; v) = \text{span}\{v, Av, A^2v, \dots, A^{m-1}v\}$$

is called the *Krylov subspace* of dimension m generated by the matrix A and the vector v .

Let us see why such subspaces are important. By repeatedly applying the Richardson iteration (2.5), we obtain

$$\begin{aligned} x_{m+1} &= x_0 + r_0 + r_1 + \dots + r_m = x_0 + \sum_{j=0}^m (I - A)^j r_0 \\ &\in x_0 + \text{span}\{r_0, Ar_0, A^2r_0, \dots, A^m r_0\} \equiv x_0 + \mathcal{K}_{m+1}(A; r_0) \end{aligned}$$

which means that the Richardson iteration produces approximations x_m such that $x_m - x_0$ lie in Krylov subspaces of increasing dimension. This also shows that we can write

$$x_{m+1} = x_0 + q_m(A)r_0$$

where q_m is a polynomial of degree m ; in this case it is $q_m(t) = \sum_{j=0}^m (1-t)^j$. We can also write

$$r_{m+1} = b - Ax_{m+1} = (I - Aq_m(A))r_0 = p_{m+1}(A)r_0$$

as we observed earlier in (2.12). Indeed $p_{m+1}(t) = 1 - tq_m(t)$.

The central idea of Krylov subspace methods is to select, at each iteration, an *optimal* (in an appropriate sense) approximation from the Krylov subspace. This leads to polynomials with improved damping properties and results in faster convergence and better performance for a wider class of coefficient matrices. In another point of view, all Krylov subspace methods can be interpreted as constructing polynomials p_m such that

$$p_m(\lambda_j) \approx 0.$$

Thus, solving $Ax = b$ becomes equivalent to a problem of polynomial approximation over the spectrum of A . Moreover, preconditioning can be incorporated to further enhance efficiency, either in general settings or for specific applications.

We will also see that Krylov subspace methods projects the original problem of dimension n onto a subspace of dimension $m \ll n$. The projected problem is then solved using standard techniques, and the solution is lifted back to obtain an approximation of the original problem.

3.1 Classification of methods

The Krylov subspace methods aim to compute an optimal approximation $x_m \in x_0 + \mathcal{K}_m(A; r_0)$ and can be classified into the following categories:

1. **Galerkin methods:** Determine $x_m \in x_0 + \mathcal{K}_m(A; r_0)$ such that the residual is orthogonal to the current Krylov subspace,

$$b - Ax_m \perp \mathcal{K}_m(A; r_0).$$

2. **Minimum residual methods:** Determine $x_m \in x_0 + \mathcal{K}_m(A; r_0)$ such that the Euclidean norm of the residual, $\|b - Ax_m\|_2$, is minimized.
3. **Petrov–Galerkin methods:** Determine $x_m \in x_0 + \mathcal{K}_m(A; r_0)$ such that the residual $b - Ax_m$ is orthogonal to another appropriately chosen m -dimensional subspace.

To construct approximations corresponding to the different approaches, we require a suitable basis for the Krylov subspace, one that can be extended in a stable and meaningful way as the dimension increases. The standard (power) basis,

$$\{r_0, Ar_0, \dots, A^{m-1}r_0\},$$

is the first choice. However, this basis is not well suited for numerical computations. As j increases, the vectors $A^j r_0$ tend to the dominant eigenvector of A (power method!), which causes the basis vectors to become nearly linearly dependent in finite precision arithmetic. This is analogous to the ill-conditioning of the monomial basis $\{1, x, \dots, x^{m-1}\}$ for the polynomial space $\mathbb{P}_m$ (Vandermonde matrix!).

A natural approach is to construct an orthogonal basis for the Krylov subspace. We should also avoid first forming the nonorthogonal and ill-conditioned set of vectors and then orthogonalizing it. To address this, Arnoldi proposed a numerically stable algorithm (in most cases!) that constructs an orthogonal basis incrementally from dimension 1 to m , in a manner similar to the Gram–Schmidt process [1, 1951].

3.2 Arnoldi algorithm

Given $A \in \mathbb{R}^{n \times n}$, a nonzero vector r_0 , and an integer $m \leq n$, the goal is to construct an orthonormal basis $\{v_1, v_2, \dots, v_m\}$ for the Krylov subspace $\mathcal{K}_m(A; r_0)$. We start by setting

$$v_1 = \frac{r_0}{\|r_0\|_2}.$$

Next, we compute Av_1 , orthogonalize it against v_1 , and normalize the result to obtain v_2 . More generally, suppose that we have already constructed an orthonormal basis $\{v_1, \dots, v_k\}$ for $\mathcal{K}_k(A; r_0)$. To extend this basis, we compute $w = Av_k$, and orthonormalize w with respect to $\{v_1, \dots, v_k\}$ by removing its components from this set:

$$w \leftarrow w - \sum_{j=1}^k (v_j^T w) v_j.$$

The resulting vector, after normalization, defines v_{k+1} . We continue to compute v_m , and we also compute v_{m+1} . The orthonormalization can be done in several ways, but in practice the modified Gram–Schmidt procedure is most commonly used. In Algorithm 2 we also store coefficients $h_{j,k} = v_j^T w$, $j \leq k+1$ and form an upper Hessenberg matrix $\tilde{H}_m = (h_{j,k})$ of size $(m+1) \times m$:

$$\tilde{H}_m = \begin{bmatrix} h_{1,1} & h_{1,2} & h_{1,3} & \cdots & h_{1,m} \\ h_{2,1} & h_{2,2} & h_{2,3} & \cdots & h_{2,m} \\ & h_{1,2} & h_{1,3} & \cdots & h_{1,m} \\ & & \ddots & \ddots & \vdots \\ & 0 & & & h_{m,m} \\ & & & & h_{m+1,m} \end{bmatrix} \in \mathbb{R}^{(m+1) \times m}.$$

Algorithm 2: Arnoldi (Modified Gram–Schmidt version)

Input: $A \in \mathbb{R}^{n \times n}$; $r_0 \in \mathbb{R}^n$; dimension $m \leq n$

Output: Orthonormal basis $\{v_1, \dots, v_{m+1}\}$ and Hessenberg matrix $\tilde{H}_m \in \mathbb{R}^{(m+1) \times m}$

Set $v_1 = r_0 / \|r_0\|_2$

for $k = 1$ **to** m **do**

$w = Av_k$

for $j = 1$ **to** k **do**

$h_{j,k} = v_j^T w$

$w = w - h_{j,k} v_j$

end for

$h_{k+1,k} = \|w\|_2$

if $h_{k+1,k} = 0$ **then** Break

$v_{k+1} = w / h_{k+1,k}$

end for

If the process does not terminate before m steps (i.e., $w \neq 0$ up to rounding errors), the Arnoldi procedure produces an orthonormal basis for $\mathcal{K}_m(A; r_0)$. This can be shown by induction. The construction starts with $v_1 = r_0 / \|r_0\|_2$, so that $\text{span}\{v_1\} = \text{span}\{r_0\} = \mathcal{K}_1(A; r_0)$. Assume now that $\text{span}\{v_1, \dots, v_k\} = \mathcal{K}_k(A; r_0)$. In the next step, Arnoldi computes $w = Av_k$. Since $v_k \in \mathcal{K}_k(A; r_0)$, it follows that $w = Av_k \in \mathcal{K}_{k+1}(A; r_0)$. The vector w is then orthogonalized against $\{v_1, \dots, v_k\}$, and after normalization we obtain v_{k+1} . Because orthogonalization does not change the span, we have $v_{k+1} \in \mathcal{K}_{k+1}(A; r_0)$ and therefore $\text{span}\{v_1, \dots, v_k, v_{k+1}\} = \mathcal{K}_{k+1}(A; r_0)$. Now we summarize some important relations which can be derived from the Arnoldi algorithm.

Theorem 3.1. Assume that matrices V_m , V_{m+1} and H_m are formed as below:

$$V_m = [v_1, v_2, \dots, v_m],$$

$$V_{m+1} = [v_1, v_2, \dots, v_m, v_{m+1}],$$

$$H_m = \text{the } m \times m \text{ Hessenberg matrix obtained by removing the last row of } \tilde{H}_m.$$

Then we have the following conclusions:

1. It holds that

$$AV_m = V_{m+1}\tilde{H}_m \quad (3.1)$$

2. Arnoldi decomposition:

$$AV_m = V_m H_m + \alpha v_{m+1} \hat{e}_m^T \quad (3.2)$$

where $\alpha = h_{m+1,m}$, and $\hat{e}_m = [0, \dots, 0, 1]^T$ is the m -th standard basis of $\mathbb{R}^m$.

3. Reduced QR factorization of the Krylov matrix:

$$K_m = V_m R_m \quad (3.3)$$

where $K_m = [r_0, Ar_0, \dots, A^{m-1}r_0]$. (Here V_m is the reduced Q factor.)

4. It holds that

$$V_m^T AV_m = H_m \quad (3.4)$$

Proof. 1. It is enough to show that both sides of (3.1) have the same columns. The k -th columns of AV_m is Av_k and the k -th column $V_{m+1}\tilde{H}_m$ is $v_1 h_{1k} + v_2 h_{2k} + \dots + v_{k+1} h_{k+1,k}$. This is an obvious relation from Algorithm 2.

2. The last row of $\tilde{H}_m$ is zero except for its final entry, $h_{m+1,m} =: \alpha$, and can therefore be written as $\alpha \hat{e}_m^T$. It follows that

$$V_{m+1}\tilde{H}_m = [V_m, v_{m+1}] \begin{bmatrix} H_m \\ \alpha \hat{e}_m^T \end{bmatrix} = V_m H_m + \alpha v_{m+1} \hat{e}_m^T.$$

3. Since the columns of V_m form an orthonormal basis of the same space spanned by the columns of K_m , each column of K_m can be written as a linear combination of the columns of V_m . Hence there exists a matrix R_m such that $K_m = V_m R_m$. Moreover, the Arnoldi process constructs the vectors $v_1, \dots, v_m$ by orthogonalizing the Krylov vectors in order. Therefore, the j -th column of K_m can only involve the first j Arnoldi vectors. This means that $r_{ij} = 0$ for $i > j$.

4. This follows directly from (3.2) by left-multiplying with V_m^T . $\square$

Remark 3.1. Similar to the modified Gram–Schmidt process, the Arnoldi algorithm may suffer from loss of orthogonality when the vectors involved are nearly linearly dependent. At the k -th step, the computed vector v_{k+1} may keep a non-negligible component in the span of $\{v_1, \dots, v_k\}$ if the matrix $[v_1, \dots, v_k, Av_k]$ is ill-conditioned, i.e., if Av_k is nearly contained in $\text{span}\{v_1, \dots, v_k\}$. For this reason, more numerically stable variants of the Arnoldi process have been developed. In particular, Walker proposed an alternative formulation based on Householder reflections [19, 1988].

Remark 3.2. Sometimes the Arnoldi algorithm is applied with a preconditioner M in order to solve the preconditioned system

$$M^{-1}Ax = M^{-1}b.$$

In this case, Algorithm 2 is applied to the matrix $M^{-1}A$ with initial vector $M^{-1}r_0$. Only two modifications are needed in the algorithm. First, the initial Arnoldi vector is now

defined by

$$v_1 = \frac{z_0}{\|z_0\|_2},$$

where z_0 is obtained by solving

$$Mz_0 = r_0.$$

Second, inside the Arnoldi loop, the vector $\hat{v}$ is no longer computed as Av_k . Instead, one first computes Av_k and then solves

$$Mw = Av_k.$$

We leave it as an exercise to write the preconditioned Arnoldi algorithm.

3.3 Lanczos algorithm

If A is *symmetric*, then it follows from (3.4) that H_m is also symmetric. Since H_m is upper Hessenberg, symmetry implies that it is in fact tridiagonal. This special case of the Arnoldi process is known as the *Lanczos method*. The Lanczos relation then becomes the three-term recursion

$$h_{k+1,k}v_{k+1} = Av_k - h_{k-1,k}v_{k-1} - h_{k,k}v_k,$$

where $h_{k+1,k} = h_{k-1,k}$. For simplicity we use new notations $\beta_k = h_{k+1,k}$, $\alpha_k = h_{k,k}$, and

$$T_m = H_m = \begin{bmatrix} \alpha_1 & \beta_1 & & 0 \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{m-1} \\ 0 & & \beta_{m-1} & \alpha_m \end{bmatrix}.$$

The algorithm (simplification of the Arnoldi algorithm 2 in symmetric case) is given below.

Algorithm 3: Symmetric Lanczos Algorithm

Input: Symmetric matrix $A \in \mathbb{R}^{n \times n}$; $r_0 \in \mathbb{R}^n$; dimension $m \leq n$

Output: Orthonormal basis $\{v_1, \dots, v_{m+1}\}$ and the entries α_j and β_j of tridiagonal matrix T_m

Set $v_0 = 0$, $\beta_0 = 0$, $v_1 = r_0/\|r_0\|_2$

for $k = 1$ **to** m **do**

$w = Av_k - \beta_{k-1}v_{k-1}$

$\alpha_k = v_k^T w$

$w = w - \alpha_k v_k$

$\beta_k = \|w\|_2$

if $\beta_k = 0$ **then** Break

$v_{k+1} = w/\beta_k$

end for

Clearly, all relations in Theorem 3.1 stated for the Arnoldi method, hold for the case of the symmetric Lanczos method by replacing H by T .

The Lanczos algorithm breaks down when any of the β_k equal 0. This is not a bad situation in solving linear systems and eigenvalue problems. This actually means that we have obtained an exact solution.

Exercise 3.1. For symmetric systems, usually a two-sided preconditioned matrix $L^{-1}AL^{-T}$ is used which corresponds to system

$$L^{-1}A\tilde{x} = L^{-1}b, \quad \tilde{x} = L^{-T}x.$$

Modify the Lanczos algorithm for the above preconditioned system.

3.4 Galerkin methods

For solving the linear system $Ax = b$, we start with the initial guess x_0 . In the Galerkin approach, the m -th iterate $x_m \in x_0 + \mathcal{K}_m(A; r_0)$ is chosen such that the residual satisfies

$$b - Ax_m \perp \mathcal{K}_m(A; r_0).$$

Since the columns of V_m form an orthonormal basis of $\mathcal{K}_m(A; r_0)$, this condition is equivalent to

$$V_m^T(b - Ax_m) = 0.$$

Since $x_m \in x_0 + \mathcal{K}_m(A; r_0)$, there exists a vector $y_m \in \mathbb{R}^m$ such that $x_m = x_0 + V_m y_m$. On the other hand, $r_0 = \|r_0\|_2 v_1$, so we get $V_m^T r_0 = \|r_0\|_2 \hat{e}_1$. Substituting into the Galerkin condition yields

$$V_m^T A V_m y_m = \|r_0\|_2 \hat{e}_1.$$

This can be interpreted as a projection of the large system $Ax = b$ onto the small Krylov subspace $\mathcal{K}_m(A; r_0)$. From (3.4), we have $V_m^T A V_m = H_m$ and therefore the problem reduces to solving the $m \times m$ Hessenberg system

$$H_m y_m = \|r_0\|_2 \hat{e}_1, \tag{3.5}$$

after which the approximate solution is recovered as $x_m = x_0 + V_m y_m$. We call this method the *full Arnoldi-Galerkin method* (in short *full ArGal*). The system (3.5) can be solved efficiently. For example, we can use Givens rotations to reduce H_m to triangular form in $\mathcal{O}(m^2)$ flops, then use backward substitution at a cost of $\mathcal{O}(m^2)$ for solving the triangular system.

The residual r_m plays an important role in monitoring the accuracy of the approximation. It is not necessary to compute it explicitly as $b - Ax_m$. Instead, see the following exercise.

Exercise 3.2. Using Arnoldi relations and equation (3.5), show that

$$r_m = b - Ax_m = -h_{m+1,m} (\hat{e}_m^T y_m) v_{m+1}, \tag{3.6}$$

and therefore

$$\|r_m\|_2 = h_{m+1,m} |\hat{e}_m^T y_m| = h_{m+1,m} |y_m^{(m)}|, \tag{3.7}$$

where y_m is the solution of (3.5) and $y_m^{(m)}$ is its last element.

The relation (3.6) shows that the residual r_m is in direction v_{m+1} . This also follows directly from the Galerkin condition $r_m \perp \mathcal{K}_m(A; r_0)$, which implies that r_m is orthogonal to the basis

$\{v_1, \dots, v_m\}$. Hence, r_m must lie in the direction of v_{m+1} . Another direct consequence is that, in Galerkin methods, the successive residuals are mutually orthogonal, i.e.,

$$r_j^T r_k = 0, \quad j \neq k.$$

This follows from the fact that vectors $\{v_j\}$ form an orthonormal set.

Exercise 3.3. Prove that if A is positive definite, the energy norm $\|x - x_m\|_A$ is minimal over subspace $\mathcal{K}_m(A; r_0)$. In other words, x_m is the best approximation for x in the A -norm. Here $\|x\|_A = \sqrt{x^T A x}$. Now, it should be more clear why this approach is called Galerkin method.

To solve $Ax = b$, some approaches can be employed. We explore a couple of approaches in the following.

Full (unrestarted) method. Starting with $m = 1$ and the initial guess x_0 , the Krylov subspace dimension is increased at each iteration. At step $m \geq 1$, we construct an orthonormal basis V_m of the Krylov subspace $\mathcal{K}_m(A; r_0)$ using the Arnoldi process, and compute the approximation

$$x_m = x_0 + V_m y_m,$$

where y_m solves the reduced system (3.5). The algorithm is given below.

Algorithm 4: Full Arnoldi–Galerkin (ArGal) method.

Input: $A \in \mathbb{R}^{n \times n}$; vector $b \in \mathbb{R}^n$; initial guess x_0 ; tolerance ε ; maximum iteration m_{max} .
Output: Number of iterations m_{iter} ; final solution $x_{m_{iter}}$; relative residual norm $relres$ of final solution; indicator $flag$.

Set $r_0 = b - Ax_0$, $\rho_0 = \|r_0\|_2$, $v_1 = r_0/\rho_0$, $relres = 1$, $flag = 0$, $m = 1$
while $relres > \varepsilon$ **and** $m \leq m_{max}$ **do**
 % Arnoldi
 $w = Av_m$
 for $j = 1$ **to** m **do**
 $h_{j,m} = v_j^T w$
 $w = w - h_{j,m} v_j$
 end for
 $h_{m+1,m} = \|w\|_2$
 if $h_{m+1,m} = 0$ **then** Break
 $v_{m+1} = w/h_{m+1,m}$
 Update matrices H_m and V_m

 % Galerkin
 Solve $m \times m$ system $H_m y = \|r_0\|_2 \hat{e}_1$ for y_m (update the previous decomposition of H_{m-1})
 Compute the solution vector $x_m = x_0 + V_m y_m$
 Compute $relres = h_{m+1,m} |y^{(m)}| / \|b\|_2$
 Set $m = m + 1$
end while
Set $m_{iter} = m - 1$
if $m_{iter} = m_{max}$ **then** $flag = 1$

For a prescribed tolerance ε , the iteration is terminated if the residual $r_m = b - Ax_m$

satisfies the relative error

$$\frac{\|r_m\|_2}{\|b\|_2} \leq \varepsilon.$$

Otherwise, the Krylov subspace dimension is increased to $m + 1$ and the process continues.

Here and in all algorithms in the sequel, the output parameter *flag* indicates whether the prescribed accuracy was attained before reaching the maximum number of iterations:

$$flag = \begin{cases} 0, & \text{desired accuracy achieved,} \\ 1, & \text{maximum iterations reached.} \end{cases}$$

In this algorithm, the successive solutions $x_1, x_2, \dots, x_m, \dots$ lie in nested affine spaces

$$x_0 + \mathcal{K}_1(A; r_0) \subset x_0 + \mathcal{K}_2(A; r_0) \subset \dots \subset x_0 + \mathcal{K}_m(A; r_0) \subset \dots$$

At each iteration, there is no need to reconstruct the Arnoldi matrices from scratch; instead, they can be updated from the previous step. See Exercise 3.10 in Section 3.8 in the following. This means that x_m carries the information from all previous iterations.

Theorem 3.2 (Happy breakdown). If the Arnoldi algorithm breaks down at an intermediate step m , then the iterate x_m of the full Arnoldi-Galerkin algorithm is the exact solution of system $Ax = b$.

Proof. The Arnoldi Algorithm 2 breaks down at step m if $h_{m+1,m} = 0$. But this means that $\|r_m\|_2 = 0$ according to (3.7). $\square$

However, sometimes convergence may be slow unless an effective preconditioner is used. Sometimes it is useful to restart the algorithm with an updated residual.

Restarting. Each iteration of the Arnoldi process requires the storage of one additional basis vector. Therefore, if many iterations are needed to achieve a prescribed accuracy with the full Arnoldi-Galerkin method, the storage and computational costs may become very large. To overcome this, one can use a *restarted* version of the method, in which the Arnoldi process is restarted every k iterations, using the most recent iterate as the new initial vector for the next Arnoldi cycle. In this way, the storage requirement is limited to $\mathcal{O}(k)$ vectors.

Each block of k iterations is called a *cycle*. Starting from the initial guess x_0 , the first cycle computes the approximations $x_1, \dots, x_k$ by increasing the dimension of the Krylov subspace from 1 to k . In particular x_k is the solution at the end of first cycle. Now, we reset the dimension to one and start the second cycle with the initial guess x_k (instead of x_0) and residual

$$r_k = b - Ax_k,$$

as the starting vector (instead of r_0) for generating new nested Krylov subspaces of dimensions 1 to k to produce approximate solutions $x_{k+1}, \dots, x_{2k}$. The process continues in the same manner, with each cycle using the final approximation from the previous cycle as its initial guess. Convergence may occur at an intermediate iteration within a cycle when the relative residual norm falls below ε . In this case, the maximum number of iteration $m_{max} = k$ does not achieved ($m_{iter} < k$) and the loop over that cycle is stopped. The restarted algorithm is given below.

In the restarted approach, the approximate solution is refined by repeatedly constructing new Krylov subspaces using the most recent residual vector. Although restarting significantly reduces the storage and computational costs, it may affect the convergence. Recall that in Krylov subspace methods the residual has the form $r_m = p_m(A)r_0$, where the polynomial p_m is expected to damp efficiently the components of r_0 associated with the spectrum of A . When the method is restarted, the Krylov subspace and the polynomial approximation process are rebuilt from the new residual and the spectral information accumulated during the previous cycle is ignored. This may lead, in some situations, to stagnation or slow convergence.

Algorithm 5: A restarted Arnoldi–Galerkin algorithm (ArGal(k) method)

Input: $A \in \mathbb{R}^{n \times n}$; vector b ; initial guess x_0 ; tolerance ε ; restart number k , maximum number of cycles j_{max} .

Output: Number of cycles j_{iter} ; final solution x ; relative residual norm $relres$ of final solution; indicator $flag$.

Set $j = 0$, $flag = 0$

while $j < j_{max}$ **do**

 Run the full Arnoldi–Galerkin algorithm 4 with inputs A , b , x_0 , ε and $m_{max} = k$ and outputs x and m_{iter}

if $m_{iter} < k$ **then** *Break*

 Set $x_0 = x$ and $j = j + 1$

end while

Set $j_{iter} = j$

if $j_{iter} = j_{max}$ **then** $flag = 1$

Other restarting strategies are also possible. For example, one may repeatedly restart with a fixed Krylov subspace dimension using the latest residual, or gradually increase the subspace dimension while using the previous solution as the new initial guess. We do not discuss these variants further here.

In both algorithms above, if A is *symmetric*, then the resulting algorithms are known as *Lanczos–Galerkin methods* [10, 1952]. In this case the symmetric tridiagonal system

$$T_m y_m = \|r_0\|_2 \hat{e}_1$$

can be efficiently solved by LU or QR factorization. In the non-restarted version, the reduced solution y_m can be updated recursively from the previous iterate y_{m-1} . Consequently, the approximation x_m can be written in the form

$$x_m = x_{m-1} + \text{correction term.}$$

and we do not need to store Lanczos matrices but vectors. This recursive structure is one of the main advantages of the full method. In contrast, this is generally not possible for restarted algorithm unless inside the cycles. However, the purpose of restarting is to keep the dimension m small while improving the approximation over successive cycles.

Exercise 3.4. Assume that A is a symmetric positive definite matrix. Write the Lanczos version of Algorithm 4. Use the Cholesky factorization $T_m = L_m L_m^T$ where L_m is the lower

bidiagonal matrix

$$L_m = \begin{bmatrix} \theta_1 & & & & \\ \ell_1 & \theta_2 & & & \\ & \ell_2 & \theta_3 & & \\ & & \ddots & \ddots & \\ & & & \ell_{m-1} & \theta_m \end{bmatrix}.$$

Derive the recursive formulas for computing the entries ℓ_j and θ_j . Next, express the solution y_m of the system $T_m y_m = \|r_0\|_2 \hat{e}_1$ as an update of the previous solution y_{m-1} of $T_{m-1} y_{m-1} = \|r_0\|_2 \hat{e}_1$. Finally, incorporate all of these recursions into a complete algorithm, and call it the *Lanczos–Galerkin algorithm* for solving the symmetric positive definite system $Ax = b$.

The Lanczos–Galerkin algorithm for symmetric positive definite systems is identical with the *Conjugate Gradient* (CG) method [8, 1952], as we will detail it in Section 3.6. In this case CG provides a more efficient algorithm compared to the Lanczos algorithm.

If A is symmetric but *indefinite*, Cholesky is no longer applicable. One may use $T_m = L_m D_m L_m^T$ factorization where

$$L_m = \begin{bmatrix} 1 & & & & \\ \ell_1 & 1 & & & \\ & \ell_2 & 1 & & \\ & & \ddots & \ddots & \\ & & & \ell_{m-1} & 1 \end{bmatrix}, \quad D_m = \text{diag}([\theta_1, \dots, \theta_m]).$$

The recursions are $\theta_1 = \alpha_1$ and for $j = 1, \dots, m-1$,

$$\ell_j = \frac{\beta_j}{\theta_j}, \quad \theta_{j+1} = \alpha_{j+1} - \ell_j^2 \theta_j.$$

Then the system $T_m y_m = \|r_0\|_2 \hat{e}_1$ is solved by $L_m e_m L_m^T y_m = \|r_0\|_2 \hat{e}_1$. This requires forward substitution, diagonal scaling, and backward substitution. The solution y_m can be obtained recursively from the previous solution y_{m-1} . However, there is an important issue: if some μ_j is very small, the factorization becomes unstable or breaks down. This corresponds to the fact that the Galerkin system may become nearly singular.

A more stable algorithm is to solve the Galerkin system using QR factorization of T_m . One can use Givens rotations to eliminate the under diagonal elements $\beta_1, \dots, \beta_{m-1}$ and compute a QR factorization

$$T_m = Q_m R_m,$$

where Q_m is orthogonal and R_m is upper triangular. Then

$$R_m y_m = \|r_0\|_2 g_m, \quad g_m = Q_m^T \hat{e}_1. \quad (3.8)$$

Since T_m is tridiagonal, the matrix R_m has an upper triangular (more precisely, upper tridiagonal) structure. This makes it possible to derive three-term recursions for updating the

approximation x_m from the previous iterate x_{m-1} without storing the Lanczos matrices T_m and V_m . The implementation details are given in Appendix A.1. However, it is preferable to first study the MINRES implementation in Section 3.8, since the two procedures share some similarities.

Exercise 3.5. In 1847, Cauchy developed an iterative method for solving (nonlinear) systems of equations which is now known as **steepest descent** method. At each step the algorithm updates the solution via

$$x_{j+1} = x_j + \alpha_j p_j$$

where p_j is a search direction and α_j is the step length (see also the next section). In steepest descent method the search direction is chosen to be $p_j = r_j$. The method for linear equation $Ax = b$ for a symmetric positive definite matrix A is given in Algorithm 6.

Algorithm 6: Steepest descent method for solving $Ax = b$

Input: Symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$; vector b ; initial guess x_0 ; tolerance ε ; maximum iteration j_{max} .
Output: Number of iterations j_{iter} ; final solution $x_{j_{iter}}$; relative residual norm $relres$ of final solution; indicator $flag$.

Set $r_0 = b - Ax_0$, $flag = 0$, $relres = 1$, $j = 0$
while $relres > \varepsilon$ **and** $j < j_{max}$ **do**
 $\alpha_j = (r_j^T r_j) / (r_j^T A r_j)$
 $x_{j+1} = x_j + \alpha_j r_j$
 $r_{j+1} = r_j - \alpha_j A r_j$
 $relres = \|r_{j+1}\|_2 / \|b\|_2$
 Set $j = j + 1$
end while
Set $j_{iter} = j$
if $j_{iter} = j_{max}$ **then** $flag = 1$

Show that the steepest descent method is an special case of restarted Arnoldi-Galerkin Algorithm 5 with restart number $k = 1$ (one-dimensional method).

Exercise 3.6. Assume that A is symmetric positive definite, and define the A -norm $\|x\|_A^2 = x^T A x$. Using the fact that $r_j = b - Ax_j = A(x - x_j)$, and other relations in the steepest descent algorithm show that

$$\|x - x_{j+1}\|_A^2 = \|x - x_j\|_A^2 - \alpha_j r_j^T r_j.$$

Now, show that $r_j^T r_{j+1} = 0$ and conclude from the above equality that

$$\|x - x_{j+1}\|_A^2 = \|x - x_j\|_A^2 \left(1 - \frac{r_j^T r_j}{r_j^T A r_j} \frac{r_j^T r_j}{r_j^T A^{-1} r_j} \right).$$

Now use the Kantorovich inequality (the proof is rather technical!)

$$1 \leq (u^T A u)(u^T A^{-1} u) \leq \frac{1}{4}(\kappa^{1/2} + \kappa^{-1/2})^2, \quad \forall u \text{ with } u^T u = 1,$$

to show that

$$\|x - x_j\|_A \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^j \|x - x_0\|_A \quad (3.9)$$

for the steepest descent method, where κ is the condition number of A in norm 2. What is your conclusion from the error bound (3.9)?

3.5 Conjugate Gradient method

The Conjugate Gradient (CG) method was developed independently by Hestenes and Stiefel and published in their joint paper [8] in 1952.

Assume that A is *symmetric positive definite* and consider the linear system $Ax = b$. The solution is equivalently the minimizer of the quadratic functional

$$\phi(x) = \frac{1}{2}x^T Ax - b^T x.$$

This naturally leads to iterative minimization methods. At each iteration, a new approximation x_{m+1} is obtained by correcting the previous iterate x_m along a suitable direction p_m :

$$x_{m+1} = x_m + \alpha_m p_m. \quad (3.10)$$

The vector p_m is called a *search direction*. Once the direction p_m has been chosen, the problem reduces to minimizing ϕ on line $x_m + \alpha p_m$. Equivalently, we seek the parameter α_m that minimizes $\phi(x_m + \alpha p_m)$ with respect to α . Differentiating with respect to α gives

$$0 = \frac{d}{d\alpha} \phi(x_m + \alpha p_m) = p_m^T (Ax_m + \alpha A p_m - b).$$

Using $r_m = b - Ax_m$, we obtain

$$\alpha_m = \frac{p_m^T r_m}{p_m^T A p_m}. \quad (3.11)$$

This optimal value α_m is called an *exact line search* at iteration m . Note that $\alpha_m = 0$ if and only if $p_m^T r_m = 0$. This situation must be avoided, since then $x_{m+1} = x_m$ and the iteration makes no progress. Therefore, the search direction p_m should not be orthogonal to the residual r_m , unless $r_m = 0$, in which case x_m is already the exact solution. Whenever $p_m^T r_m \neq 0$, the exact line search guarantees the strict decrease

$$\phi(x_{m+1}) < \phi(x_m).$$

In the steepest descent method (Exercise 3.5), the search direction is chosen simply as $p_m = r_m$. Since $r_m = b - Ax_m = -\nabla \phi(x_m)$, the vector p_m points in the direction of the steepest decrease of the functional ϕ at the point x_m . Although this is a natural choice, the method often performs poorly in practice because it lacks memory. At each iteration, only information from the current iterate x_m is used to compute x_{m+1} , while all information from previous iterations is ignored. Recall from Exercise 3.5 that the steepest descent method can be interpreted as a restarted one-dimensional Lanczos–Galerkin method. At each step, a new

one-dimensional Krylov subspace is constructed from the current residual, and therefore no nested sequence of growing Krylov subspaces is preserved.

The CG method can be viewed as a simple but highly effective modification of steepest descent that incorporates memory from previous iterations. As we will see later, CG is precisely the full (unrestarted) Lanczos–Galerkin method (Exercise 3.4) in which the Krylov subspace dimension m increases by one at each iteration.

Here, however, we derive the CG method independently of Krylov subspaces and the Lanczos process, but then we show the connection. Assume that x is the exact solution and define the error

$$e_m = x - x_m.$$

By the notation $x \perp y$ we mean that x and y are orthogonal, i.e., $x^T y = 0$, while $x \perp_A y$ means that x and y are A -orthogonal (or A -conjugate), i.e., $x^T A y = 0$. We start with the following Lemma.

Lemma 3.3. Let $x_{m+1} = x_m + \alpha_m p_m$ be obtained from an exact line search which means that α_m comes from (3.11). Then $r_{m+1} \perp p_m$ and $e_{m+1} \perp_A p_m$.

Proof. From (3.10) we obtain

$$r_{m+1} = r_m - \alpha_m A p_m. \quad (3.12)$$

Taking the inner product with p_m gives $r_{m+1}^T p_m = r_m^T p_m - \alpha_m p_m^T A p_m = 0$ because α_m is chosen by the exact line search (3.11). Since $A e_{m+1} = r_{m+1}$, we obtain $p_m^T A e_{m+1} = p_m^T r_{m+1} = 0$. $\square$

The above lemma states that the minimum of ϕ along the line $x_m + \alpha p_m$ is attained when the directional derivative vanishes, i.e., $\nabla \phi(x_{m+1}) \cdot p_m = -r_{m+1}^T p_m = 0$.

Repeated use of (3.10) gives

$$x_{m+1} = x_0 + \alpha_0 p_0 + \cdots + \alpha_m p_m, \quad (3.13)$$

which means that our approximation for x comes from affine space $x_0 + \mathcal{P}_{m+1}$ where

$$\mathcal{P}_{m+1} = \text{span}\{p_0, p_1, \dots, p_m\}.$$

Since $e_{m+1} = x - x_{m+1}$, we obtain

$$e_{m+1} = e_0 - (\alpha_0 p_0 + \cdots + \alpha_m p_m) = x - (x_0 + \alpha_0 p_0 + \cdots + \alpha_m p_m).$$

The CG method finds the best approximation to $e_0 = x - x_0$ in the A -norm over subspace $\mathcal{P}_{m+1}$. On the other words, x_{m+1} is (energy) projection of e_0 into subspace $\mathcal{P}_{m+1}$. That is, CG minimizes $\|e_{m+1}\|_A = \|x - p\|_A$, where $p \in x_0 + \mathcal{P}_{m+1}$ and $\|z\|_A = \sqrt{z^T A z}$. The best approximation theorem for inner product spaces (projection theorem) implies that the error must be A -orthogonal to the basis of the approximation space:

$$e_{m+1} \perp_A p_j, \quad j = 0, 1, \dots, m. \quad (3.14)$$

From Lemma 3.3, we already know that $e_{m+1} \perp_A p_m$. The following theorem gives sufficient conditions for having (3.14) for all values of $j \leq m$.

Theorem 3.4. If an iterative method $x_{m+1} = x_m + \alpha_m p_m$ employs search directions which are mutually A -conjugate:

$$p_k \perp_A p_j, \quad j \neq k,$$

and uses exact line searches (3.11), then the errors satisfy (3.14). Consequently,

$$e_{m+1} = \arg \min_{p \in \mathcal{P}_{m+1}} \|e_0 - p\|_A.$$

Proof. We prove (3.14) by induction on m . For $m = 0$, the statement follows directly from Lemma 3.3. Assume now that $e_m \perp_A p_j$ for $j = 0, \dots, m-1$. Since $e_{m+1} = e_m - \alpha_m p_m$, we obtain

$$e_{m+1}^T A p_j = e_m^T A p_j - \alpha_m p_m^T A p_j.$$

For $j = 0, \dots, m-1$, the induction hypothesis and the conjugacy condition imply that both terms vanish. For $j = m$, the result follows from Lemma 3.3. $\square$

The remaining question is how to construct the search directions p_j . In the steepest descent method, the search direction is chosen as $p_m = r_m$. However, these directions do not, in general, satisfy the A -conjugacy condition, and therefore the method does not enjoy the optimal approximation property described above.

In the CG method, the search directions are constructed by A -orthogonalizing the residual against the previous search directions. The first direction is chosen as $p_0 = r_0$. For $m \geq 1$, the next direction is defined by

$$p_m = r_m + \beta_{m-1} p_{m-1}. \quad (3.15)$$

The coefficient β_{m-1} is selected so that $p_m \perp_A p_{m-1}$. This condition yields

$$\beta_{m-1} = -\frac{r_m^T A p_{m-1}}{p_{m-1}^T A p_{m-1}}. \quad (3.16)$$

In fact, this choice guarantees that all search directions are mutually A -conjugate. See the following lemma.

Lemma 3.5. With p_m defined by (3.15) and (3.16), the residual vectors r_m are orthogonal to all previous search directions:

$$r_m^T p_j = 0, \quad j = 0, \dots, m-1, \quad (3.17)$$

and the search directions are mutually A -conjugate:

$$p_m^T A p_j = 0, \quad j = 0, 1, \dots, m-1. \quad (3.18)$$

Proof. We prove relations (3.17) and (3.18) jointly by induction. From Lemma 3.3, we know that r_m is orthogonal to the previous search direction p_{m-1} , and by construction p_m is A -conjugate to p_{m-1} . Thus both (3.17) and (3.18) hold for $m = 1$. Assume now that the relations are true for some $m \geq 1$. We prove them for $m+1$. Using (3.12), we obtain

$$r_{m+1}^T p_j = r_m^T p_j - \alpha_m p_m^T A p_j, \quad j = 0, \dots, m-1.$$

By the induction hypothesis, both terms on the right-hand side vanish, and therefore

$$r_{m+1}^T p_j = 0, \quad j = 0, \dots, m-1.$$

For $j = m$, the relation $r_{m+1}^T p_m = 0$ follows directly from Lemma 3.3. Hence (3.17) holds for $m+1$.

Next, we write for $j = 0, \dots, m-1$,

$$\begin{aligned} p_{m+1}^T A p_j &= r_{m+1}^T A p_j + \beta_m p_m^T A p_j && \text{using (3.15)} \\ &= r_{m+1}^T A p_j && \text{using induction hypothesis} \\ &= \alpha_j^{-1} r_{m+1}^T [r_j - r_{j+1}] && \text{using (3.12)} \\ &= \alpha_j^{-1} r_{m+1}^T [p_j - \beta_{j-1} p_{j-1} - p_{j+1} + \beta_j p_j] && \text{using (3.15)} \\ &= 0. \end{aligned}$$

This expression is zero because, using the induction hypothesis, we have already shown that $r_{m+1}^T p_j = 0$ for $j = 0, \dots, m-1$. Note that for $j = 0$, the term $p_j - \beta_{j-1} p_{j-1}$ appearing in the second last line should be interpreted simply as p_0 , since $r_0 = p_0$. Finally for $j = m$, by construction, we have $p_{m+1}^T A p_m = 0$. Hence (3.18) also holds for $m+1$. $\square$

Lemma 3.6. The residual vectors r_m are orthogonal to all previous residual vectors:

$$r_m^T r_j = 0, \quad j = 0, \dots, m-1. \quad (3.19)$$

Proof. Since $r_0 = p_0$, relation (3.17) gives $r_m^T r_0 = r_m^T p_0 = 0$. For $j = 1, \dots, m-1$, using (3.17) and (3.15) we obtain

$$0 = r_m^T p_j = r_m^T (r_j + \beta_{j-1} p_{j-1}) = r_m^T r_j,$$

which completes the proof. $\square$

We summarize the resulting CG algorithm below.

Algorithm 7: Conjugate Gradient (CG) method for solving $Ax = b$

Input: Symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$; vector $b \in \mathbb{R}^n$; initial vector x_0 ; tolerance ε ; maximum iteration m_{max} .

Output: Number of iterations m_{iter} ; final solution $x_{m_{iter}}$; relative residual norm $relres$ of final solution; indicator $flag$.

Set $r_0 = b - Ax_0$, $p_0 = r_0$, $relres = 1$, $flag = 0$, $m = 0$

while $relres > \varepsilon$ **and** $m < m_{max}$ **do**

 Compute $\alpha_m = (r_m^T r_m) / (p_m^T A p_m)$

 Update $x_{m+1} = x_m + \alpha_m p_m$

 Update $r_{m+1} = r_m - \alpha_m A p_m$

 Compute $\beta_m = (r_{m+1}^T r_{m+1}) / (r_m^T r_m)$

 Update $p_{m+1} = r_{m+1} + \beta_m p_m$

 Compute $relres = \|r_{m+1}\|_2 / \|b\|_2$

 Set $m = m + 1$

end while

Set $m_{iter} = m$

if $m_{iter} = m_{max}$ **then** $flag = 1$

In the algorithm, using the orthogonality properties mentioned above, the coefficients α_m and β_m are reformulated and simplified as follows:

$$\alpha_m = \frac{p_m^T r_m}{p_m^T A p_m} = \frac{r_m^T r_m}{p_m^T A p_m}, \quad \beta_m = -\frac{r_{m+1}^T A p_m}{p_m^T A p_m} = \frac{r_{m+1}^T r_{m+1}}{r_m^T r_m}.$$

For proofs replace $A p_m$ from (3.12) and p_m from (3.15) and use orthogonality relations. We leave it as an exercise.

Algorithm 7 requires only one matrix–vector product, two vector inner products, three scalar–vector products, and three vector updates per iteration. Thus, the total cost per iteration is

$$2 \cdot \text{nnz}(A) + 10n.$$

Note that, in the computation of β_m , the inner product $r_m^T r_m$ has already been computed in the previous iteration and therefore does not need to be computed again. It can also be used to compute the residual for no extra cost.

3.6 CG connection to Lanczos process

We now reveal the connection between the CG method and the symmetric Lanczos process. We start with the following interesting theorem.

Theorem 3.7. At the m -th step of the CG algorithm (assuming $r_j \neq 0$ for all j), we have

$$\mathcal{P}_{m+1} = \mathcal{R}_{m+1} = \mathcal{K}_{m+1}(A; r_0).$$

where $\mathcal{P}_{m+1} = \text{span}\{p_0, \dots, p_m\}$ and $\mathcal{R}_{m+1} = \text{span}\{r_0, \dots, r_m\}$.

Proof. The proof is by induction on m . For $m = 0$, the statement is trivial since $p_0 = r_0$. Assume now that the statement holds for some $m \geq 0$. We prove it for $m + 1$. We first show that

$$\mathcal{R}_{m+2} \subseteq \mathcal{K}_{m+2}(A; r_0).$$

By the induction hypothesis, it is sufficient to prove that $r_{m+1} \in \mathcal{K}_{m+2}(A; r_0)$. From (3.12), we have $r_{m+1} = r_m - \alpha_m A p_m$. Since $p_m \in \mathcal{K}_{m+1}(A; r_0)$ by the induction hypothesis, it follows that $A p_m \in \mathcal{K}_{m+2}(A; r_0)$. Moreover, $r_m \in \mathcal{K}_{m+1}(A; r_0) \subseteq \mathcal{K}_{m+2}(A; r_0)$. Hence $r_{m+1} \in \mathcal{K}_{m+2}(A; r_0)$.

Next, we show that

$$\mathcal{P}_{m+2} \subseteq \mathcal{R}_{m+2}.$$

By the induction hypothesis, we already know that $p_j \in \mathcal{R}_{m+1}$, for $j = 0, \dots, m$. Thus it remains only to show that $p_{m+1} \in \mathcal{R}_{m+2}$. This follows immediately from the recurrence relation (3.15), $p_{m+1} = r_{m+1} + \beta_m p_m$.

We have therefore established that the three spaces are nested. To finish the proof, it remains to show that these three spaces have the same dimension. Since $\mathcal{K}_{m+2}(A; r_0)$ is generated by at most $m + 2$ vectors, its dimension is at most $m + 2$. On the other hand, the vectors $p_0, \dots, p_{m+1}$ are mutually A -conjugate and nonzero. Because A is positive definite, A -conjugate nonzero vectors are linearly independent. Therefore, the dimension of $\mathcal{P}_{m+2}$ is

exactly $m + 2$, which means that all three spaces have the same dimension $m + 2$ and hence they are equal. $\square$

Corollary 3.8. The CG method is a Krylov subspace method with Galerkin approach for solving $Ax = b$ with a symmetric positive definite matrix A .

Proof. From equation (3.13) we observe that the CG solution x_{m+1} belongs to space $x_0 + \mathcal{P}_{m+1}$ which is equal to space $x_0 + \mathcal{K}_{m+1}(A; r_0)$. Besides, according to (3.14), this CG solution is obtained by requiring that $(x - x_{m+1}) \perp_A \mathcal{P}_{m+1}$ which is equivalent to $(b - Ax_{m+1}) \perp \mathcal{P}_{m+1} = \mathcal{K}_{m+1}(A; r_0)$. Thus CG is a Galerkin method. $\square$

This result shows that the solutions produced by the CG method are identical with those obtained from the full (unrestarted) Lanczos–Galerkin algorithm (see Exercise 3.4). In other words, the CG method can be derived directly from the Galerkin projection applied to the Lanczos process. The projected matrix T_{m+1} is symmetric tridiagonal and positive definite. The derivation then proceeds from the Cholesky factorization $T_{m+1} = L_{m+1}L_{m+1}^T$, and definition of the search direction in terms of Lanczos matrices. Up to a normalization factor, the search direction matrix $[p_0, \dots, p_m] = V_{m+1}L_{m+1}^{-T}$. The short recurrences of the CG method follow from the bidiagonal structure of L_{m+1} .

Remark 3.3. The scalars α_j and β_j in the CG method are different from the Lanczos coefficients that are often denoted by the same symbols. We also remark that the implementation of CG method as in Algorithm 7 is more memory efficient than its implementation via the Lanczos–Galerkin algorithm. For this reason, Algorithm 7 is usually the preferred algorithm in practice. The CG algorithm is just an efficient implementation of the Lanczos–Galerkin algorithm for symmetric positive definite systems. If the system matrix is symmetric but not positive definite, one must instead use the Lanczos–Galerkin algorithm, or the more efficient MINRES algorithm (see Section 3.8).

3.7 Convergence of the CG method

One remarkable property of the CG method is that the residual vectors are mutually orthogonal; see (3.19). This immediately leads to the following theorem.

Theorem 3.9. In the absence of rounding errors, the CG algorithm applied to an $n \times n$ symmetric positive definite system $Ax = b$ produces the exact solution in at most n iterations.

Proof. Suppose that after $n - 1$ iterations the exact solution has not yet been obtained. Then we compute the next solution x_n . The next residual vector $r_n \in \mathbb{R}^n$ is orthogonal to the previous residual vectors $r_0, r_1, \dots, r_{n-1} \in \mathbb{R}^n$. Since these vectors are mutually orthogonal and nonzero, they form a basis of $\mathbb{R}^n$. Therefore, r_n is orthogonal to a basis of $\mathbb{R}^n$, which implies that $r_n = 0$. Consequently, x_n is the exact solution. $\square$

One can also prove Theorem 3.9 using the fact that r_n is orthogonal to the search directions p_j for $j = 0, \dots, n - 1$. This finite termination property, however, is valid only in exact arithmetic. In practical computations, rounding errors destroy exact orthogonality, and therefore exact termination is generally not observed. Moreover, in applications the dimen-

sion n is usually very large, and we expect to obtain an accurate approximation in a number of iterations far smaller than n . For this reason, it is important to study the convergence behavior of the method at intermediate iterations and understand how convergence can be accelerated.

Every element of the Krylov subspace $\mathcal{K}_m(A; r_0) = \text{span}\{r_0, Ar_0, \dots, A^{m-1}r_0\}$ can be written in the form $q(A)r_0$, where $q \in \mathbb{P}_m$ is a polynomial of order at most m (degree at most $m-1$). Conversely, for every polynomial $q \in \mathbb{P}_m$, the vector $q(A)r_0$ belongs to $\mathcal{K}_m(A; r_0)$. Hence

$$\mathcal{K}_m(A; r_0) = \{q(A)r_0 : q \in \mathbb{P}_m\}.$$

The CG method computes an approximation $x_m \in x_0 + \mathcal{K}_m(A; r_0)$ by orthogonalization of the residual $b - Ax_m$ to the Krylov subspace $\mathcal{K}_m(A; r_0)$. It is equivalent to minimizing the error $e_m = x - x_m$ in the A -norm (See Exercise (3.3)). Since $x_m \in x_0 + \mathcal{K}_m(A; r_0)$, it has the representation $x_m = x_0 + q(A)r_0$, where $q \in \mathbb{P}_m$. Using the identity $r_0 = b - Ax_0 = A(x - x_0)$, we obtain

$$\begin{aligned} e_m &= x - x_m \\ &= x - x_0 - q(A)A(x - x_0) \\ &= (I - q(A)A)e_0 \\ &=: p(A)e_0, \end{aligned}$$

where $p(t) = 1 - q(t)t$ belongs to $\mathbb{P}_{m+1}$ and satisfies $p(0) = 1$. This leads to the following theorem.

Theorem 3.10. Let x_m be the m -th iterate of the CG method and let $e_m = x - x_m$ be the corresponding error. Then

$$\|e_m\|_A = \min_{p \in \mathbb{P}_{m+1}, p(0)=1} \|p(A)e_0\|_A, \quad (3.20)$$

where $\mathbb{P}_{m+1}$ denotes the space of polynomials of order at most $m+1$ (degree at most m).

Theorem 3.10 makes it possible to derive convergence bounds for the A -norm of the error in terms of the eigenvalues of A . Assume that the positive definite matrix A has eigenvalues $\lambda_1, \dots, \lambda_n$ and corresponding orthonormal eigenvectors $v_1, \dots, v_n$. The initial error can be expanded as

$$e_0 = c_1 v_1 + \dots + c_n v_n, \quad \|e_0\|_A^2 = e_0^T A e_0 = \sum_{j=1}^n c_j^2 \lambda_j.$$

Since $p(A)v_j = p(\lambda_j)v_j$, we obtain (try to prove it!)

$$\|p(A)e_0\|_A^2 = (p(A)e_0)^T A (p(A)e_0) = \sum_{j=1}^n c_j^2 \lambda_j p(\lambda_j)^2.$$

Hence the minimization problem becomes

$$\|e_m\|_A^2 = \min_{p \in \mathbb{P}_{m+1}, p(0)=1} \sum_{j=1}^n c_j^2 \lambda_j p(\lambda_j)^2,$$

To derive a term independent of coefficients c_j , we estimate

$$\sum_{j=1}^n c_j^2 \lambda_j p(\lambda_j)^2 \leq \max_{1 \leq j \leq n} p(\lambda_j)^2 \cdot \sum_{j=1}^n c_j^2 \lambda_j = \max_{1 \leq j \leq n} p(\lambda_j)^2 \cdot \|x\|_A^2.$$

This leads to the following theorem.

Theorem 3.11. Let x_m be the m -th iterate of the CG method and let $e_m = x - x_m$ be the corresponding error. Then

$$\frac{\|e_m\|_A}{\|e_0\|_A} \leq \left(\min_{p \in \mathbb{P}_{m+1}, p(0)=1} \max_{1 \leq j \leq n} |p(\lambda_j)| \right), \quad (3.21)$$

where λ_j are the eigenvalues of A .

The left-hand side of the error bound (3.21) is the relative error in the energy norm. This bound provides yet another proof of the finite termination property of the CG method. Taking $m = n$, we obtain

$$\frac{\|e_n\|_A}{\|e_0\|_A} \leq \left(\max_{1 \leq j \leq n} |p(\lambda_j)| \right)$$

for every polynomial $p \in \mathbb{P}_{n+1}$ satisfying $p(0) = 1$. Now choose

$$p(t) = c(t - \lambda_1) \cdots (t - \lambda_n),$$

where the constant c is chosen such that $p(0) = 1$. Then $p(\lambda_j) = 0$ for all $j = 1, \dots, n$, which implies that $\|e_n\|_A = 0$, and consequently $e_n = 0$ and $x = x_n$.

Exercise 3.7. Show that if A has $k < n$ distinct eigenvalues, then the CG method gives the exact solution after at most k iterations.

Thus, convergence may occur very rapidly when A has only a few distinct eigenvalues with high multiplicity. More generally, suppose that the eigenvalues of A are grouped into k tight clusters, where $k \ll n$. If we choose

$$p(t) = c(t - \mu_1) \cdots (t - \mu_k),$$

where $\mu_1, \dots, \mu_k$ are representative points of the clusters (for example, the cluster centers), then $p(\lambda_j)$ is small for all eigenvalues λ_j , since each eigenvalue lies close to one of the roots μ_i . Consequently, $\max_j |p(\lambda_j)|$ is small, and estimate (3.21) implies that the error norm $\|e_k\|_A$ is small.

Finally, by choosing p in the error bound (3.21) to be a suitably scaled Chebyshev polynomial, one can derive the following classical convergence estimate. To see the details of the proof refer for example to [13, 20, 2003].

Theorem 3.12. The residual of the CG iterate at step m satisfies

$$\|e_m\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^m \|e_0\|_2, \quad (3.22)$$

where $\kappa = \lambda_n/\lambda_1$ is the spectral condition number of A .

The estimate (3.22) is generally pessimistic, since its derivation relies on some crude upper bounds. Nevertheless, it clearly shows that the CG method converges more rapidly for well-conditioned matrices. Moreover, it motivates the use of suitable preconditioners to reduce the condition number and accelerate the convergence.

Exercise 3.8. Consider the linear system $Ax = b$, with starting vector x_0 , and assume that the CG method generates iterates x_m and residuals r_m for this system. Now consider the system $\tilde{A}\tilde{x} = \tilde{b}$ where $\tilde{A} = Q^T A Q$, $\tilde{x} = Q^T x$ and $\tilde{b} = Q^T b$, for an orthogonal matrix Q . Prove that the CG algorithm applied on the new system with $\tilde{x}_0 = Q^T x_0$ generates iterates $\tilde{x}_m = Q^T x_m$ and residuals $\tilde{r}_m = Q^T r_m$. In particular, $\|r_m\|_2 = \|\tilde{r}_m\|_2$.

Exercise 3.8 shows that an orthogonal transformation of the system leads to exactly the same CG iteration process. This means that to study the convergence behavior of the CG method, we can restrict ourselves to diagonal systems (apart from the effects of rounding errors).

3.8 Minimum residual methods

Up to here, we have focused on the Galerkin approach, which is particularly effective for positive definite systems. In this section, we turn to minimum residual methods, which are generally more robust and effective for indefinite systems. In this class of methods, we seek an approximation

$$x_m \in x_0 + \mathcal{K}_m(A; r_0)$$

such that the Euclidean norm of the residual, $\|b - Ax_m\|_2$, is minimized. As before, we write $x_m = x_0 + V_m y_m$ for a vector $y_m \in \mathbb{R}^m$. Since $b - Ax_0 = r_0 = \|r_0\|_2 v_1 = \rho V_{m+1} \hat{e}_1$ for $\rho_0 = \|r_0\|_2$, we obtain

$$\begin{aligned} \|b - Ax_m\|_2 &= \|\rho_0 V_{m+1} \hat{e}_1 - A V_m y_m\|_2 \\ &= \|\rho_0 V_{m+1} \hat{e}_1 - V_{m+1} \tilde{H}_m y_m\|_2 \\ &= \|\rho_0 \hat{e}_1 - \tilde{H}_m y_m\|_2, \end{aligned}$$

where we used the Arnoldi relation (3.1) and invariance property of Euclidean norm against orthogonal matrices. Thus, the original problem reduces to the least-squares problem

$$\min_{y \in \mathbb{R}^m} \|\rho_0 \hat{e}_1 - \tilde{H}_m y\|_2, \quad (3.23)$$

where $\tilde{H}_m$ is an upper Hessenberg matrix of size $(m+1) \times m$.

Exercise 3.9. Show that the condition that $\|b - Ax_m\|_2$ is minimized in minimum residual methods is equivalent to $r_m \perp A\mathcal{K}_m(A; r_0)$.

3.9 GMRES methods

The GMRES (Generalized Minimal RESidual) method, introduced by Yousef Saad and Martin H. Schultz [14, 1986] for solving nonsymmetric linear systems, and several of its variants are based on the minimum residual formulation above.

In GMRES, the reduced least-squares problem (3.23) is solved efficiently by computing a QR factorization of $\tilde{H}_m$ using Givens rotations at a cost of $\mathcal{O}(m^2)$ flops. Since $\tilde{H}_m$ is an upper Hessenberg matrix, its QR factorization

$$\tilde{H}_m = Q_m R_m$$

can be computed efficiently using only m successive Givens rotations $G_1, \dots, G_m$, which eliminate the subdiagonal entries $h_{j+1,j}$ for $j = 1, \dots, m$. The orthogonal factor is then given by $Q_m = G_m G_{m-1} \cdots G_1$, while R_m is an upper triangular matrix of size $(m+1) \times m$, whose last row is zero.

$$\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ & \times & \times & \times \\ & & \times & \times \\ & & & \times \end{bmatrix} = \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ & 0 & \times & \times \\ & & 0 & \times \\ & & & 0 \end{bmatrix}$$

$$\tilde{H}_m = [\tilde{Q}_m \quad q_m] \begin{bmatrix} \tilde{R}_m \\ 0 \end{bmatrix}$$

Assume that $Q_m = [\tilde{Q}_m \quad q_m]$ where $\tilde{Q}_m$ consists of the first m columns and q_m is the last column of Q_m . Also assume that $\tilde{R}_m$ is the $m \times m$ upper triangular matrix obtained by removing the last zero row of R_m . The least-squares solution y is then obtained by solving the upper triangular system

$$\tilde{R}_m y_m = \|r_0\|_2 \tilde{Q}_m^T \hat{e}_1. \quad (3.24)$$

The residual norm can be computed without explicitly forming the approximate solution x_m . Indeed,

$$\|r_m\|_2 = \|b - Ax_m\|_2 = \|r_0\|_2 \cdot \|q_m^T \hat{e}_1\|_2 = \|r_0\|_2 \cdot |\tau_m|, \quad (3.25)$$

where τ_m denotes the first component of q_m . In the sequel, we consider τ_m as the last component of the first column of Q_m^T (or the first row of Q_m).

The *full* GMRES method constructs iterates $x_1, x_2, \dots, x_m, \dots$, from the nested Krylov spaces

$$x_0 + \mathcal{K}_1(A; r_0) \subset x_0 + \mathcal{K}_2(A; r_0) \subset \cdots \subset x_0 + \mathcal{K}_m(A; r_0) \subset \cdots$$

At iteration m , the Arnoldi matrices are not recomputed from scratch; instead, they are updated from the previous step by appending one additional column to each matrix. Likewise, the QR factorization of $\tilde{H}_m$ is updated recursively from the previous factorization. The resulting algorithm is given below and the recursive process is detailed after that.

Algorithm 8: Full GMRES method.

Input: $A \in \mathbb{R}^{n \times n}$; vector b ; initial guess x_0 ; tolerance ε ; maximum iteration m_{max} .
Output: Number of iterations m_{iter} ; final solution $x_{m_{iter}}$; relative residual norm $relres$ of final solution; indicator $flag$.

Set $r_0 = b - Ax_0$, $\rho_0 = \|r_0\|_2$, $v_1 = r_0/\rho_0$, $relres = 1$, $flag = 0$, $m = 1$
while $relres > \varepsilon$ **and** $m \leq m_{max}$ **do**
 % Arnoldi
 $w = Av_m$
 for $j = 1$ **to** m **do**
 $h_{j,m} = v_j^T w$
 $w = w - h_{j,m}v_j$
 end for
 $h_{m+1,m} = \|w\|_2$
 if $h_{m+1,m} = 0$ **then** *Break*
 $v_{m+1} = w/h_{m+1,m}$
 Update matrices $\tilde{H}_m$ and V_m

 % GMRES
 Solve $\min_y \|\tilde{H}_m y - \rho_0 \hat{e}_1\|_2$ using QR factorization for solution y_m
 Compute $x_m = x_0 + V_m y_m$
 Compute $relres = \|r_m\|_2 / \|b\|_2$ ($= \rho_0 |\tau_m| / \|b\|_2$)
 Set $m = m + 1$
end while
Set $m_{iter} = m - 1$
if $m_{iter} = m_{max}$ **then** $flag = 1$

Now we describe how important parts of the above GMRES algorithm (solving the least squares problem and computing the residual) can be implemented efficiently. For $m = 1$, the Arnoldi process produces the basis vector $V_1 = [v_1]$ and the Hessenberg matrix

$$\tilde{H}_1 = \begin{bmatrix} h_{11} \\ h_{21} \end{bmatrix}.$$

We eliminate the subdiagonal entry h_{21} using the Givens rotation

$$G_1 = \begin{bmatrix} c_1 & s_1 \\ -s_1 & c_1 \end{bmatrix}, \quad c_1 = \frac{h_{11}}{\sqrt{h_{11}^2 + h_{21}^2}}, \quad s_1 = \frac{h_{21}}{\sqrt{h_{11}^2 + h_{21}^2}}.$$

Multiplying from the left gives

$$G_1 \tilde{H}_1 = \begin{bmatrix} h_{11}^{(1)} \\ 0 \end{bmatrix} =: \tilde{H}_1^{(1)}.$$

Thus we obtain the QR factorization $\tilde{H}_1 = Q_1 R_1$, where $Q = G_1^T$ and $R_1 = \tilde{H}_1^{(1)}$. To solve system (3.24) and compute the residual norm (3.25), we need the first column of $Q_1^T = G_1$. In this case,

$$g^{[1]} = G_1 \hat{e}_1 = \begin{bmatrix} c_1 \\ -s_1 \end{bmatrix}.$$

Hence the solution is $y_1 = \|r_0\|_2 c_1 / h_{11}^{(1)}$, and $x_1 = V_1 y_1$, and

$$\|r_1\|_2 = \|r_0\|_2 |s_1|.$$

Since $s_1 \in [0, 1]$ (it is positive because h_{21} is positive), we conclude that $\|r_1\|_2 \leq \|r_0\|_2$, so the residual is nonincreasing. If the residual is not sufficiently small, we proceed to the second step $m = 2$. The Arnoldi process produces the second basis vector v_2 and the second column of $\tilde{H}_2$, namely $h_2 = [h_{12}, h_{22}, h_{32}]^T$. We update

$$V_2 = [V_1, v_2].$$

Since the rotation G_1 from the previous step has already triangularized the first column of $\tilde{H}_2$, we now apply it only to the new column h_2 :

$$G_1 \circ h_2 = \begin{bmatrix} h_{12}^{(1)} \\ h_{22}^{(1)} \\ h_{32} \end{bmatrix}.$$

From here on, by operator $G_j \circ$ we mean that the 2×2 rotation matrix G_j applies on two rows j and $j + 1$ only and keep the other rows unchanged. Next, we introduce the Givens rotation

$$G_2 = \begin{bmatrix} c_2 & s_2 \\ -s_2 & c_2 \end{bmatrix}, \quad c_2 = \frac{h_{22}^{(1)}}{\sqrt{(h_{22}^{(1)})^2 + h_{32}^2}}, \quad s_2 = \frac{h_{32}}{\sqrt{(h_{22}^{(1)})^2 + h_{32}^2}},$$

to eliminate the entry h_{32} . Applying $G_2 \circ$ from the left gives

$$G_2 \circ G_1 \circ h_2 = \begin{bmatrix} h_{12}^{(1)} \\ h_{22}^{(2)} \\ 0 \end{bmatrix},$$

which keeps the first element unchanged, modifies the second element, and makes the last element zero. Now we append this new column to $\tilde{H}_1^{(1)}$ and obtain the upper triangular matrix

$$\tilde{H}_2^{(2)} = \begin{bmatrix} h_{11}^{(1)} & h_{12}^{(1)} \\ 0 & h_{22}^{(2)} \\ 0 & 0 \end{bmatrix} =: R_2.$$

It is clear that $\tilde{H}_2^{(2)}$ is indeed obtained by $\tilde{H}_2^{(2)} = G_2 \circ G_1 \circ \tilde{H}_2$. The first column of $Q_2^T = G_2 \circ G_1 \circ I_{3 \times 3}$ is computed as

$$g_2 = G_2 \circ G_1 \circ \hat{e}_1 = G_2 \circ \begin{bmatrix} g_1 \\ 0 \end{bmatrix} = \begin{bmatrix} c_2 & s_2 \\ -s_2 & c_2 \end{bmatrix} \circ \begin{bmatrix} c_1 \\ -s_1 \\ 0 \end{bmatrix} = \begin{bmatrix} c_1 \\ -c_2 s_1 \\ s_1 s_2 \end{bmatrix} =: \begin{bmatrix} \tilde{g}_2 \\ \tau_2 \end{bmatrix}, \quad \tau_2 = s_1 s_2$$

where the last component of g_2 is denoted by τ_2 and the remaining by $\tilde{g}_2$. We further we denote the square triangular part of R_2 by $\tilde{R}_2$ (removing the zero row), the least squares solution y_2 is obtained by solving the upper triangular system

$$\tilde{R}_2 y_2 = \|r_0\|_2 \tilde{g}_2$$

which is obtained by backward substitution. Then $x_2 = V_2 y_2$, and according to (3.25) we can write

$$\|r_2\|_2 = \|r_0\|_2 |\tau_2| = \|r_0\|_2 |s_1 s_2|.$$

Again we observe that $\|r_2\|_2 = s_2\|r_1\|_2 \leq \|r_1\|_2$ because $s_2 \in [0, 1]$. The residual is non-increasing, hopefully it approaches zero if we continue the iterations. Note that we can compute τ_2 recursively by setting $\tau_0 = 1$, $\tau_1 = -s_1\tau_0$ and $\tau_2 = -s_2\tau_1$.

We now describe the general step. Assume that after step $m-1$ we have already computed the QR factorization

$$R_{m-1} = \tilde{H}_{m-1}^{(m-1)} = G_{m-1} \circ \cdots \circ G_1 \circ \tilde{H}_{m-1}, \quad g_{m-1} = G_{m-1} \circ \cdots \circ G_1 \circ \hat{e}_1 =: \begin{bmatrix} \tilde{g}_{m-1} \\ \tau_{m-1} \end{bmatrix}$$

where $\tilde{H}_{m-1}^{(m-1)}$ is upper triangular, and $\tau_{m-1} = -s_{m-1}\tau_{m-2}$. At step m , the Arnoldi process produces the new basis vector v_m and the new Hessenberg column $h_m = [h_{1m}, h_{2m}, \dots, h_{m+1,m}]^T$. The previously computed rotations $G_1, \dots, G_{m-1}$ are then applied successively to h_m and transfer it to a new column:

$$G_{m-1} \circ \cdots \circ G_1 \circ h_m = \begin{bmatrix} h_{1m}^{(1)} \\ h_{2m}^{(2)} \\ \vdots \\ h_{mm}^{(m-1)} \\ h_{m+1,m} \end{bmatrix}.$$

Next, we construct the new Givens rotation

$$G_m = \begin{bmatrix} c_m & s_m \\ -s_m & c_m \end{bmatrix}, \quad c_m = \frac{h_{mm}^{(m-1)}}{\sqrt{(h_{mm}^{(m-1)})^2 + h_{m+1,m}^2}}, \quad s_m = \frac{h_{m+1,m}}{\sqrt{(h_{mm}^{(m-1)})^2 + h_{m+1,m}^2}},$$

which applies on the last two entries and eliminates the last entry $h_{m+1,m}$:

$$G_m \circ G_{m-1} \circ \cdots \circ G_1 \circ h_m = \begin{bmatrix} h_{1m}^{(1)} \\ h_{2m}^{(2)} \\ \vdots \\ h_{mm}^{(m)} \\ 0 \end{bmatrix}.$$

We append this column to $\tilde{H}_{m-1}^{(m-1)}$ and obtain the upper triangular matrix

$$\tilde{H}_m^{(m)} = \begin{bmatrix} h_{11}^{(1)} & h_{12}^{(1)} & \cdots & h_{1m}^{(1)} \\ & h_{22}^{(2)} & \cdots & h_{2m}^{(2)} \\ & & \ddots & \vdots \\ & & & h_{mm}^{(m)} \\ & & & & 0 \end{bmatrix} =: R_m.$$

Finally, the first column of Q_m^T , i.e. g_m , is obtained by applying G_m on g_{m-1} appended by zero, keeping in mind that $G_m \circ$ operates on m -th and $(m+1)$ -st components (last two components):

$$g_m = G_m \circ \begin{bmatrix} g_{m-1} \\ 0 \end{bmatrix} = \begin{bmatrix} c_m & s_m \\ -s_m & c_m \end{bmatrix} \circ \begin{bmatrix} \tilde{g}_{m-1} \\ \tau_{m-1} \\ 0 \end{bmatrix} = \begin{bmatrix} \tilde{g}_{m-1} \\ c_m \tau_{m-1} \\ -s_m \tau_{m-1} \end{bmatrix} =: \begin{bmatrix} \tilde{g}_m \\ \tau_m \end{bmatrix} \quad (3.26)$$

where

$$\tilde{g}_m = \begin{bmatrix} \tilde{g}_{m-1} \\ c_m \tau_{m-1} \end{bmatrix}, \quad \tau_m = -s_m \tau_{m-1}. \quad (3.27)$$

Now, the solution y_m is obtained by solving the upper triangular system

$$\tilde{R}_m y_m = \|r_0\|_2 \tilde{g}_m$$

using backward substitution and the m -th iterate x_m is computed via $x_m = V_m y_m$. The residual norm according to (3.25) satisfies

$$\|r_m\|_2 = \|r_0\|_2 |\tau_m| = s_m \|r_{m-1}\|_2. \quad (3.28)$$

Since $s_j \in [0, 1]$, we conclude that

$$\|r_m\|_2 \leq \|r_{m-1}\|_2 \leq \cdots \leq \|r_0\|_2 \quad (3.29)$$

By increasing m , the norm of the residual goes to zero unless we get $s_j = 1$ for all j greater than some fixed m .

Remark 3.4. For implementation, the old entries h_{jm} are not stored permanently. Instead, at each step the Hessenberg columns are overwritten from top to bottom as the Givens rotations are applied. Thus only one upper triangular factor needs to be stored and expanded at every iteration. All Givens parameters c_j and s_j must also be stored since all previously computed rotations are needed when processing a new column. The main drawback, however, is that the entire matrix $V_m \in \mathbb{R}^{n \times m}$ must be stored and used to compute x_m from y_m .

Theorem 3.13 (Happy breakdown). If the Arnoldi algorithm breaks down at an intermediate step m , then the iterate x_m of the full GMRES algorithm is the exact solution of system $Ax = b$.

Proof. The Arnoldi Algorithm 2 breaks down at step m if $h_{m+1,m} = 0$. But this means that the Givens parameter $s_m = 0$ and $\|r_m\|_2 = 0$ according to (3.28). $\square$

Exercise 3.10. System (3.24) is solvable if R_m is non-singular which means if all diagonal elements of R_m are non-zero. Show that in GMRES process R_m cannot be singular, unless x_{m-1} is the exact solution of $Ax = b$.

In the following exercises we explore some relations between the Arnoldi-Galerkin (ArGal) and GMRES algorithms.

Exercise 3.11. Come back to the reduced square system $H_m y_m = \|r_0\|_2 \hat{e}_1$ of the Arnoldi-Galerkin method in (3.5), and its residual norm $\|r_m\|_2 = h_{m+1,m} |y_m^{(m)}|$ in (3.7). Modify the QR-Givens process given above for solving this system. Write the residual in terms of QR and Givens parameters and compare it with the residual formula (3.28) of the full GMRES method.

Final solution: If we denote the the residual of GMRES by r_m^{GMRES} and that of the

Arnoldi–Galerkin by r_m^{ArGal} , you will show that

$$r_m^{\text{GMRES}} = s_m^2 r_{m-1}^{\text{GMRES}} + c_m^2 r_m^{\text{ArGal}},$$

and if we denote the norms of the residuals by ρ_m^{GMRES} and ρ_m^{ArGal} , receptively, you will show that

$$\rho_m^{\text{ArGal}} = \frac{1}{c_m} \rho_m^{\text{GMRES}}$$

where s_m and c_m are the m -th Givens parameters. This division by c_m causes the convergence curve of the ArGal algorithm to be sometimes considerably more oscillatory than that of GMRES.

Exercise 3.12. Show that if at any step m , the GMRES method stagnates, i.e. $x_m = x_{m-1}$, then H_m is singular and thus the ArGal system $H_m y_m = \|r_0\|_2 \hat{e}_1$ is not solvable which means that the iterate x_m of ArGal is not defined (breaks down). Conversely, if ArGal breaks down at step m then GMRES stagnates.

Compared with the CG or Lanczos process, the Arnoldi process does not use short recurrences. Consequently, each iteration requires the storage of additional vector h_m . The upper triangular matrix R should be stored and is expanding step by step. The whole matrix V_m must also be stored. These impose a severe limitation on the full GMRES method when A is a large matrix and m gets larger and larger. For this reason, GMRES is commonly used in a *restarted* form. The method GMRES(k) is a variant of GMRES in which the Arnoldi process is restarted every k iterations, using the most recent iterate as the new initial vector for the next Arnoldi cycle. In this way, the storage requirement is limited to $\mathcal{O}(k)$ vectors.

All remarks concerning the restarted Arnoldi–Galerkin algorithm in Section 3.4, including the interpretation of cycles, stopping criteria, and convergence monitoring, apply equally to the restarted GMRES algorithm presented below.

Algorithm 9: GMRES(k) method (restarted GMRES)

Input: $A \in \mathbb{R}^{n \times n}$; vector b ; initial guess x_0 ; tolerance ε ; restart number k , maximum number of cycles $j_{\max}$.

Output: Number of cycles j_{iter} ; final solution x ; relative residual norm relres of final solution; indicator flag .

Set $j = 0$, $\text{flag} = 0$

while $j < j_{\max}$ **do**

 Run the full GMRES algorithm 8 with inputs A , b , x_0 , ε and $m_{\max} = k$ and outputs x and m_{iter}

if $m_{\text{iter}} < k$ **then** *Break*

 Set $x_0 = x$ and $j = j + 1$

end while

Set $j_{\text{iter}} = j$

if $j_{\text{iter}} = j_{\max}$ **then** $\text{flag} = 1$

For a general matrix A , convergence of GMRES(k) is not guaranteed. Moreover, it is often difficult to determine a priori an appropriate value for the restart parameter k . Choosing k too small may lead to slow convergence or even stagnation, while choosing k too large diminishes

the computational and storage advantages of restarting. Nevertheless, GMRES(k) remains one of the most widely used methods for solving large nonsymmetric linear systems because of its robustness and modest memory requirements.

3.10 MINRES method

If the matrix A is *symmetric* (not necessarily positive definite), then the Arnoldi process reduces to the Lanczos process, and the resulting Krylov method becomes a special case of GMRES known as the MINRES (MINimum RESidual) method introduced by Paige and Saunders [12, 1975]. Historically, MINRES was developed more than a decade before GMRES. The latter was introduced by Saad and Schultz in [14, 1986] to extend the minimum residual approach to general nonsymmetric matrices.

In MINRES, the Hessenberg matrix $\tilde{H}_m$ becomes the symmetric tridiagonal matrix $\tilde{T}_m$, which simplifies the computations and reduces the storage requirements. In particular, in the QR–Givens process there is no need to form and store a general upper triangular matrix. We will see that in an efficient implementation we do not need to store the matrix V_m as well. In the QR factorization

$$\tilde{T}_m = Q_m R_m$$

the matrix R_m has the upper tridiagonal form

$$R_m = \begin{bmatrix} \alpha_1^{(1)} & \beta_1^{(1)} & \gamma_1 & & & \\ & \alpha_2^{(2)} & \beta_2^{(2)} & \gamma_2 & & 0 \\ & & \ddots & \ddots & \ddots & \\ & & & & \gamma_{m-2} & \\ & 0 & & & \beta_{m-1}^{(2)} & \\ & & & & \alpha_m^{(2)} & \\ & & & & & 0 \end{bmatrix}.$$

Most entries of R_m are affected by two successive Givens rotations, except for the entries in the first row and the elements γ_j , which are generated by only one Givens rotation. In fact, at the first step only G_1 acts on the first column of $\tilde{T}_1$. At the second step, G_1 and G_2 act on the second column, and at any step $m > 2$, only the three rotations G_{m-2} , G_{m-1} , and G_m act on the m -th column of $\tilde{T}_m$, namely

$$h_m = [0, \dots, 0, \beta_{m-1}, \alpha_m, \beta_m]^T,$$

since its first $m - 2$ entries are already zero. The action of G_{m-2} introduces the additional nonzero entry γ_{m-2} and updates the entry β_{m-1} . Next, the rotation G_{m-1} further changes β_{m-1} and also updates α_m . Finally, the rotation G_m updates α_m once more and eliminates the subdiagonal entry β_m . The first column of Q_m^T , i.e.

$$g_m = G_m \circ \dots \circ G_1 \circ \hat{e}_1,$$

has exactly the same form as in (3.26). We leave the detailed derivation, which follows the similar routine as in the general case, as an exercise.

At each step, the values $\alpha_j^{(i)}$ and $\beta_j^{(i)}$ are modified to new values with higher superscripts, after which the previous values are no longer needed. For simplicity of notation, we therefore drop the superscripts and write R_m in the form

$$R_m = \begin{bmatrix} \alpha_1 & \beta_1 & \gamma_1 & & & \\ & \alpha_2 & \beta_2 & \gamma_2 & & 0 \\ & & \ddots & \ddots & \ddots & \\ & & & & & \gamma_{m-2} \\ & 0 & & & & \beta_{m-1} \\ & & & & & \alpha_m \\ & & & & & 0 \end{bmatrix},$$

keeping in mind that these coefficients α_j and β_j are different from the Lanczos coefficients appearing in $\tilde{T}_m$. For implementation, the matrix R_m is not stored. Even we do not need to store α , β , and γ values as vectors. We overwrite these values step by step.

As before, let $\tilde{R}_m$ denote the $m \times m$ matrix obtained by removing the last (zero) row of R_m , and let $\tilde{g}_m$ denote the vector obtained by deleting the last element of g_m . A direct approach is to continue as done in GMRES to solve the upper triangular (here upper tridiagonal) system

$$\tilde{R}_m y_m = \|r_0\|_2 \tilde{g}_m, \quad (3.30)$$

and then compute $x_m = V_m y_m$. With this approach, R_m does not need to be stored as a matrix but all columns of V_m must still be kept, which is not ideal and practical for large problems and large values of m . Instead, we derive an elegant algorithm like CG algorithm which needs only short recurrences and avoids the storage of the old Lanczos vectors.

The matrix R_m is nonsingular ($\alpha_j \neq 0$ for all j), otherwise one of the previous iterates is already the exact solution. We already know that $\{v_1, \dots, v_m\}$ forms an orthonormal basis for the Krylov subspace $\mathcal{K}_m(A; r_0)$ from which the approximation x_m is selected. We now construct another basis $\{p_0, p_1, \dots, p_{m-1}\}$ for $\mathcal{K}_m(A; r_0)$. Define the matrix

$$P_m = [p_0, p_1, \dots, p_{m-1}]$$

by

$$P_m = V_m \tilde{R}_m^{-1}. \quad (3.31)$$

Since V_m is orthogonal and $\tilde{R}_m^{-1}$ is upper triangular, this expression can be viewed as the QR factorization of P_m . In particular, P_m is nonsingular, and its columns form a basis for the same subspace spanned by the columns of V_m , i.e. $\mathcal{K}_m(A; r_0)$. From $P_m \tilde{R}_m = V_m$ we obtain

$$[p_0, p_1, p_2, \dots, p_{m-1}] \begin{bmatrix} \alpha_1 & \beta_1 & \gamma_1 & & & \\ & \alpha_2 & \beta_2 & \gamma_2 & & 0 \\ & & \ddots & \ddots & \ddots & \\ & & & & & \gamma_{m-2} \\ & 0 & & & & \beta_{m-1} \\ & & & & & \alpha_m \end{bmatrix} = [v_1, v_2, v_3, \dots, v_m].$$

Comparing columns immediately yields the recursions

$$\begin{aligned} p_0 &= \frac{1}{\alpha_1} v_1, \\ p_1 &= \frac{1}{\alpha_2} (v_2 - \beta_1 p_0), \\ p_k &= \frac{1}{\alpha_{k+1}} (v_{k+1} - \gamma_{k-1} p_{k-2} - \beta_k p_{k-1}), \quad k \geq 2. \end{aligned} \tag{3.32}$$

Thus, at step $k + 1$, the new vector p_k can be computed using only the current Lanczos vector v_{k+1} together with the two previous vectors p_{k-2} and p_{k-1} . This gives the desired short recurrence. Next, using (3.30), the MINRES approximation can be written as

$$x_m = x_0 + V_m y_m = x_0 + P_m \tilde{R}_m y_m = x_0 + \|r_0\|_2 P_m \tilde{g}_m.$$

Now we see how a recursive update can be derived for x_m from previous iterate x_{m-1} . By writing P_m as $P_m = [P_{m-1}, p_{m-1}]$ and according to (3.27), we can write

$$\begin{aligned} x_m &= x_0 + \|r_0\|_2 P_m \tilde{g}_m \\ &= x_0 + \|r_0\|_2 [P_{m-1}, p_{m-1}] \begin{bmatrix} \tilde{g}_{m-1} \\ c_m \tau_{m-1} \end{bmatrix} \\ &= x_0 + \|r_0\|_2 P_{m-1} \tilde{g}_{m-1} + \|r_0\|_2 c_m \tau_{m-1} p_{m-1} \\ &= x_{m-1} + \mu_{m-1} p_{m-1}, \end{aligned}$$

where

$$\mu_{m-1} = \|r_0\|_2 c_m \tau_{m-1}.$$

We remind that the quantities τ_m are computed recursively from

$$\tau_0 = 1, \quad \tau_k = -s_k \tau_{k-1}, \quad k \geq 1.$$

The above derivation shows that MINRES enjoys short recurrences and avoids storing the entire Lanczos basis or any matrix with $\mathcal{O}(m^2)$ number of entries. By shifting the index by one, the iteration takes the form

$$x_{m+1} = x_m + \mu_m p_m,$$

which is analogous to (3.10) in the CG algorithm. Here, the vector p_m plays the role of a search direction, while μ_m acts as the corresponding step length. The residual norm need not be computed explicitly from $r_m = b - Ax_m$, since it is available for free via $\|r_m\|_2 = \|r_0\|_2 |\tau_m|$. The full method is given in Algorithm 10.

Regarding the construction of the search directions, the main difference between CG and MINRES is that MINRES requires one additional vector to compute the new direction p_m . For readability, we wrote the MINRES algorithm using indexed quantities. In an actual implementation, however, x_m , α_m , γ_m , μ_m , and τ_m do not need to be stored for all iterations; their values are simply overwritten at each step. Also, only two β values, two Givens rotations, and two vectors p need to be stored simultaneously.

	Av	$u^T v$	$u + v$	αv	total cost
CG	1	2	3	3	$2 \cdot \text{nnz}(A) + 10n$
MINRES	1	3	5	5	$2 \cdot \text{nnz}(A) + 16n$

Table 2: Cost comparison of the CG and MINRES algorithms. Here $\text{nnz}(A)$ denotes the number of nonzero entries in A .

Comparing the number of matrix-vector products (Av), inner products ($u^T v$), vector additions ($u + v$), and scalar-vector multiplications (αv) required per iteration, we obtain the following cost comparison table for CG and MINRES.

The cost of computing and applying the Givens rotations in MINRES is only $\mathcal{O}(1)$ per iteration and is therefore neglected in the table. Apart from the matrix-vector product. Let η denote the average number of nonzero entries per row of A , so that $\text{nnz}(A) \approx \eta n$. For large values of η , the matrix-vector multiplication dominates the overall cost and the difference between the two methods becomes small. In terms of memory requirements, MINRES requires only one additional search direction vector compared with CG. The main advantage of MINRES is its broader applicability. While CG is restricted to symmetric positive definite systems, MINRES can be applied to arbitrary symmetric matrices, including indefinite systems.

Algorithm 10: MINRES method

Input: Symmetric matrix $A \in \mathbb{R}^{n \times n}$; vector b ; initial guess x_0 ; tolerance ε ; maximum iteration $m_{\max}$.

Output: Number of iterations m_{iter} ; final solution $x_{m_{\text{iter}}}$; relative residual norm relres of final solution; indicator flag .

Set $r_0 = b - Ax_0$, $\rho_0 = \|r_0\|_2$, $\text{relres} = 1$, $\text{flag} = 0$, $m = 1$,

Set $v_0 = 0$, $v_1 = r_0/\rho_0$, $\beta_0 = 0$,

Set $p_{-2} = p_{-1} = 0$, $\gamma_{-1} = \gamma_0 = 0$, $G_{-1} = G_0 = 0$, $\tau_0 = 1$.

while $\text{relres} > \varepsilon$ **and** $m \leq m_{\max}$ **do**

 % Lanczos process

$w = Av_m - \beta_{m-1}v_{m-1}$

$\alpha_m = v_m^T w$

$w = w - \alpha_m v_m$

$\beta_m = \|w\|_2$

if $\beta_m = 0$ **then Break**

$v_{m+1} = w/\beta_m$

 % MINRES

 Apply rotations G_{m-2} and G_{m-1} to compute γ_{m-2} and to update β_{m-1} and α_m

 Compute the new Givens values c_m and s_m , form G_m and apply it to update α_m once more

 Update $p_{m-1} = (v_m - \gamma_{m-2}p_{m-3} - \beta_{m-1}p_{m-2})/\alpha_m$

 Compute $\mu_{m-1} = \rho_0 c_m \tau_{m-1}$

 Update $x_m = x_{m-1} + \mu_{m-1} p_{m-1}$

 Update $\tau_m = -s_m \tau_{m-1}$

 Compute $\text{relres} = \rho_0 |\tau_m| / \|b\|_2$ (which equals to $\|r_m\| / \|b\|_2$)

 Set $m = m + 1$

end while

Set $m_{\text{iter}} = m - 1$

if $m_{\text{iter}} = m_{\max}$ **then** $\text{flag} = 1$

Exercise 3.13. Use Exercise 3.9 and prove that the MINRES search directions satisfy

$$r_m^T A p_j = 0, \quad j = 0, 1, \dots, m-1.$$

This is the MINRES analogue of the CG orthogonality condition (3.17), but here we have A -orthogonality instead.

Exercise 3.14. Assume that A is symmetric. Use the Arnoldi relation (3.1) in its Lanczos version and definition P_m via (3.31) to show that

$$v_i^T A p_j = 0, \quad i > j + 1.$$

Then show that $P^T A P$ is a symmetric tridiagonal matrix and hence the MINRES search directions satisfy

$$p_i^T A p_j = 0, \quad \text{whenever } |i - j| > 1.$$

Compare it with the CG search direction property (3.4).

Exercise 3.15. Show that the same idea of replacing the Arnoldi basis $\{v_1, \dots, v_m\}$ by the basis $\{p_0, \dots, p_{m-1}\}$ does not resolve the storage problem in the GMRES algorithm.

In practice, restarted versions of MINRES are not used because MINRES does not suffer from the storage limitations encountered in full GMRES.

3.11 Convergence of GMRES and MINRES methods

The first result, concerns the finite termination of the GMRES (and also MINRES as a special case) at at most n iterations.

Theorem 3.14. In the absence of rounding errors, the full GMRES algorithm applied to an $n \times n$ nonsingular system $Ax = b$ produces the exact solution in at most n iterations.

Proof. Assume that after $n - 1$ iterations the exact solution has not yet been obtained. This means that the Arnoldi algorithm 2 has already been produced n orthogonal vectors $v_1, \dots, v_n \in \mathbb{R}^n$. Now, the Arnoldi algorithm goes to produce the vector $w = Av_n \in \mathbb{R}^n$ and makes it orthogonal to n previous basis vectors $v_1, \dots, v_n$ of $\mathbb{R}^n$. Then w is necessarily zero and hence $h_{n+1,n} = \|w\|_2 = 0$. This gives $s_n = 0$ and thus $\|r_n\|_2 = 0$. $\square$

However, this would be impractical for large systems if there were many steps required for convergence.

For analysis we use the fact that since $x_m \in x_0 + \mathcal{K}_m(A; r_0)$, the residual r_m can be written as $r_m = p(A)r_0$ where $p \in \mathbb{P}_{m+1}$ and $p(0) = 1$. Then similar to Theorem 3.10 for the CG method, we have the following theorem

Theorem 3.15. Let x_m be the m -th iterate of the full GMRES method and let $r_m =$

$b - Ax_m$ be the corresponding residual. Then

$$\|r_m\|_2 = \min_{p \in \mathbb{P}_{m+1}, p(0)=1} \|p(A)r_0\|_2, \quad (3.33)$$

where $\mathbb{P}_{m+1}$ denotes the space of polynomials of order at most $m+1$ (degree at most m).

The proof uses the fact that in GMRES the residual norm $\|r_m\|_2$ is minimized over the Krylov space $\mathcal{K}_m(A; r_0)$.

The analysis for general matrices is not possible, so we assume that A has a set of n independent eigenvectors $v_1, \dots, v_n$ corresponding to eigenvalues $\lambda_1, \dots, \lambda_n$. This means that A is diagonalizable. Then the eigenvectors form a basis for $\mathbb{R}^n$ and we can expand the residual r_0 as $r_0 = c_1 v_1 + \dots + c_n v_n$ for some coefficients c_j . It is not difficult to see that

$$r_m = p(A)r_0 = \sum_{j=1}^n c_j p(\lambda_j) v_j. \quad (3.34)$$

If A is *symmetric*, then eigenvectors form an *orthonormal* basis for $\mathbb{R}^m$ and we can simply show that

$$\|r_m\|_2^2 = r_m^T r_m = \sum_{j=1}^n c_j^2 p(\lambda_j)^2$$

and similar to Theorem 3.11 we can prove that

Theorem 3.16. Let x_m be the m -th iterate of the MINRES method and let $r_m = b - Ax_m$ be the corresponding residual. Then

$$\frac{\|r_m\|_2}{\|r_0\|_2} \leq \left(\min_{p \in \mathbb{P}_{m+1}, p(0)=1} \max_{1 \leq j \leq n} |p(\lambda_j)| \right), \quad (3.35)$$

where λ_j are the eigenvalues of symmetric matrix A .

Similar to CG, the convergence is rapid when the eigenvalues of A are grouped into a few tight clusters. If A does not have this property, then an appropriate preconditioner M so that $M^{-1}A$ exhibits such a clustered spectrum should be used.

If A is not symmetric but still diagonalizable, then A can be decomposed as $A = X\Lambda X^{-1}$ where $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$ and $X = [v_1, v_2, \dots, v_n]$. In this case X is called the eigenmatrix of A . The following theorem establishes a bound on the residual norm of the GMRES algorithm for solving $Ax = b$.

Theorem 3.17. Assume that A is diagonalizable with eigenvalues λ_j and eigenmatrix X . Let x_m be the m -th iterate of the GMRES method for solving $Ax = b$ and let $r_m = b - Ax_m$ be the corresponding residual. Then

$$\frac{\|r_m\|_2}{\|r_0\|_2} \leq \kappa(X) \left(\min_{p \in \mathbb{P}_{m+1}, p(0)=1} \max_{1 \leq j \leq n} |p(\lambda_j)| \right). \quad (3.36)$$

where $\kappa(X) = \|X\|_2 \cdot \|X^{-1}\|_2$ is the condition number of X .

Proof. Similar to the previous theorem, we have $r_m = p(A)r_0$, where $p \in \mathbb{P}_{m+1}$ satisfies $p(0) = 1$. However, in the nonsymmetric case the eigenvectors v_j are generally not orthogonal, so we cannot write the compact expression (3.34). Nevertheless, since $p(A) = Xp(\Lambda)X^{-1}$, we obtain

$$\|r_m\|_2 = \|p(A)r_0\|_2 = \|Xp(\Lambda)X^{-1}r_0\|_2 \leq \|X\|_2 \|X^{-1}\|_2 \|p(\Lambda)\|_2 \|r_0\|_2.$$

Because $p(\Lambda)$ is diagonal, its 2-norm is $\|p(\Lambda)\|_2 = \max_{1 \leq j \leq n} |p(\lambda_j)|$. Hence

$$\|r_m\|_2 \leq \kappa(X) \max_{1 \leq j \leq n} |p(\lambda_j)| \|r_0\|_2.$$

This estimate holds for all Krylov subspace methods whose iterates are chosen from $\mathcal{K}_m(A; r_0)$. Since GMRES minimizes the residual norm over $\mathcal{K}_m(A; r_0)$, the above estimate immediately yields (3.36). $\square$

Since the condition number of the eigenmatrix is not known and can be very large, results of the above theorem are of limited practical interest. It can be useful only when it is known that the matrix is nearly normal, in which case $\kappa(X) \approx 1$. Note that a matrix $A \in \mathbb{C}^{n \times n}$ is called normal if

$$AA^T = A^T A.$$

Examples of normal matrices are symmetric matrices, unitary matrices, and skew-symmetric matrices.

We observed in (3.29) that the sequence of residual norms generated by the GMRES algorithm is non-increasing, although it is not necessarily strictly decreasing. A remarkable result due to Greenbaum *et al.* [7, 1996] states that any non-increasing convergence history is possible for GMRES. More precisely, given any positive sequence

$$\|r_0\|_2 = \rho_0 \geq \rho_1 \geq \cdots \geq \rho_{n-1} > 0,$$

there exist a matrix $A \in \mathbb{R}^{n \times n}$ and an initial residual vector r_0 such that the residuals produced by GMRES satisfy $\|r_k\|_2 = \rho_k$ for $k = 1, \dots, n-1$. Moreover, the matrix A can be constructed with any prescribed set of eigenvalues. Consequently, no useful convergence estimate for GMRES can be derived solely from the spectrum of A . In particular, the GMRES iteration may stagnate for many consecutive steps. The following exercise shows that a stagnation may occur for as many as $n-1$ iterations, with convergence taking place only at the final step.

Exercise 3.16. Show that GMRES applied to the linear system $Ax = b$ with initial guess $x_0 = \hat{e}_1$ makes no progress until step n , where

$$A = \begin{bmatrix} 0 & 1 & & \\ & \ddots & \ddots & \\ & & 0 & 1 \\ -c_{n-1} & \cdots & -c_1 & -c_0 \end{bmatrix} \in \mathbb{R}^{n \times n}.$$

Note that A is the companion matrix associated with the polynomial $p(t) = t^n + c_{n-1}t^{n-1} +$

$\cdots + c_1 t + c_0$ whose roots are the eigenvalues of A .

Despite these negative theoretical results, in practice GMRES (with a good preconditioner) is *often* has a superlinear convergence (like CG method). In other words, the rate of convergence improves as the iteration proceeds.

3.12 Petrov–Galerkin methods

The Galerkin method imposes the orthogonality condition $r_m \perp \mathcal{K}_m(A, r_0)$, where the approximation x_m is sought in the affine space $x_m \in x_0 + \mathcal{K}_m(A, r_0)$. For symmetric positive definite matrices, this choice leads to efficient and optimal algorithms such as the CG method. However, for nonsymmetric matrices, the Galerkin condition is generally no longer appropriate, since orthogonality of the residual with respect to the same Krylov subspace does not necessarily guarantee good convergence properties. The minimal residual approach partially overcomes this difficulty by minimizing the Euclidean norm of the residual, $\|r_m\|_2 = \|b - Ax_m\|_2$ over the Krylov subspace. This strategy gives rise to robust and stable methods such as GMRES. Nevertheless, both the Arnoldi–Galerkin and GMRES methods require long recurrences for nonsymmetric matrices due to the upper Hessenberg structure produced by the Arnoldi process. Consequently, the computational cost and storage requirements increase with the iteration count m , since all previously generated basis vectors must be stored and orthogonalized. One possible remedy is to use restarted variants of these methods. In restarted methods, the dimension of the Krylov subspace is kept to a fixed value k , after which the iteration is restarted using the updated residual vector. In this way, the storage requirements and computational costs remain bounded while still allowing the improvement of the approximate solution.

There exists the following natural question: for nonsymmetric systems, are there Krylov subspace methods that employ short recurrences and thereby avoid the storage and computational costs associated with the full Arnoldi–Galerkin and GMRES methods?

To address this question, we introduce the more general framework of *Petrov–Galerkin methods*. Recall that in the standard Galerkin approach, the approximate solution x_m is sought in the Krylov subspace $x_0 + \mathcal{K}_m$ (the *trial space*), and the orthogonality condition is imposed against the same subspace $\mathcal{K}_m$ (the *test space*). The main idea of the Petrov–Galerkin framework is to use two different spaces. The approximate solution is still sought in the trial space $x_0 + \mathcal{K}_m$, but the residual orthogonality condition is imposed with respect to a different test space $\mathcal{L}_m$. This asymmetry between the trial and test spaces provides additional flexibility and is particularly advantageous for nonsymmetric linear systems.

Suppose that $x_m = x_0 + V_m y_m$, where the columns of V_m form a basis of $\mathcal{K}_m$, and let the columns of W_m form a basis of $\mathcal{L}_m$. Then the residual becomes

$$r_m = r_0 - AV_m y_m.$$

Imposing the Petrov–Galerkin orthogonality condition, $W_m^T r_m = 0$, gives $W_m^T(r_0 - AV_m y) = 0$, or equivalently,

$$(W_m^T AV_m) y_m = W_m^T r_0. \quad (3.37)$$

The matrix $W_m^T AV_m$ is generally nonsymmetric, even if V_m and W_m are orthonormal bases. Therefore, at each iteration one solves the reduced linear system (3.37). The effectiveness of

the method depends on the choice of the left and right Krylov subspaces and on the resulting conditioning of this projected matrix. Note that, the Galerkin method is a special case where $W_m = V_m$.

Petrov–Galerkin methods form the basis of several important iterative algorithms for nonsymmetric linear systems. In particular, the BiConjugate Gradient (BiCG) method arises from choosing $\mathcal{K}_m = \mathcal{K}_m(A; r_0)$ and $\mathcal{L}_m = \mathcal{K}_m(A^T, \tilde{r}_0)$ where $\tilde{r}_0$ is an arbitrary nonzero starting vector satisfying $(\tilde{r}_0, r_0) \neq 0$. Such methods avoid the long recurrences and storage growth encountered in minimal residual methods, since they can be implemented using short recurrences.

3.13 BiCG method

As we mentioned, the BiCG is a Petrov-Galerkin approach for solving non-symmetric system $Ax = b$ where the trial and test spaces are respectively chosen as

$$\mathcal{K}_m(A, r_0) = \text{span}\{r_0, Ar_0, \dots, A^{m-1}r_0\}$$

the Krylov subspace generated by A , and

$$\mathcal{L}_m = \mathcal{K}_m(A^T, \tilde{r}_0) = \text{span}\{\tilde{r}_0, A^T\tilde{r}_0, \dots, (A^T)^{m-1}\tilde{r}_0\}$$

the Krylov subspace generated by A^T . Here $\tilde{r}_0$ is an arbitrary nonzero starting vector satisfying

$$(\tilde{r}_0, r_0) \neq 0.$$

We look for bases for the trial and test spaces such that the projected matrix in the reduced system (3.37), $W_m^T A V_m$ becomes tridiagonal. In this way, the resulting iterative method can be implemented using short three-term recurrences. To achieve this goal, we construct two sets of vectors $\{v_1, \dots, v_{m+1}\}$ and $\{w_1, \dots, w_{m+1}\}$ satisfying the *biorthogonality conditions*

$$w_i^T v_j = 0, \quad i \neq j, \quad w_j^T v_j = 1. \quad (3.38)$$

These vectors can be generated by the *nonsymmetric Lanczos* process (also called the *two-sided Lanczos algorithm*), originally proposed by Lanczos in [9, 1950]. This process reduces the nonsymmetric matrix A to a tridiagonal matrix of the form

$$T_m = \begin{bmatrix} \alpha_1 & \beta_2 & & 0 \\ \delta_2 & \ddots & \ddots & \\ & \ddots & \ddots & \beta_m \\ 0 & & \delta_m & \alpha_m \end{bmatrix} = W_m^T A V_m, \quad (3.39)$$

and generates additional scalars β_{m+1} and δ_{m+1} and vectors v_{m+1} and w_{m+1} .

The biorthogonality condition (3.38) implies that $W_m^T V_m = I$, and equivalently $V_m^T W_m = I$. Assuming that the reduction (3.39) exists, we obtain $A V_m = V_m T_m$ and $A^T W_m = W_m T_m^T$. Comparing the k -th columns of these relations yields

$$\begin{aligned} A v_k &= \beta_k v_{k-1} + \alpha_k v_k + \delta_{k+1} v_{k+1}, \\ A^T w_k &= \delta_k w_{k-1} + \alpha_k w_k + \beta_{k+1} w_{k+1}, \end{aligned}$$

where we set $v_0 = w_0 = 0$, and choose the initial vectors v_1 and w_1 such that $w_1^T v_1 = 1$. Rearranging the above equations gives

$$\tilde{v}_{k+1} := \delta_{k+1} v_{k+1} = Av_k - \alpha_k v_k - \beta_k v_{k-1}, \quad (3.40)$$

$$\tilde{w}_{k+1} := \beta_{k+1} w_{k+1} = A^T w_k - \alpha_k w_k - \delta_k w_{k-1}. \quad (3.41)$$

Multiplying equation (3.40) from the left by w_k^T and using the biorthogonality relation (3.38), we obtain

$$\alpha_k = w_k^T A v_k.$$

To enforce the biorthogonality condition for $i = j = k + 1$, it suffices to choose β_{k+1} and δ_{k+1} such that $\beta_{k+1} \delta_{k+1} = \tilde{w}_{k+1}^T \tilde{v}_{k+1}$. Hence, there is some freedom in the choice of these parameters. In what follows, the normalization is chosen so that $\|v_k\|_2 = 1$.

Theorem 3.18. After m steps of the above biorthogonalization process, let $W_m = [w_1, \dots, w_m]$, $V_m = [v_1, \dots, v_m]$, and let T_m denote the tridiagonal matrix defined in (3.39). Then the following properties hold:

1. $W_m^T A V_m = T_m$,
2. $A V_m = V_m T_m + \delta_{m+1} v_{m+1} \tilde{e}_m^T$,
3. $A^T W_m = W_m T_m^T + \beta_{m+1} w_{m+1} \tilde{e}_m^T$.

Furthermore, the set $\{v_1, \dots, v_m\}$ forms a basis for the Krylov subspace $\mathcal{K}_m(A; v_1)$, while $\{w_1, \dots, w_m\}$ forms a basis for $\mathcal{K}_m(A^T; w_1)$.

Proof. Property 1 follows directly from the construction. Properties 2 and 3 are immediate consequences of equations (3.40) and (3.41), respectively. The biorthogonality condition (3.38) implies that both sets $\{v_1, \dots, v_m\}$ and $\{w_1, \dots, w_m\}$ are linearly independent in $\mathbb{R}^n$. Moreover, relation (3.40) shows recursively that each vector in the standard Krylov basis $\{v_1, Av_1, \dots, A^{m-1}v_1\}$ of $\mathcal{K}_m(A; v_1)$ can be expressed as a linear combination of $v_1, \dots, v_m$. Therefore, $\{v_1, \dots, v_m\}$ spans $\mathcal{K}_m(A; v_1)$ and hence forms a basis for this subspace. Similarly, $\{w_1, \dots, w_m\}$ is a basis for $\mathcal{K}_m(A^T; w_1)$. $\square$

The nonsymmetric Lanczos algorithm (Lanczos bi-orthogonalization algorithm, or two-sided Lanczos algorithm) is given in Algorithm 11.

There are two situations in which Algorithm 11 may break down. The first case occurs when either $v_{k+1} = 0$ or $w_{k+1} = 0$ (or both). In this situation, the algorithm has successfully identified an invariant subspace. Indeed, if $v_{k+1} = 0$, then by property 2 of Theorem 3.18 we obtain $AV_k = V_k T_k$, which shows that the column space of V_k is an A -invariant subspace. Similarly, if $w_{k+1} = 0$, then property 3 gives $A^T W_k = W_k T_k^T$, so the column space of W_k is an A^T -invariant subspace. This type of breakdown is called a *happy breakdown*, since the algorithm terminates after having extracted exact invariant subspace information. If $v_{k+1} = 0$ then the approximate solution is exact. If $w_{k+1} = 0$ then the approximate solution to the dual system is exact (see Exercise 3.18).

The second case, known as a *serious breakdown*, occurs when $w_k^T v_k = 0$, while neither v_{k+1} nor w_{k+1} is zero. In this situation, the required biorthogonality condition between the newly

generated vectors cannot be enforced, and the recurrence relations break down. Fortunately, several remedies have been developed that allow the algorithm to continue successfully in most cases. Such modified schemes are commonly referred to as *look-ahead Lanczos algorithms*.

Algorithm 11: Lanczos Bi-orthogonalization Algorithm

Input: Nonsymmetric matrix $A \in \mathbb{R}^{n \times n}$; $r_0 \in \mathbb{R}^n$; $\tilde{r}_0 \in \mathbb{R}^n$ with $r_0^T \tilde{r}_0 \neq 0$, dimension $m \leq n$

Output: Bi-orthonormal bases $\{v_1, \dots, v_{m+1}\}$ and $\{w_1, \dots, w_{m+1}\}$, so that $w_i^T v_j = \delta_{ij}$, and the entries α_j , β_j and δ_j of tridiagonal matrix T_m , and δ_m and β_m .

Set $v_0 = 0$, $w_0 = 0$, $\beta_1 = 0$, $\delta_1 = 0$, $v_1 = r_0 / \|r_0\|_2$, $w_1 = \tilde{r}_0 / \|\tilde{r}_0\|_2$

for $k = 1$ **to** m **do**

$$\alpha_k = w_k^T A v_k$$

$$v_{k+1} = A v_k - \alpha_k v_k - \beta_k v_{k-1}$$

$$w_{k+1} = A^T w_k - \alpha_k w_k - \delta_k w_{k-1}$$

$$\gamma_k = w_{k+1}^T v_{k+1}$$

if $\gamma_k = 0$ **then** *Break*

if $v_{k+1} = 0$ **or** $w_{k+1} = 0$ **then** *Break*

$$\delta_{k+1} = \|v_{k+1}\|_2 \text{ and } \beta_{k+1} = \gamma_k / \delta_{k+1}$$

$$v_{k+1} = v_{k+1} / \delta_{k+1} \text{ and } w_{k+1} = w_{k+1} / \beta_{k+1}$$

end for

Remark 3.5. Despite the symmetric Lanczos case, the basis $\{v_1, \dots, v_m\}$ is generally not orthogonal. The same is true for the basis $\{w_1, \dots, w_m\}$. Instead, the two sets satisfy a biorthogonality relation.

We now use the vectors $\{v_1, \dots, v_m\}$ and $\{w_1, \dots, w_m\}$ as bases for the trial and test spaces $\mathcal{K}_m(A; r_0)$ and $\mathcal{K}_m(A^T; \tilde{r}_0)$, respectively. Applying the Petrov–Galerkin approach to the nonsymmetric linear system $Ax = b$, the reduced system (3.37) takes the form

$$T_m y_m = \|r_0\|_2 \hat{e}_1, \quad (3.42)$$

since $W_m^T A V_m = T_m$ and $W_m^T r_0 = \|r_0\|_2 \hat{e}_1$, due to the fact that $r_0 = \|r_0\|_2 v_1$ and the bi-orthogonality of v_1 and w_j for $j = 1, \dots, m$.

Exercise 3.17. Using the second item of Theorem 3.18 and the system (3.42) show that

$$r_m = -\delta_{m+1} (e_m^T y_m) v_{m+1} \quad (3.43)$$

$$\|r_m\|_2 = \delta_{m+1} |y_m^{(m)}| \|v_{m+1}\|_2. \quad (3.44)$$

This shows that r_m is aligned with (parallel to) v_{m+1} , which means that $\{r_0, r_1, \dots, r_{m-1}\}$ forms a basis for $\mathcal{K}_m(A; r_0)$. In additions,

$$r_m \perp w_j, \quad j = 1, \dots, m. \quad (3.45)$$

Exercise 3.18. The Lanczos bi-orthogonalization algorithm solves not only the original system $Ax = b$ but also the dual system $A^T \tilde{x} = \tilde{b}$ for some vector $\tilde{b}$, if $\tilde{r}_0 = \tilde{b} - A^T \tilde{x}$. In this case, the solution $\tilde{x}_m$ comes from subspace $\tilde{x}_0 + \mathcal{K}_m(A^T; \tilde{r}_0)$, while the orthogonality

is enforced on subspace $\mathcal{K}_m(A; r_0)$. Show that $\tilde{x}_m = \tilde{x}_0 + W_m \tilde{y}_m$ where $\tilde{y}_m$ solves

$$T_m^T \tilde{y}_m = \|\tilde{r}_0\|_2 \hat{e}_1, \quad (3.46)$$

and the residual can be computed as

$$\tilde{r}_m = -\beta_{m+1}(e_m^T \tilde{y}_m) w_{m+1} \quad (3.47)$$

$$\|\tilde{r}_m\|_2 = \beta_{m+1} |\tilde{y}_m^{(m)}| \|w_{m+1}\|_2. \quad (3.48)$$

This shows that $\tilde{r}_m$ is parallel to w_{m+1} , which means that $\{\tilde{r}_0, \tilde{r}_1, \dots, \tilde{r}_{m-1}\}$ forms a basis for $\mathcal{K}_m(A^T; \tilde{r}_0)$. In additions,

$$\tilde{r}_m \perp v_j, \quad j = 1, \dots, m. \quad (3.49)$$

Exercise 3.19. From relations (3.43) and (3.47) conclude that

$$\tilde{r}_j^T r_m = 0, \quad j = 0, 1, \dots, m-1. \quad (3.50)$$

In order to write the solution x_m in terms of previous iterate x_{m-1} , we use the same idea as for the derivation of the CG algorithm from the symmetric Lanczos algorithm. Assume that T_m has an LU decomposition

$$T_m = \begin{bmatrix} \alpha_1 & \beta_2 & & 0 \\ \delta_2 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_m \\ 0 & & \delta_m & \alpha_m \end{bmatrix} = \begin{bmatrix} 1 & & & 0 \\ \ell_2 & 1 & & \\ & \ddots & \ddots & \\ 0 & & \ell_m & 1 \end{bmatrix} \begin{bmatrix} \theta_1 & \beta_2 & & 0 \\ & \theta_2 & \ddots & \\ & & \ddots & \beta_m \\ 0 & & & \theta_m \end{bmatrix} =: L_m U_m$$

where ℓ_j and θ_j are computed recursively as follows

$$\theta_1 = \alpha_1, \quad \ell_k = \frac{\delta_k}{\theta_{k-1}}, \quad \theta_k = \alpha_k - \ell_k \beta_k, \quad k = 2, \dots, m.$$

Now, we define the matrix $P_m = [p_0, p_1, \dots, p_{m-1}]$ as

$$P_m = V_m U_m^{-1}.$$

From (3.42), the solution is then expressed as

$$\begin{aligned} x_m &= x_0 + V_m y_m \\ &= x_0 + \rho_0 V_m (T_m^{-1} \hat{e}_1) \\ &= x_0 + \rho_0 V_m U_m^{-1} L_m^{-1} \hat{e}_1 \\ &= x_0 + \rho_0 P_m L_m^{-1} \hat{e}_1. \end{aligned}$$

Assume that $z_m = \rho_0 L_m^{-1} \hat{e}_1$. Since L_m is unit lower triangular, the vector $z_m = [\sigma_0, \dots, \sigma_{m-1}]^T$ satisfies the recurrence

$$\sigma_0 = \rho_0, \quad \sigma_k = -\ell_{k+1} \sigma_{k-1}, \quad k = 1, 2, \dots$$

Therefore,

$$x_m = x_0 + \sum_{j=0}^{m-1} \sigma_j p_j.$$

In particular, the iterates satisfy the recursive relation

$$x_{m+1} = x_m + \sigma_m p_m, \quad (3.51)$$

The vectors p_j are called the *search directions* and are computed using relation $P_m U_m = V_m$ recursively as

$$p_0 = \frac{1}{\theta_1} v_1, \quad p_k = \frac{1}{\theta_{k+1}} [v_{k+1} - \beta_{k+1} p_{k-1}], \quad k = 1, 2, \dots \quad (3.52)$$

If we further are intrested in solving the dual system $A^T \tilde{x} = \tilde{b}$, we similarly define the matrix $\tilde{P}_m = [\tilde{p}_0, \tilde{p}_1, \dots, \tilde{p}_{m-1}]$ as

$$\tilde{P}_m = W_m L_m^{-T}.$$

Then we can show that the solution of the reduced system (3.46) can be expressed as

$$\tilde{x}_{m+1} = \tilde{x}_m + \tilde{\sigma}_m \tilde{p}_m, \quad (3.53)$$

where $\tilde{\sigma}_j$ satisfies

$$\tilde{\sigma}_0 = \frac{\tilde{\rho}_0}{\theta_1}, \quad \tilde{\sigma}_k = -\frac{\beta_{k+1}}{\theta_{k+1}} \tilde{\sigma}_{k-1}, \quad k = 1, 2, \dots,$$

for $\tilde{\rho}_0 = \|\tilde{r}_0\|_2$. In additions, using $\tilde{P}_m L_m^T = W_m$, the search directions are recursively computed as

$$\tilde{p}_0 = w_1, \quad \tilde{p}_k = w_{k+1} - \ell_{k+1} \tilde{p}_{k-1}, \quad k = 1, 2, \dots \quad (3.54)$$

If we are only interested in solving the original system $Ax = b$, then we do not need to compute $\tilde{P}_m$ and iterations $\tilde{x}_m$. In this case the the initial residual $\tilde{r}_0$ is usually selected as $\tilde{r}_0 = r_0$.

There can be a *serious breakdown* in the bi-orthogonalization process before a good approximate solution is found. If this happens, the method has to be restarted with the new starting vector r_k . The Krylov subspaces that have been built are then discarded and the possibility of faster convergence wasted.

However, the algorithm is usually reformulated in a more efficient CG-like form, known as the BiCG algorithm [4, 1976], which we now derive. First, observe that the columns of P_m and $\tilde{P}_m$ are A -bi-orthogonal, i.e., $\tilde{p}_j^T A p_k = \delta_{jk}$, because

$$\tilde{P}_m^T A P_m = L_m^{-1} W_m^T A V_m U_m^{-1} = L_m^{-1} T_m U_m^{-1} = L_m^{-1} L_m U_m U_m^{-1} = I.$$

In the BiCG formulation, the vectors p_m and $\tilde{p}_m$ are obtained from scaled versions of (3.52) and (3.54). More precisely, we work with the scaled vectors

$$(c_{m+1} \theta_{m+1}) p_m, \quad d_{m+1} \tilde{p}_m,$$

where, according to (3.43) and (3.47), $c_{m+1} = -\delta_{m+1}y_m^{(m)}$ and $d_{m+1} = -\beta_{m+1}\tilde{y}_m^{(m)}$. These scaling factors are chosen so that the recurrences simplify to

$$p_m = r_m + \gamma_{m-1}p_{m-1}, \quad (3.55)$$

$$\tilde{p}_m = \tilde{r}_m + \gamma_{m-1}\tilde{p}_{m-1}, \quad (3.56)$$

where we continued to use the same notation p_m and $\tilde{p}_m$ after scaling. Under this scaling, the coefficients σ_m and $\tilde{\sigma}_m$ are scaled by $\sigma_m/(c_{m+1}\theta_{m+1})$ and $\tilde{\sigma}_m/d_{m+1}$, respectively, and we can use Lanczos relations and two last equations of systems (3.42) and (3.46) to show that

$$\frac{\sigma_m}{c_{m+1}\theta_{m+1}} = \frac{\tilde{\sigma}_m}{d_{m+1}}.$$

For simplicity we continue to use the same notation σ_m for both in the sequel. We note that, the iterates x_m , $\tilde{x}_m$ and the residuals r_m , $\tilde{r}_m$ remain unchanged under the scaling. Thus we have

$$x_{m+1} = x_m + \sigma_m p_m, \quad (3.57)$$

$$\tilde{x}_{m+1} = \tilde{x}_m + \sigma_m \tilde{p}_m. \quad (3.58)$$

In additions, from (3.57) and (3.58) we immediately obtain

$$r_{m+1} = r_m - \sigma_m A p_m, \quad (3.59)$$

$$\tilde{r}_{m+1} = \tilde{r}_m - \sigma_m A^T \tilde{p}_m. \quad (3.60)$$

Now, we compute parameters σ_m and γ_m using orthogonality and bi-orthogonality relations. In the Petrov–Galerkin framework,

$$r_{m+1} \perp \mathcal{K}_{m+1}(A^T; \tilde{r}_0),$$

and therefore

$$r_{m+1}^T \tilde{r}_j = 0, \quad j = 0, \dots, m,$$

since $\{\tilde{r}_0, \dots, \tilde{r}_m\}$ forms a basis for the test space $\mathcal{K}_{m+1}(A^T; \tilde{r}_0)$ (why? because of (3.47)). In particular,

$$r_{m+1}^T \tilde{r}_m = 0,$$

Using this relation and (3.59), we obtain

$$\sigma_m = \frac{\tilde{r}_m^T r_m}{\tilde{r}_m^T A p_m} = \frac{\tilde{r}_m^T r_m}{(\tilde{p}_m - \gamma_{m-1}\tilde{p}_{m-1})^T A p_m} = \frac{\tilde{r}_m^T r_m}{\tilde{p}_m^T A p_m},$$

where the last equality follows from the A -biorthogonality relation. To compute γ_m , we increase the index in (3.55) by one and multiply it from the left by $\tilde{r}_m^T$ and use the orthogonality relation $\tilde{r}_m^T r_{m+1} = 0$, to obtain

$$\gamma_m = \frac{\tilde{r}_m^T p_{m+1}}{\tilde{r}_m^T p_m}.$$

Using (3.60), the numerator becomes

$$\tilde{r}_m^T p_{m+1} = (\tilde{r}_{m+1} - \sigma_m A^T \tilde{p}_m)^T p_{m+1} = \tilde{r}_{m+1}^T p_{m+1} - 0 = \tilde{r}_{m+1}^T (r_{m+1} + \gamma_m p_m) = \tilde{r}_{m+1}^T r_{m+1},$$

where the last equality follows from the fact that $\tilde{r}_{m+1}$ is orthogonal to the test space $\mathcal{K}_{m+1}(A; r_0)$, which contains p_m . Similarly, from (3.55), the denominator can be rewritten as

$$\tilde{r}_m^T p_m = \tilde{r}_m^T (r_m + \gamma_{m-1} p_{m-1}) = \tilde{r}_m^T r_m,$$

since $\tilde{r}_m$ is orthogonal to the test space $\mathcal{K}_m(A; r_0)$, which contains p_{m-1} . Consequently,

$$\gamma_m = \frac{\tilde{r}_{m+1}^T r_{m+1}}{\tilde{r}_m^T r_m}.$$

In summary, the Lanczos biorthogonalization process gives the following BiCG algorithm.

Algorithm 12: Bi-Conjugate Gradient (BiCG) method for solving $Ax = b$

Input: Matrix $A \in \mathbb{R}^{n \times n}$; vector $b \in \mathbb{R}^n$; initial vector x_0 ; tolerance ε ; maximum iteration m_{max} .

Output: Number of iterations m_{iter} ; final solution $x_{m_{iter}}$; relative residual norm $relres$ of final solution; indicator $flag$.

Set $r_0 = b - Ax_0$, $\tilde{r}_0 = r_0$, $p_0 = r_0$, $\tilde{p}_0 = \tilde{r}_0$, $relres = 1$, $flag = 0$, $m = 0$.

while $relres > \varepsilon$ **and** $m < m_{max}$ **do**

 Compute $\sigma_m = (\tilde{r}_m^T r_m) / (\tilde{p}_m^T A p_m)$

 Update $x_{m+1} = x_m + \sigma_m p_m$

 Update $r_{m+1} = r_m - \sigma_m A p_m$

 Update $\tilde{r}_{m+1} = \tilde{r}_m - \sigma_m A^T \tilde{p}_m$

 Compute $\gamma_m = (\tilde{r}_{m+1}^T r_{m+1}) / (\tilde{r}_m^T r_m)$

 Update $p_{m+1} = r_{m+1} + \gamma_m p_m$

 Update $\tilde{p}_{m+1} = \tilde{r}_{m+1} + \gamma_m \tilde{p}_m$

 Compute $relres = \|r_{m+1}\|_2 / \|b\|_2$

 Set $m = m + 1$

end while

Set $m_{iter} = m$

if $m_{iter} = m_{max}$ **then** $flag = 1$

This algorithm requires only two matrix–vector products (one with A and the other with A^T), three vector inner products (plus an additional inner product if we want to compute the residual norm), five vector updates, and five scalar–vector products, per iteration. Note that, in the computation of γ_m , the inner product $\tilde{r}_m^T r_m$ has already been computed in the previous iteration and therefore does not need to be computed again.

The BiCG method avoids the long recurrences and storage growth encountered Arnoldi–Galerkin and GMRES methods, since it is implemented using short term recurrences. However, it may suffer from irregular convergence behavior.

3.14 Quasi-MINRES (QMR) algorithm

A natural question is whether one can combine the biorthogonal Lanczos factorization with a minimum residual approach in order to obtain a MINRES-like method for nonsymmetric systems. The answer is positive, and the resulting algorithm is called the *quasi-minimal residual* (QMR) method [5, 1991].

From item 2 of Theorem 3.18, we have

$$AV_m = V_{m+1}\tilde{T}_m, \quad \tilde{T}_m = \begin{bmatrix} T_m \\ \delta_{m+1}\hat{e}_m^T \end{bmatrix},$$

where $\tilde{T}_m$ is an $(m+1) \times m$ tridiagonal matrix. Setting $\rho_0 = \|r_0\|_2$, the residual associated with the approximation

$$x_m = x_0 + V_m y_m$$

can be written as

$$r_m = r_0 - AV_m y_m = \rho_0 v_1 - V_{m+1} \tilde{T}_m y_m = V_{m+1}(\rho_0 \hat{e}_1 - \tilde{T}_m y_m),$$

similar to the GMRES derivation in Section 3.8. The difference here is that V_{m+1} is generally not unitary, and therefore $\|r_m\|_2 \neq \|\rho_0 \hat{e}_1 - \tilde{T}_m y_m\|_2$. Nevertheless,

$$\|r_m\|_2 \leq \|V_{m+1}\|_2 \|\rho_0 \hat{e}_1 - \tilde{T}_m y_m\|_2.$$

Moreover, if in the biorthogonal Lanczos process we scale the vectors so that $\delta_{k+1} = \|\tilde{v}_{k+1}\|_2$, as in Algorithm 11, then $\|v_{k+1}\|_2 = 1$ for all $k \geq 0$, and hence

$$\|V_{m+1}\|_2 \leq \|V_{m+1}\|_F \leq \sqrt{m+1}.$$

In QMR, the vector y_m is chosen as the solution of the least-squares problem

$$\min_{y \in \mathbb{R}^m} \|\rho_0 \hat{e}_1 - \tilde{T}_m y\|_2,$$

analogously to the MINRES method. Since this procedure minimizes only an upper bound for $\|r_m\|_2$, rather than the residual norm itself, the method is called *quasi*-minimum residual, or simply QMR.

Because the subdiagonal entries of $\tilde{T}_m$ are nonzero (otherwise the biorthogonal Lanczos process would terminate by a happy breakdown), the least-squares problem always has a unique solution, even if the tridiagonal matrix T_m is singular. Thus QMR avoids the *serious breakdown* that may occur in the BiCG algorithm.

The iterates $x_m = x_0 + V_m y_m$ are updated using the same QR–Givens strategy employed in MINRES, now applied to the tridiagonal matrix $\tilde{T}_m$ obtained from the nonsymmetric Lanczos algorithm 11. We leave the details and the algorithms as an exercise to readers.

Exercise 3.20. Apply the same QR–Givens strategy employed in MINRES to write the QMR algorithm. Also show that

$$\|r_m\|_2 \leq \sqrt{m+1} \cdot |s_1 s_2 \cdots s_m| \cdot \|r_0\|_2$$

where s_j are Givens parameters associated to Givens rotations G_j .

Experiments show that although QMR has a much smoother convergence behaviour than BiCG, it is not essentially faster than BiCG.

3.15 Transpose-free methods

For nonsymmetric systems, Krylov methods based on the Arnoldi process require only matrix-vector products with A . In contrast, methods derived from the Lanczos biorthogonalization process require products with both A and A^T , which can be disadvantageous in some applications. In particular, the data structures or discretizations arising from differential equations may allow efficient computation of Ax while making $A^T x$ considerably more expensive. This has motivated the development of *transpose-free* Lanczos-based methods.

The conjugate gradient squared (CGS) method [15, 1989] is a transpose-free variant of the BiCG algorithm that avoids matrix-vector products with A^T . However, compared with BiCG, the often irregular convergence behavior of the residual can be amplified in CGS. There also exist transpose-free variants of the QMR algorithm.

Another widely used transpose-free method is BiCGSTAB [18, 1992], which was designed to also reduce the erratic convergence behavior of CGS. Although BiCGSTAB performs well for many problems and typically converges more smoothly than CGS, its convergence may still become irregular for difficult problems, and in some cases it can be significantly slower than CGS.

4 Preconditioned iterative methods

As we pointed out earlier in Section 2.4, preconditioners are essential for obtaining fast and reliable convergence of iterative methods. Recall that the *left-preconditioned system* has the form

$$M^{-1}Ax = M^{-1}b, \quad (4.1)$$

while the *right-preconditioned system* is given by

$$AM^{-1}y = b, \quad y = Mx. \quad (4.2)$$

In many situations, it is desirable that the preconditioned system preserves important structural properties of the original matrix A . For example, if A is symmetric positive definite and we want to still use the CG method, then the preconditioning must preserve symmetry and positive definiteness. In this case we form a *symmetric preconditioned system*, which transforms the original system into

$$L^{-1}AL^{-T}y = L^{-1}b, \quad y = L^T x, \quad (4.3)$$

where $M = LL^T$. The matrix L may be chosen as the Cholesky factor of M , the symmetric square root of M , or any nonsingular matrix satisfying $M = LL^T$. If A and M are symmetric positive definite, then so is $L^{-1}AL^{-T}$. In all cases, the preconditioner M is chosen so that

1. M approximates A sufficiently well,
2. linear systems of the form $Mv = d$ are inexpensive to solve.

The first requirement is directly related to the convergence rate of the iterative method. Recall that Krylov subspace methods generate residuals of the form $r_m = p_m(A)r_0$, where

p_m is a polynomial of degree m satisfying $p_m(0) = 1$. Hence, convergence depends on how effectively the polynomial p_m damps the spectral components associated with the eigenvalues of A . If A is replaced by the preconditioned matrix $M^{-1}A$ (or AM^{-1} or $L^{-1}AL^{-T}$), and if M is a good approximation of A , then the eigenvalues of the preconditioned matrix tend to be more favorably distributed, often clustered around 1 and far from zero which leads to significantly faster convergence.

Note that the matrices $M^{-1}A$, AM^{-1} and $L^{-1}AL^{-T}$ have the same eigenvalues, and therefore their asymptotic convergence behavior is often expected to be similar. However, eigenvalues alone do not fully determine the convergence of Krylov subspace methods, especially for nonsymmetric systems, where factors such as nonnormality also play an important role. One important difference is that, in the right-preconditioned formulation, the true residual $r_m = b - Ax_m$ is computed directly and is therefore readily available for monitoring convergence. While the left-preconditioned formulation naturally monitors the preconditioned residual $M^{-1}r_m$, whose norm may differ significantly from that of the true residual.

The second requirement implies that the cost of applying the preconditioner remains reasonable. In practice, the products $M^{-1}A$ and AM^{-1} are never formed explicitly. Instead, iterative methods compute matrix-vector products with A and apply the action of M^{-1} separately by solving systems of the form $Mv = d$. Therefore, a preconditioner is useful only if these solves can be carried out much more cheaply than solving the original system $Ax = b$ directly.

The efficiency of a preconditioned iterative method is thus determined by a balance between the improved convergence obtained from the preconditioner and the computational cost required to apply it at each iteration. In Section 4.5, we present a brief overview of several important classes of preconditioners. Before discussing their construction, however, we first examine how preconditioning is incorporated into Krylov subspace methods such as CG, GMRES, and BiCG.

4.1 Preconditioned CG algorithm

In practical applications, the CG method is used together with a suitable preconditioner M . However, the left-preconditioned system

$$M^{-1}Ax = M^{-1}b$$

is generally no longer symmetric (and positive definite). Consequently, the standard CG algorithm cannot be applied directly to this system. The same holds true for a right-preconditioner. To preserve the structure required by the CG method, it is natural to choose a symmetric positive definite preconditioner M . Using the Cholesky factorization

$$M = LL^T,$$

the preconditioned system can be rewritten in the equivalent form

$$L^{-1}AL^{-T}\tilde{x} = L^{-1}b, \quad \tilde{x} = L^Tx. \quad (4.4)$$

The matrix $\tilde{A} = L^{-1}AL^{-T}$ is again symmetric positive definite (prove it!) and therefore the CG algorithm 7 can be safely applied to the transformed system with A replaced by $\tilde{A}$ and b by $\tilde{b} = L^{-1}b$. When $\tilde{x}$ is computed, the solution x is recovered by solving $L^Tx = \tilde{x}$.

However, the more efficient implementation is to avoid forming Cholesky factor L and matrix $\tilde{A}$ explicitly and carry out the operations in the original coordinate system. If we apply the CG algorithm on preconditioned system (4.4) and use $\tilde{x}$, $\tilde{r}$, and $\tilde{p}$ as new notations, we will have

$$\begin{aligned}\tilde{x}_{m+1} &= \tilde{x}_m + \alpha_m \tilde{p}_m, & \alpha_m &= \frac{\tilde{r}_m^T \tilde{r}_m}{\tilde{p}_m^T (L^{-1} A L^{-T}) \tilde{p}_m}, \\ \tilde{r}_{m+1} &= \tilde{r}_m - \alpha_m (L^{-1} A L^{-T}) \tilde{p}_m, \\ \tilde{p}_{m+1} &= \tilde{r}_{m+1} + \beta_m \tilde{p}_m.\end{aligned}$$

If we define $x = L^{-T} \tilde{x}$, $r = L \tilde{r}$ and $p = L^{-T} \tilde{p}$, we obtain the following preconditioned CG algorithm for solving $Ax = b$ with symmetric positive definite preconditioner M .

Algorithm 13: Preconditioned Conjugate Gradient (PCG) method for solving $Ax = b$

Input: Symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$; vector $b \in \mathbb{R}^n$; symmetric positive definite preconditioner M ; initial vector x_0 ; tolerance ε ; maximum iteration m_{max} .

Output: Number of iterations m_{iter} ; final solution $x_{m_{iter}}$; relative residual norm $relres$ of final solution; indicator $flag$.

Set $r_0 = b - Ax_0$, $relres = 1$, $flag = 0$, $m = 0$

Solve $Mz_0 = r_0$ and set $p_0 = z_0$

while $\|r_m\|_2 > \varepsilon$ **and** $m < m_{max}$ **do**

 Compute $\alpha_m = (r_m^T z_m) / (p_m^T A p_m)$

 Update $x_{m+1} = x_m + \alpha_m p_m$

 Update $r_{m+1} = r_m - \alpha_m A p_m$

 Solve $Mz_{m+1} = r_{m+1}$

 Compute $\beta_m = (r_{m+1}^T z_{m+1}) / (r_m^T z_m)$

 Update $p_{m+1} = z_{m+1} + \beta_m p_m$

 Compute $relres = \|r_{m+1}\|_2 / \|b\|_2$

 Set $m = m + 1$

end while

Set $m_{iter} = m$

if $m_{iter} = m_{max}$ **then** $flag = 1$

If the preconditioner is given in incomplete Cholesky factored form as $M = LL^T$ then L can be used directly in place of M . In this case, the vectors z_j are computed efficiently by forward and backward substitutions.

For the CG method, as observed in the convergence analysis, it is not necessary for the preconditioner M to approximate A so closely that $M^{-1}A \approx I$. Rather, the preconditioner is particularly effective when the preconditioned matrix $M^{-1}A$ has only a few distinct eigenvalues, or when it has a few isolated large eigenvalues while the remaining eigenvalues are tightly clustered.

4.2 Preconditioned GMRES algorithm

For GMRES and other Krylov subspace methods for nonsymmetric systems, all three forms of preconditioning (left, right, and two-sided) can be applied.

In the left-preconditioned formulation, the iterations are applied to the system $M^{-1}Ax = M^{-1}b$. Accordingly, the initial residual becomes

$$z_0 = M^{-1}(b - Ax_0),$$

and the Arnoldi process is started with z_0 instead of $r_0 = b - Ax_0$. The main additional cost associated with the preconditioner arises in the Arnoldi process. At each iteration, after computing Av_m , one must solve the system

$$Mw = Av_m.$$

Thus, applying the preconditioner amounts to solving one linear system with matrix M per iteration, similar to the CG algorithm. The method then monitors the norm of the preconditioned residuals, $\|z_m\|_2 = \|M^{-1}(b - Ax_m)\|_2$ which is available at no additional cost, and terminates when the prescribed tolerance is reached. If desired, the norm of the true residual $\|r_m\|_2 = \|b - Ax_m\|_2$ can also be monitored. This requires one additional matrix-vector product and one inner product to compute the norm.

Algorithm 14: Left-preconditioned GMRES algorithm.

Input: $A \in \mathbb{R}^{n \times n}$; vector b ; initial guess x_0 ; tolerance ε ; maximum iteration m_{max} ; preconditioner M .

Output: Number of iterations m_{iter} ; final solution $x_{m_{iter}}$; relative residual norm $relres$ of final solution; indicator $flag$.

Set $r_0 = b - Ax_0$

Solve $Mz_0 = r_0$; $\rho_0 = \|z_0\|_2$, $v_1 = z_0/\rho_0$, $relres = 1$, $flag = 0$, $m = 1$

while $relres > \varepsilon$ **and** $m \leq m_{max}$ **do**

 % Arnoldi

 Solve $Mw = Av_m$

for $j = 1$ **to** m **do**

$h_{j,m} = v_j^T w$

$w = w - h_{j,m}v_j$

end for

$h_{m+1,m} = \|w\|_2$

if $h_{m+1,m} = 0$ **then** *Break*

$v_{m+1} = w/h_{m+1,m}$

 Update matrices $\tilde{H}_m$ and V_m

 % GMRES

 Solve $\min_y \|\tilde{H}_m y - \rho_0 \hat{e}_1\|_2$ using QR factorization for solution y_m

 Compute $x_m = x_0 + V_m y_m$

 Compute $relres = \|z_m\|_2 / \|M^{-1}b\|_2$

 Set $m = m + 1$

end while

Set $m_{iter} = m - 1$

if $m_{iter} = m_{max}$ **then** $flag = 1$

The right-preconditioned GMRES method is applied to the system

$$AM^{-1}\tilde{x} = b, \quad \tilde{x} = Mx.$$

In practice, however, the auxiliary variable $\tilde{x}$ need not be formed explicitly. Indeed, if x_0 is an initial guess for the original system, then the corresponding initial guess for the preconditioned system is $\tilde{x}_0 = Mx_0$, and the associated residual is

$$b - AM^{-1}\tilde{x}_0 = b - Ax_0 = r_0.$$

Thus, the Arnoldi process starts directly from the original residual r_0 . At the m th iteration, GMRES computes an approximation of the form

$$\tilde{x}_m = \tilde{x}_0 + V_m y_m,$$

where $V_m = [v_1, \dots, v_m]$ is an orthonormal basis of the Krylov subspace $\mathcal{K}_m(AM^{-1}; r_0)$. Multiplying by M^{-1} yields

$$x_m = x_0 + M^{-1}V_m y_m.$$

Therefore, rather than applying the preconditioner to the initial residual as in the left-preconditioned formulation, the preconditioning operation is incorporated into the basis vectors generated during the Arnoldi process. Since Arnoldi is applied to the operator AM^{-1} , each iteration requires the computation $w = AM^{-1}v_m$, or $w = Au_m$ where $u_m = M^{-1}v_m$. Then the approximate solution can be written as

$$x_m = x_0 + U_m y_m, \quad U_m = [u_1, \dots, u_m].$$

An important advantage of right preconditioning is that the residual of the preconditioned system coincides with the true residual of the original system. The resulting right-preconditioned GMRES algorithm is given below. This algorithm is also called *flexible* GMRES (FGMRES).

Algorithm 15: Right-preconditioned GMRES algorithm.

Input: $A \in \mathbb{R}^{n \times n}$; vector b ; initial guess x_0 ; tolerance ε ; maximum iteration m_{max} ; preconditioner M .

Output: Number of iterations m_{iter} ; final solution $x_{m_{iter}}$; relative residual norm $relres$ of final solution; indicator $flag$.

Set $r_0 = b - Ax_0$; $\rho_0 = \|r_0\|_2$, $v_1 = r_0/\rho_0$, $relres = 1$, $flag = 0$, $m = 1$

while $relres > \varepsilon$ **and** $m \leq m_{max}$ **do**

 % Arnoldi

 Solve $Mu_m = v_m$ and set $w = Au_m$

for $j = 1$ **to** m **do**

$h_{j,m} = v_j^T w$

$w = w - h_{j,m}v_j$

end for

$h_{m+1,m} = \|w\|_2$

if $h_{m+1,m} = 0$ **then** *Break*

$v_{m+1} = w/h_{m+1,m}$

 Update matrices $\tilde{H}_m$ and U_m

 % GMRES

 Solve $\min_y \|\tilde{H}_m y - \rho_0 \hat{e}_1\|_2$ using QR factorization for solution y_m

 Compute $x_m = x_0 + U_m y_m$

 Compute $relres = \|r_m\|_2/\|b\|_2$

 Set $m = m + 1$

end while

Set $m_{iter} = m - 1$

if $m_{iter} = m_{max}$ **then** $flag = 1$

Exercise 4.1. In some cases, M is the result of a factorization of the form $M = LU$. Then there is an option to apply GMRES on two-sided preconditioned system

$$L^{-1}AU^{-1}\tilde{x} = L^{-1}b, \quad x = U^{-1}\tilde{x}.$$

Drive the preconditioned GMRES process and write down the algorithm.

4.3 Preconditioned MINRES algorithm

Since MINRES is designed for symmetric linear systems, standard left or right preconditioning destroys symmetry and is therefore not applicable. Instead, one applies a two-sided preconditioner and solves the equivalent system

$$L^{-1}AL^{-T}\tilde{x} = L^{-1}b, \quad x = L^{-T}\tilde{x},$$

where L is a nonsingular matrix for which linear systems with coefficient matrix L (and L^T) can be solved cheaply. Here, the matrix L need not be triangular and is not necessarily a Cholesky factor.

Given an initial guess x_0 for the original system, the corresponding initial guess for the preconditioned system is $\tilde{x}_0 = L^T x_0$. To obtain the starting vector for the Lanczos process, we solve

$$Lz_0 = r_0,$$

where $r_0 = b - Ax_0$. The Lanczos process is then applied to the preconditioned matrix $L^{-1}AL^{-T}$ with starting vector z_0 . Consequently, the first step of the Lanczos iteration in Algorithm 10 is replaced by

$$\begin{aligned} &\text{Solve } L^T u_m = v_m \text{ for } u_m, \\ &\text{Solve } Lw = Au_m \text{ for } w, \\ &w = w - \beta_{m-1}v_{m-1}. \end{aligned}$$

The remaining steps of the MINRES iteration are unchanged. After computing an approximate solution $\tilde{x}$ of the preconditioned system, the corresponding approximation to the original system is obtained by solving

$$L^T x = \tilde{x}.$$

An alternative and a few less costly implementation computes approximations to the original solution x directly during the iteration. In this case, the algorithm is initialized with x_0 , while the Lanczos process still starts from z_0 . Furthermore, in the recurrence for the search directions p_{m-1} in Algorithm 10, the vector v_m is replaced by u_m . This modification generates approximations to the solution of the original system without explicitly forming $\tilde{x}$ (why?). This implementation avoids the final solve $L^T x = \tilde{x}$.

In either implementation, the residual norms produced by MINRES satisfy $\|z_m\|_2 = |\tau_m| \cdot \|z_0\|_2$, and correspond to the residuals of the preconditioned system rather than those of the original system.

Exercise 4.2. Using the discussion above, derive a complete algorithm for preconditioned MINRES with input preconditioner factor L .

4.4 Preconditioned BiCG algorithm

For the BiCG method, preconditioning algorithm simultaneously uses both A and its transpose A^T and the preconditioner must be applied to both systems. The standard formulation is obtained by applying left preconditioning to $Ax = b$ and to its transpose, $A^T \tilde{x} = \tilde{b}$, which yields

$$M^{-1}Ax = M^{-1}b, \quad A^T M^{-T} \tilde{x} = M^{-T} \tilde{b}.$$

Let $r_m = b - Ax_m$ and $\tilde{r}_m = r_m$ denote the residuals associated with these two systems. We introduce the preconditioned residuals

$$z_m = M^{-1}r_m, \quad \tilde{z}_m = M^{-T}\tilde{r}_m.$$

The preconditioned BiCG method is obtained from the BiCG recurrences by replacing the residuals r_m and $\tilde{r}_m$ with the preconditioned residuals z_m and $\tilde{z}_m$ in the computation of the search directions and recurrence coefficients. Starting from an initial guess x_0 , we set $r_0 = b - Ax_0$ and $\tilde{r}_0 = r_0$, and compute

$$Mz_0 = r_0, \quad M^T \tilde{z}_0 = \tilde{r}_0.$$

The initial search directions are then chosen as $p_0 = z_0$, and $\tilde{p}_0 = \tilde{z}_0$. At iteration m , the BiCG coefficient is computed as

$$\sigma_m = \frac{\tilde{z}_m^T r_m}{\tilde{p}_m^T A p_m},$$

and the iterates and residuals are updated according to

$$x_{m+1} = x_m + \sigma_m p_m, \quad r_{m+1} = r_m - \sigma_m A p_m, \quad \tilde{r}_{m+1} = \tilde{r}_m - \sigma_m A^T \tilde{p}_m.$$

The new residuals are then preconditioned by solving

$$Mz_{m+1} = r_{m+1}, \quad M^T \tilde{z}_{m+1} = \tilde{r}_{m+1}.$$

Finally, the coefficient

$$\gamma_m = \frac{\tilde{z}_{m+1}^T r_{m+1}}{\tilde{z}_m^T r_m}$$

is used to update the search directions

$$p_{m+1} = z_{m+1} + \gamma_m p_m, \quad \tilde{p}_{m+1} = \tilde{z}_{m+1} + \gamma_m \tilde{p}_m.$$

Compared with the unpreconditioned BiCG method, each iteration requires one solve with M and one solve with M^T . This additional cost is often offset by a substantial reduction in the number of iterations required for convergence.

Exercise 4.3. Write down the algorithm of the preconditioned BiCG method.

We can derive preconditioned versions of the other Krylov subspace methods in similar ways. We leave these derivations as exercises for the reader.

4.5 Different types of preconditioners

Designing an effective preconditioner is often the most challenging aspect of solving large sparse linear systems iteratively. Preconditioners can be broadly classified into two categories. The first category consists of *algebraic preconditioners*, which are constructed solely from the matrix A without reference to the underlying physical problem that generated the linear system. The second category consists of *problem-dependent preconditioners*, which use additional information about the mathematical model, differential operator, or discretization underlying the system $Ax = b$.

4.5.1 Algebraic preconditioners

Algebraic preconditioners are attractive because they can be applied to a wide range of problems with little additional information. Two types of such preconditioners are given below.

Classical iteration splittings. As discussed in Section 2.4, the matrices arising from classical iterative methods can be interpreted as preconditioners for the Richardson iteration. These matrices can also be used as preconditioners for Krylov subspace methods. Let

$$A = D - F - E,$$

where D is the diagonal part of A , F is the strictly lower triangular part, and E is the strictly upper triangular part. Several preconditioners can then be derived from classical matrix splittings.

Method	Preconditioner M
Jacobi	D
Gauss–Seidel	$D - F$
SOR	$D - \omega F$
Symmetric Gauss–Seidel (SGS)	$(D - F)D^{-1}(D - E)$
Symmetric SOR (SSOR)	$(D - \omega F)D^{-1}(D - \omega E)$

Table 3: Preconditioners obtained from classical matrix splittings. Here $\omega \in (0, 2)$ is a relaxation constant.

These preconditioners are inexpensive to construct and sometimes improve convergence substantially. The Jacobi preconditioner is simple because applying M^{-1} requires only diagonal scaling, but it is less effective. For Gauss–Seidel and SOR preconditioners, applying M^{-1} involves solving triangular systems by forward substitution. Symmetric Gauss–Seidel and SSOR require both forward and backward substitutions and are used for symmetric systems.

More generally, many large sparse matrices can be reordered and partitioned into blocks,

$$A = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1p} \\ A_{21} & A_{22} & \cdots & A_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ A_{p1} & A_{p2} & \cdots & A_{pp} \end{bmatrix}$$

where the diagonal blocks A_{ii} are square. A simple preconditioning strategy is to use its block diagonal part

$$M = \begin{bmatrix} A_{11} & & \\ & \ddots & \\ & & A_{pp} \end{bmatrix},$$

as a preconditioner, which is called *block Jacobi preconditioner*. Or one can use a block triangular approximation which then is called *block Gauss-Seidel preconditioner*. The resulting preconditioner captures the dominant local interactions while neglecting weaker couplings between different blocks.

Although these preconditioners may not highly reduce the condition number, they often cluster the eigenvalues of the preconditioned matrix more favorably, which is sufficient to accelerate Krylov subspace methods.

Incomplete LU (ILU) preconditioners. A more sophisticated class of algebraic preconditioners is obtained from incomplete matrix factorizations. Suppose that A is sparse. In the exact LU factorizations many entries of A that are initially zero become nonzero during the elimination process. These newly created nonzero entries are called *fill-in*. Instead of computing the exact factorization $A = LU$, we can compute incomplete factors $\tilde{L}$ and $\tilde{U}$ with a prescribed sparsity structure such that the factorization error

$$E = A - \tilde{L}\tilde{U},$$

is small. Then we use

$$P = \tilde{L}\tilde{U}$$

as the preconditioner. For symmetric positive definite matrices, the corresponding approach is the incomplete Cholesky factorization $A = \tilde{L}\tilde{L}^T + E$. The effectiveness of ILU preconditioners depends on their controlling on the excessive fill-in associated with exact Gaussian elimination. For example, one can only compute those entries of $\tilde{L}$ and $\tilde{U}$ which are associated to nonzero entries of A . In general, we assume that the sparsity pattern is given by the index set

$$\mathcal{S} = \{(i, j) : a_{ij} \neq 0\} \subset \{(i, j) : 1 \leq i, j \leq n\},$$

where the diagonal indices (i, i) are always included in $\mathcal{S}$. Then the ILU factorization algorithm can be formulated as follows.

Algorithm 16: Incomplete LU (ILU) factorization with a prescribed nonzero pattern

Input: $A \in \mathbb{R}^{n \times n}$; prescribed sparsity pattern $\mathcal{S}$

Output: Incomplete factors L and U where L is stored below the diagonal and U on and above the diagonal of A .

```

for  $k = 1$  to  $n - 1$  do
  for  $i = k + 1$  to  $n$  and if  $(i, k) \in \mathcal{S}$  do
     $a_{ik} = a_{ik} / a_{kk}$ 
    for  $j = k + 1$  to  $n$  and if  $(i, j) \in \mathcal{S}$  do
       $a_{ij} = a_{ij} - a_{ik}a_{kj}$ 
    end for
  end for
end for

```

There are some variants of Algorithm 16 obtained by reordering the loops. Some of these variants are more efficient for sparse matrices because they better exploit the sparsity structure and reduce unnecessary computations.

The ILU factorization defined by Algorithm 16 does not necessarily exist for an arbitrary matrix A . An important class of matrices for which the factorization is well defined and the algorithm is generally robust is the class of nonsingular M -matrices [13]. A nonsingular matrix A is called an M -matrix if its off-diagonal entries are nonpositive,

$$a_{ij} \leq 0, \quad i \neq j,$$

and its inverse is component-wise nonnegative,

$$A^{-1} \geq 0.$$

Such matrices arise naturally in the discretization of elliptic PDEs and in the analysis of Markov chains.

The incomplete factorization with no fill-in, denoted by ILU(0), is obtained by taking the sparsity pattern $\mathcal{S}$ to be exactly the nonzero pattern of A . Since no additional fill-in is permitted, the resulting factors remain sparse and inexpensive to store and apply. ILU(0) is an effective preconditioner for matrices arising from low-order discretizations of elliptic PDEs, particularly when the coefficient matrix is an M -matrix or is diagonally dominant.

For more challenging problems, however, ILU(0) may provide too crude an approximation of A . In such cases, it is better to allow a controlled amount of fill-in beyond the original sparsity pattern. This leads to the family of *level-of-fill incomplete factorizations*, denoted by ILU(p), where p is a nonnegative integer.

The simplest extension is ILU(1), whose sparsity pattern consists of the original nonzero pattern of A together with all fill-in entries generated during the construction of ILU(0). Higher fill levels are obtained recursively by allowing additional fill-in generated at the preceding level. More details see [13, 2003].

As the fill level increases, the incomplete factors provide a more accurate approximation to the exact LU factors. This improves the accuracy of the preconditioner and reduces the number of Krylov iterations required for convergence. The improved convergence, however, comes at the expense of increased storage and computational cost. In practice, ILU(1) often yields a substantially better approximation to A than ILU(0), while the additional cost remains moderate. For this reason, ILU(1) is one of the most widely used incomplete factorization preconditioners.

Similar to the the full LU factorization, an incomplete LU (ILU) factorization does not exist for every nonsingular matrix. In such cases, one may employ ILU factorization with *pivoting*. We do not pursue this topic further here. In MATLAB, an incomplete LU factorization can be computed using

```
[L,U] = ilu(A,options)
```

For example, an incomplete LU factorization with threshold pivoting can be obtained by setting the `type` field of `options` to `ilutp`. Other choices of options may produce a

factorization in which the columns of U or the rows of L are permuted to improve numerical stability (pivoting). We refer the reader to the MATLAB documentation on `ilu` for a detailed description of the available options and their effects.

4.5.2 Problem-dependent preconditioners

When knowledge of the underlying problem is available, more effective preconditioners can often be constructed by simplifying the original model while preserving its dominant features. The guiding principle is the following: if a simplified version of the problem can be solved efficiently and still captures the essential behavior of the original system, then that simplified problem may serve as an effective preconditioner. Following [17], here we give some examples of such preconditioners.

Coarse-grid approximations. Many discretized PDEs exhibit low-frequency error components that are reduced only slowly by standard iterative methods. A coarse-grid approximation can efficiently capture these global error modes by solving a related problem on a much coarser mesh. By transferring information between the fine and coarse grids using suitable restriction and interpolation operators, one obtains a preconditioner of the schematic form

$$M = T_{C \rightarrow F} \circ A_C \circ T_{F \rightarrow C},$$

where $T_{F \rightarrow C}$ and $T_{C \rightarrow F}$ denote the restriction and interpolation operators, respectively, and A_C is the discretization matrix on the coarse grid. The action of M approximately eliminates the low-frequency components of the error, while the Krylov subspace iteration effectively reduces the remaining high-frequency components. This idea forms the basis of multigrid methods, which are among the most powerful preconditioning techniques for elliptic problems.

Low-order discretization preconditioners. For a given differential equation, lower-order discretizations usually produce sparser and less expensive operators than higher-order discretizations on the same grid. These lower-order operators often retain the essential features of the original problem and can therefore be used as preconditioners for the corresponding higher-order discretizations.

More generally, the low-order discretization may be constructed either on the same grid or different grids, in which case suitable transfer operators are used to move information between the two discretizations. For example, one can use a low-order finite difference or finite element discretizations as preconditioners for high-order finite element, spectral element, or spectral methods.

Local approximations. In some applications the global coupling in the system is weak, and local interactions dominate the behavior of the solution. A preconditioner can then be constructed by neglecting long-range couplings and solving a collection of local subproblems. Such approximations are often inexpensive and highly parallelizable.

Domain decomposition. A closely related class of preconditioner to block preconditioners is provided by *domain decomposition* techniques. It is used when A represents a PDE equation (such as Poisson's equation) on a computational domain Ω . Then, the domain Ω is partitioned into smaller subdomains Ω_j so that $\Omega = \cup \Omega_j$, and the original problem is solved on each subdomain independently. Solving each of these subproblems corresponds to a block diagonal M . This method is also called *Schur complement method* in literature. Sometimes, a few

overlap between subdomains is allowed for which we then have a few interactions between the blocks. Because the local subproblems can be solved independently, domain decomposition methods are naturally suited for parallel computing. They form the basis of many scalable solvers for large sparse systems arising from the discretization of PDEs.

Simpler forms of the problem. When the coefficient matrix arises from a variable-coefficient or nonsymmetric differential operator, a simpler approximation may be obtained by replacing variable coefficients with constants or by retaining only the symmetric part of the operator. Such approximations are often easier to invert while still capturing the dominant behavior of the original system, so can be used as effective preconditioners.

Operator splittings. Some operator equations such as PDE operators can be split into simpler components such that the resulting linear algebra system has the form

$$A = A_1 + \cdots + A_k.$$

If the individual matrices A_i can be inverted efficiently, then preconditioners can be constructed from these simpler pieces. Examples include alternating-direction implicit (ADI) methods and various additive or multiplicative Schwarz techniques.

Finally we note that, the choice of a preconditioner depends strongly on the underlying problem and the available computational resources. No universally optimal preconditioner exists. Nevertheless, the most successful preconditioners share a common philosophy: they approximate the original operator sufficiently well to improve convergence while remaining significantly easier to apply than solving the original system itself.

5 Numerical examples

We use some test matrices from *SuiteSparse Matrix Collection*¹ (formerly known as the University of Florida Sparse Matrix Collection) to evaluate the iterative methods we already derived for solving $Ax = b$. This collection is a large repository of sparse matrices arising from a wide range of scientific and engineering applications. It provides a standard set of benchmark problems for testing sparse direct and iterative solvers, preconditioners, and also eigensolvers [3, 2011].

The collection can be accessed in MATLAB through the package `ssget`, which must first be downloaded from its GitHub repository² and added to the MATLAB search path. Once installed, matrices can be loaded using commands such as

```
Problem = ssget('HB/west0479');  
A = Problem.A;
```

which downloads and loads the matrix `west0479`. You can also access to a few matrices of this collection from MATLAB matrix gallery.

¹<https://sparse.tamu.edu/>

²<https://github.com/DrTimothyAldenDavis/SuiteSparse>

Similarly, in Python, you first install the package `ssgetpy` which provides access to the collection and allows matrices to be downloaded and imported into SciPy sparse matrix formats. For example,

```
from ssgetpy import search
from scipy.io import mmread

# Find and download west0479
mat = search(name='west0479')[0]
mat.download()

# Load the Matrix Market file
A = mmread('HB/west0479/west0479.mtx').tocsr()
```

downloads the same matrix, which is finally loaded into a SciPy sparse matrix.

5.1 Symmetric (in)definite systems

As we outlined in Exercise 3.8, the convergence of the CG method depends only on the spectrum of A and it is sufficient to study its behavior on diagonal matrices with positive diagonal entries. As a first test example, we consider the matrices

```
n = 20;
A = spdiags(linspace(1, n, n)', 0, n, n);
b = ones(n,1);
L = spdiags(sqrt([1; linspace(3, 19, n-1)'])), 0, n, n);
```

Here, A is a diagonal matrix whose eigenvalues range uniformly from 1 to 20. We also define a diagonal preconditioner through its factor L , as shown above. Figure 4 compares the convergence histories of the CG and preconditioned CG methods. The stopping tolerance is chosen as $\varepsilon = 10^{-14}$, and the maximum number of iterations is set to $m_{\max} = n = 20$.

The right panel of Figure 4 displays the eigenvalue distributions of both A and the preconditioned matrix $L^{-1}AL^{-T}$. The eigenvalues of A (blue points) are spread over the interval $[1, 20]$. This results in a slower convergence. In contrast, the eigenvalues of the preconditioned matrix (red points) are much more tightly clustered, lying approximately between $1/3$ and 1.05 , with most of them concentrated near 1. This improved clustering leads to a faster convergence observed in the left panel. Using a better preconditioner leads to faster convergence. For instance, if the value 3 in the definition of L is replaced by 2, the resulting preconditioned system has eigenvalues that are even more tightly clustered around 1. Consequently, the method converges even faster.

Another observation is that at iteration $m = n = 20$, the CG method recovers the exact solution up to roundoff errors. This finite termination property, together with the effect of eigenvalue clustering on the convergence rate, is in agreement with Theorems 3.9 and 3.11.

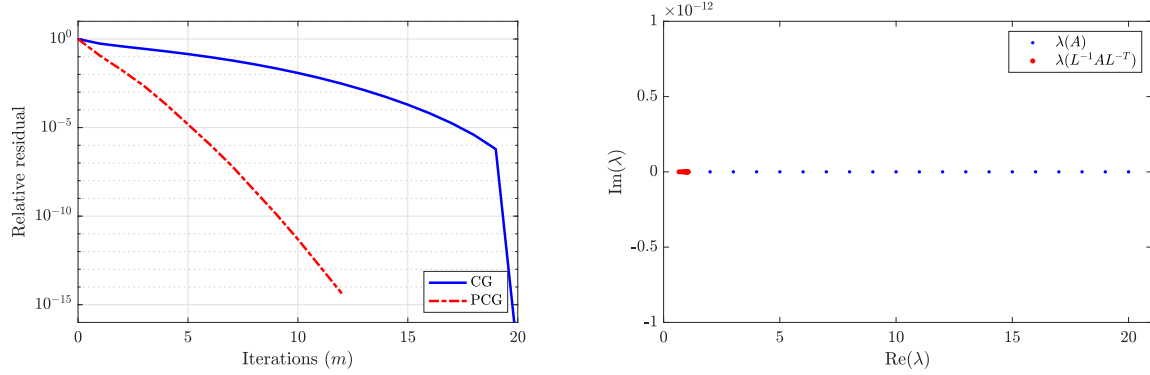


Figure 4: Convergence of CG and preconditioned CG algorithms (left), distribution of eigenvalues of the original matrix and the preconditioned matrix (right).

We next test the CG and MINRES algorithms on the matrix **bcsstk14** from the SuiteSparse Matrix Collection. This matrix arises as a stiffness matrix in a finite element discretization of a structural engineering problem. It is sparse, symmetric positive definite, and has dimension 1806×1806 . Both CG and MINRES are applicable here. The convergence histories are shown in Figure 5. In the absence of preconditioning, neither method converges. We use an incomplete Cholesky factorization with no additional fill-in as preconditioner for both methods. Since the incomplete factorization may encounter zero pivots, we add a small diagonal shift of 0.01 to the matrix. In MATLAB, this is accomplished by

```
Problem = ssget('HB/bcsstk14');
A = Problem.A;
opts.type = 'nofill'; opts.diagcomp = 0.01;
L = ichol(A, opts);
b = ones(size(A,1), 1)
```

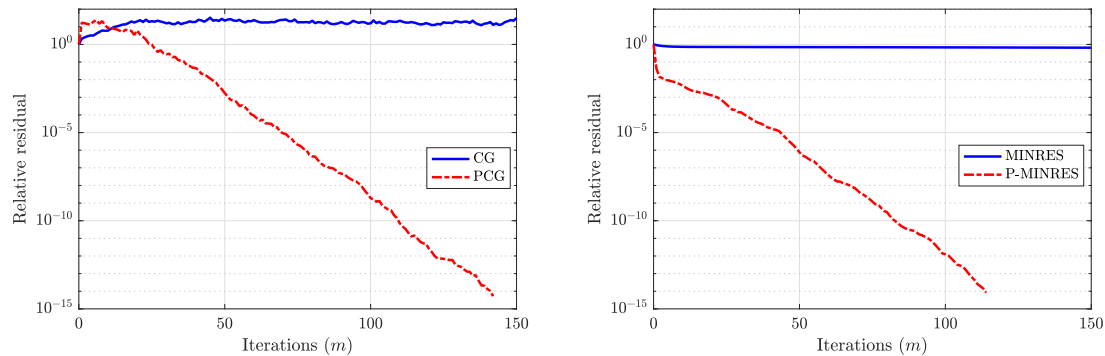


Figure 5: Convergence histories of CG and MINRES with and without incomplete Cholesky preconditioning for the matrix **bcsstk14** of size $n = 1806$.

The incomplete Cholesky preconditioner significantly accelerates convergence for both methods. In this particular example, preconditioned MINRES reaches the prescribed tol-

erance after approximately 114 iterations, whereas preconditioned CG requires about 142 iterations. Although MINRES converges faster in terms of iteration count for this problem, each CG iteration is computationally less expensive.

The collection contains a folder named `GHS_indef`, which consists of symmetric indefinite matrices. As a test problem, we consider the matrix `copter2` ($= C$), which has dimension $n = 55,476$ and arises from the discretization of a computational fluid dynamics (CFD) model of a helicopter rotor blade. We assume that $A = C + 5I$ and consider solving $Ax = b$ for $b = [1, 1, \dots, 1]^T$ as a test system for comparing MINRES, and Lanczos–Galerkin (LanGal) methods (see Section A.1) without or with preconditioning. The matrix A is still indefinite. We employ the modified symmetric Gauss–Seidel preconditioner

$$M = (D - E)|D|^{-1}(D - E)^T,$$

where $-E$ denotes the strictly lower triangular part of A , D is the diagonal matrix of A , and $|D|$ is absolute value of D . The corresponding factor

$$L = (D - E)|D|^{-1/2}$$

is supplied to the algorithms as the L factor of the preconditioner. The use of $|D|$ instead of D guarantees that the square root is well defined, since some diagonal entries of A are negative.

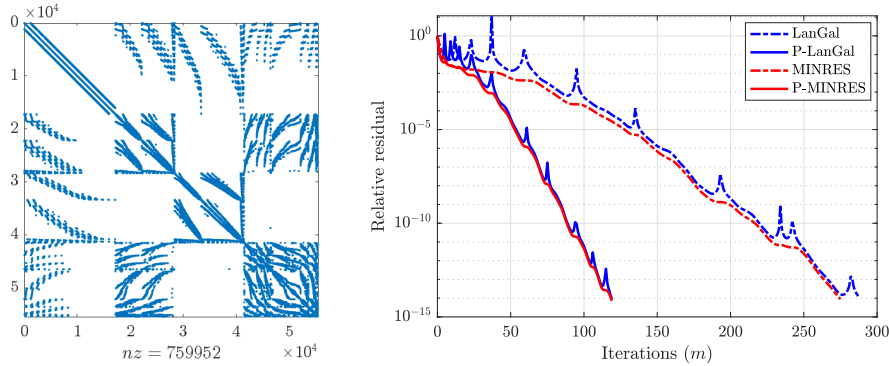


Figure 6: Left: sparsity pattern of the matrix `copter2`. Right: convergence histories of the LanGal, preconditioned LanGal, MINRES and preconditioned MINRES methods.

The stopping tolerance is set to $\varepsilon = 10^{-14}$ and the maximum number of iterations to $m_{\max} = 500$. The left panel of Figure 6 shows the sparsity pattern of the matrix `copter2`, while the right panel displays the convergence histories of the MINRES and Lanczos–Galerkin (LanGal) methods, both with and without preconditioning. The residual curve of MINRES lies consistently below that of LanGal, which is expected since MINRES minimizes the residual norm at each iteration. Moreover, the MINRES convergence history is much smoother, whereas the LanGal residual has several sudden increases during the iteration process. The preconditioner significantly improves the performance of both methods by reducing the number of iterations required to achieve a given accuracy by approximately a factor of two. A more effective preconditioner would lead to even faster convergence.

Although LanGal may perform as good as MINRES for some problems, MINRES is generally the more robust and reliable method for symmetric indefinite systems.

5.2 Nonsymmetric systems

We now compare the Arnoldi–Galerkin (ArGal), GMRES, and BiCG methods on two nonsymmetric test problems from the collection. The coefficient matrices are chosen as `cavity10` and `raefsky3`. The matrix `cavity10` has dimension $n = 2597$ and arises from the discretization of the driven cavity problem in CFD at zero Reynolds number. The matrix `raefsky3` has dimension $n = 21200$ and originates from the discretization of a fluid–structure interaction (FSI) problem involving turbulence. Both matrices are sparse and nonsymmetric, although their sparsity patterns are nearly symmetric, as shown in Figure 7.

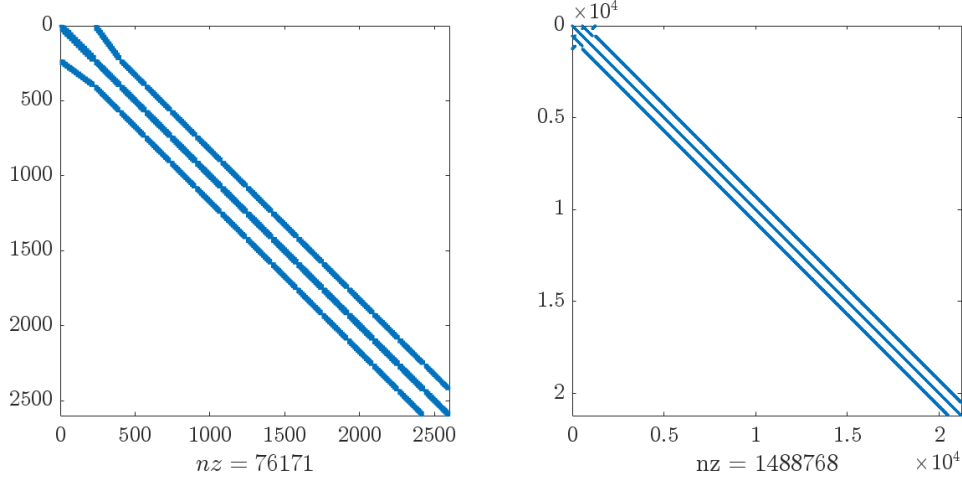


Figure 7: Sparsity patterns of the test matrices: left: `cavity10` ($n = 2597$); right: `raefsky3` ($n = 21200$).

For both problems, the stopping tolerance is set to $\varepsilon = 10^{-14}$. We employ an ILU(0) preconditioner (incomplete LU factorization with no additional fill-in). Without preconditioning, convergence of all three methods is very slow for the cavity system, while none of them converges for the FSI problem. The convergence histories obtained with ILU(0) preconditioning are shown in Figure 8.

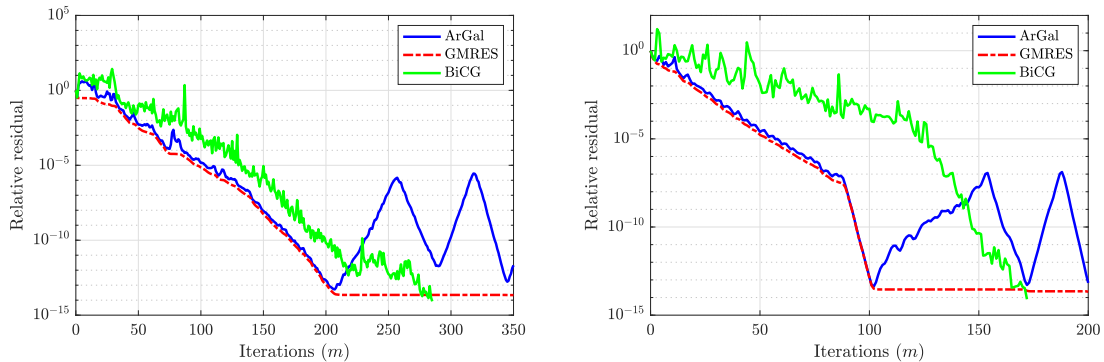


Figure 8: Convergence histories of the Arnoldi–Galerkin (ArGal), GMRES, and BiCG methods with ILU(0) as preconditioner. Left: `cavity10` system; right: `raefsky3` system.

The residuals of ArGal and GMRES are nearly similar for both problems down to a level of approximately 10^{-13} . The ArGal residuals show small oscillations and once the residual norm reaches this level, the oscillations become more and the residual may increase substantially at some iterations. In contrast, GMRES maintains a much smoother convergence history and remains stable near the attainable accuracy. The performance of ArGal and GMRES are almost the same (while GMRES residuals are smoother) if tolerance 10^{-13} or larger is used.

The BiCG method also converges for both problems, but its residual curve is highly oscillatory. Such behavior is frequently observed in BiCG and reflects the fact that the method does not minimize the residual norm. This was one of the motivations for the development of BiCGSTAB, which produces a smoother and more regular convergence history in most problems. This method also avoids the need for matrix–transpose products. We do not include numerical results for BiCGSTAB here, but the interested reader is encouraged to compare its performance with that of BiCG on the same test problems.

The role of preconditioner is crucial for the success of Krylov subspace methods. For example, allowing additional fill-in in the ILU factorization yields faster convergence for all three methods above. This improvement, however, comes at the cost of increased storage requirements and a more expensive setup phase. Such additional cost is often justified when ILU(0) is insufficient to produce satisfactory convergence. As an example, if we replace `cavity10` by `cavity26`, which corresponds to a finer discretization of the driven cavity problem with Reynolds number 1000, then ILU(0) provides no improvement in convergence. In this case, a more powerful preconditioner, such as ILU with higher levels of fill-in or a problem-specific preconditioner, becomes necessary.

In general, for large and challenging applications, problem-dependent preconditioners are often significantly more effective than purely algebraic preconditioners.

We again note that the full GMRES and full ArGal methods become computationally expensive when preconditioner is weak and many iterations are required, since increasing the Krylov subspace dimension m leads to higher storage requirements.

Finally, we apply the restarted methods ArGal(k) and GMRES(k) to the same test problems using the ILU(0) preconditioner. Four restart parameters $k = 25, 50, 75, 100$ are used. For each value of k , the maximum number of cycles is chosen as $j_{\max} = \lceil 1000/k \rceil$ so that all runs are allowed to perform approximately 1000 iterations if convergence is not achieved earlier.

The results are shown in Figure 9. The residual curves of GMRES(k) are considerably smoother than those of ArGal(k). This behavior is expected, since GMRES minimizes the Euclidean norm of the residual over the current Krylov subspace, and the quantity plotted in the figure is precisely this norm. For this reason, GMRES is generally preferred over ArGal for nonsymmetric problems. Its convergence behavior is more robust and predictable, particularly when high accuracy is required.

The effect of the restart parameter is also evident. Larger values of k generally lead to faster convergence because more information from the Krylov subspace is retained between restarts. However, this improvement comes at the expense of increased storage requirements and higher orthogonalization costs. Choosing an appropriate restart parameter therefore involves balancing convergence speed against computational and memory costs.

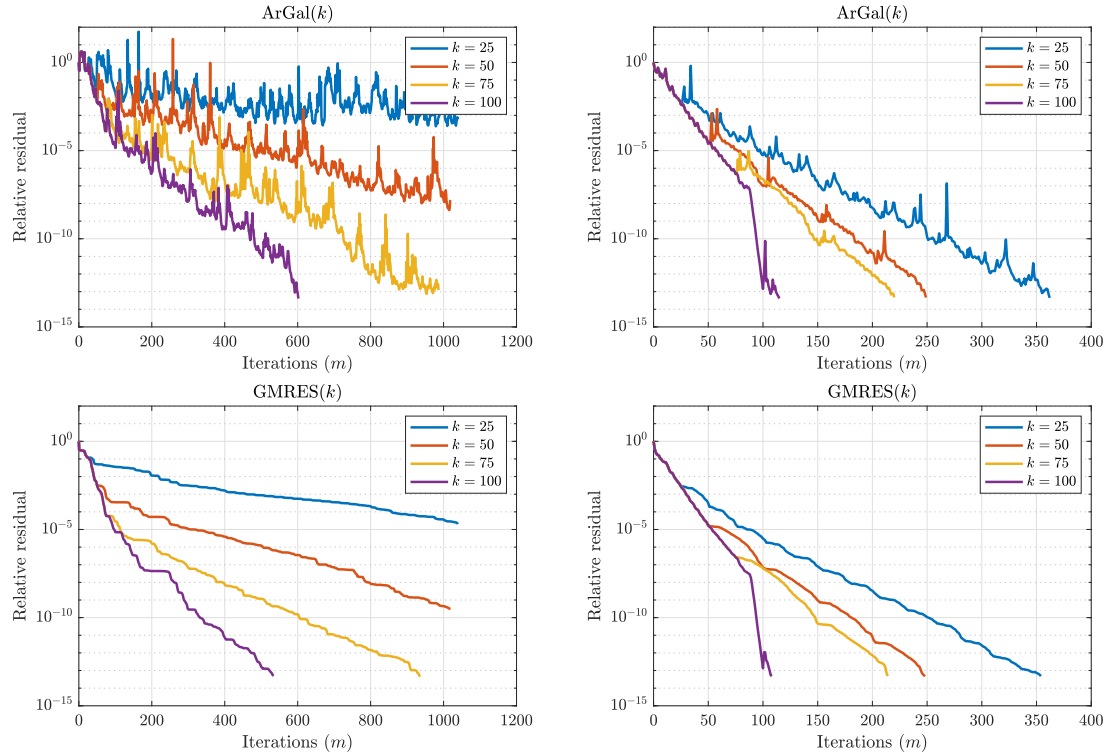


Figure 9: Convergence histories of the restarted ArGal and GMRES methods with ILU(0) preconditioning. Top row: ArGal(k); bottom row: GMRES(k). Left: cavity10 system; right: raefsky3 system.

Remark 5.1. Built-in Krylov subspace solvers in MATLAB and Python. In all of the above examples, the iterative algorithms were implemented using custom MATLAB codes written by the author. However, most scientific computing environments provide built-in implementations of Krylov subspace methods for solving large sparse linear systems.

In MATLAB, the main iterative solvers include `pcg` (preconditioned CG), `gmres`, `bicg`, `bicgstab`, `bicgstabl`, `cgs`, `minres`, `qmr`, `lsqr`, and `tfqmr` (transpose-free QMR). The function `pcg` is intended for symmetric positive definite systems, while `minres` is suitable for symmetric indefinite systems. For nonsymmetric systems, commonly used choices are `gmres`, `bicgstab`, `qmr`, and `tfqmr`. MATLAB solvers also allow left and right preconditioners to be supplied as matrices or function handles^a.

In Python, Krylov subspace methods are available in the module `scipy.sparse.linalg`. This module includes solvers such as `cg`, `gmres`, `bicg`, `bicgstab`, `cgs`, `minres`, `qmr`, `lgmres`, and `tfqmr`. These solvers can be applied to sparse matrices or to matrix-free operators defined through `LinearOperator`. Preconditioners can also be supplied^b.

^a<https://www.mathworks.com/help/matlab/math/iterative-methods-for-linear-systems.html>

^b<https://docs.scipy.org/doc/scipy/reference/sparse.linalg.html>

5.3 Which method?

Numerical experiments show that when A is symmetric positive definite, the CG method is usually the method of first choice. When A is symmetric but possibly indefinite, MINRES is often preferred over other Krylov subspace methods because of its robustness and favorable convergence properties.

For nonsymmetric systems, however, the situation is less clear. There is no single Krylov subspace method that consistently outperforms all others for all problems. The effectiveness of a method depends strongly on the properties of the underlying matrix, such as its spectrum, nonnormality, conditioning, and the availability of suitable preconditioners. Consequently, a method that performs exceptionally well for one problem may converge slowly or even fail on another. Nevertheless, GMRES and its restarted variant GMRES(k) are among the most widely used methods due to their robustness, while stabilized BiCG variants, such as BiCGSTAB, and QMR are often preferred when lower memory requirements or cheaper iterations are desired.

6 Eigenvalue computations for large sparse matrices

In the previous lectures we considered the power method and the Francis algorithm (QR algorithm) with and without shifts for computing eigenvalues and eigenvectors. The power method is inexpensive and only requires matrix–vector products, but it normally computes only one eigenvalue at a time and its convergence may be slow. On the other hand, the Francis algorithm is a powerful method for computing all eigenvalues of a dense matrix, but it is not appropriate when A is very large and sparse. Even if A is initially sparse, the transformations involved in the Francis algorithm generally destroy the sparsity.

In many large-scale applications we do not need all eigenvalues. For example, we may only be interested in a few eigenvalues of largest magnitude, the smallest eigenvalues of a symmetric matrix, or the eigenvalues close to a prescribed point. In this situation Krylov subspace methods are particularly attractive. They require mainly matrix–vector products and therefore preserve and exploit the sparsity of A .

Recall that for a nonzero vector v ,

$$\mathcal{K}_m(A; v) = \text{span}\{v, Av, A^2v, \dots, A^{m-1}v\}$$

denotes the Krylov subspace generated by A and v . The power method uses only the latest vector $A^{m-1}v$, while Krylov subspace methods use the information contained in the whole subspace $\mathcal{K}_m(A; v)$. The idea is to project the large eigenvalue problem onto this much smaller subspace and solve the resulting $m \times m$ eigenvalue problem.

We consider the eigenvalue problem

$$Ax = \lambda x, \quad x \in \mathbb{C}, \quad x \neq 0$$

for $A \in \mathbb{R}^{n \times n}$, and assume that $x_1, \dots, x_n$ are eigenvectors of A corresponding to eigenvalues $\lambda_1, \dots, \lambda_n$. In this section we do not consider the general case $A \in \mathbb{C}^{n \times n}$ but the algorithms can be modified slightly to work for this general case as well.

6.1 Rayleigh–Ritz projection

Let $v_0 \neq 0$ be the initial vector and set

$$v_1 = \frac{v_0}{\|v_0\|_2}.$$

Let $V_m = [v_1, \dots, v_m] \in \mathbb{R}^{n \times m}$ have orthonormal columns forming a basis of $\mathcal{K}_m(A; v_0)$. We seek an approximate eigenvector of A in this subspace. We approximate an eigenvector of A by a vector $u \in \mathcal{K}_m(A; v_0)$. Thus, we write

$$u = V_m z,$$

where $z \in \mathbb{R}^m$, and apply the Galerkin idea which requires the residual

$$r = Au - \theta u$$

to be orthogonal to $\mathcal{K}_m(A; v_0)$. This gives

$$V_m^T (AV_m z - \theta V_m z) = 0.$$

Since $V_m^T V_m = I$, we obtain the small eigenvalue problem

$$V_m^T A V_m z = \theta z.$$

Thus, instead of solving an $n \times n$ eigenvalue problem, we solve an $m \times m$ problem, where normally $m \ll n$.

Definition 6.1. An eigenvalue θ of the projected matrix

$$H_m = V_m^T A V_m$$

is called a *Ritz value* of A with respect to $\mathcal{K}_m(A; v_0)$. If z is a corresponding normalized eigenvector of H_m , then

$$u = V_m z$$

is called the corresponding *Ritz vector*. The pair (θ, u) is called a *Ritz pair*.

Note that

$$\|u\|_2 = \|V_m z\|_2 = \|z\|_2 = 1$$

when z is normalized. A Ritz pair is accepted as an approximation to an eigenpair of A when the residual

$$\|r\|_2 = \|Au - \theta u\|_2$$

is sufficiently small. This procedure is known as the *Rayleigh–Ritz method*. As we observed in Section 3, we can use Arnoldi algorithm (or Lanczos algorithm for symmetric A) to efficiently construct a good orthonormal basis V_m .

6.2 Arnoldi method for the eigenvalue problem

Recall the Arnoldi decomposition

$$AV_m = V_m H_m + h_{m+1,m} v_{m+1} \hat{e}_m^T, \quad (6.1)$$

where

$$H_m = V_m^T A V_m$$

is an $m \times m$ upper Hessenberg matrix. Suppose that

$$H_m z = \theta z, \quad \|z\|_2 = 1.$$

The associated Ritz vector is

$$u = V_m z.$$

Using (6.1), we obtain

$$\begin{aligned} Au - \theta u &= AV_m z - \theta V_m z \\ &= V_m H_m z + h_{m+1,m} v_{m+1} \hat{e}_m^T z - \theta V_m z \\ &= h_{m+1,m} (\hat{e}_m^T z) v_{m+1}. \end{aligned}$$

Hence,

$$\|Au - \theta u\|_2 = |h_{m+1,m}| |\hat{e}_m^T z|$$

because $\|v_{m+1}\|_2 = 1$. This is an important relation. It means that the residual norm can be computed without explicitly forming the vector $Au - \theta u$. Both quantities on the right-hand side are already available from the Arnoldi process and the small eigenvalue problem. We may therefore use, for example, the stopping criterion

$$|h_{m+1,m}| |\hat{e}_m^T z| \leq \varepsilon (|\theta| + 1).$$

The resulting procedure is summarized below.

Algorithm 17: Arnoldi method for computing a few eigenpairs

Input: Matrix $A \in \mathbb{R}^{n \times n}$; nonzero starting vector v_0 ; Krylov dimension m ; tolerance ε .

Output: Approximate eigenvalues and eigenvectors of A .

- Apply the Arnoldi algorithm with starting vector v_0 to compute $V_{m+1} = [v_1, \dots, v_{m+1}]$ and the Hessenberg matrix $\tilde{H}_m$.
- Form the leading $m \times m$ part H_m of $\tilde{H}_m$.
- Compute the eigenpairs

$$H_m z_j = \theta_j z_j, \quad j = 1, \dots, m.$$

- Select the desired Ritz values θ_j , for example those of largest magnitude.
- For every selected θ_j , compute the Ritz vector

$$u_j = V_m z_j.$$

- Compute the residual estimate

$$\rho_j = |h_{m+1,m}| |e_m^T z_j|.$$

- Accept (θ_j, u_j) as converged if

$$\rho_j \leq \varepsilon (|\theta_j| + 1).$$

The eigenvalue problem for Hessenberg matrix H_m is small and can be solved by the Francis algorithm developed earlier. Thus, Arnoldi combines two ideas: a Krylov subspace is first used to reduce the dimension, and the Francis algorithm is then applied only to the small projected matrix. As we said, the eigenvalues θ_j and eigenvectors z_j of H_m are called Ritz values and Ritz vectors. Some of them are observed to converge rapidly and when they do, we may assume θ_j and $u_j = V_m z_j$ for $j = 1, \dots, m$ are good approximations of eigenvalues and eigenvectors of A . Since $m \ll n$ for a feasible computation, one can not expect to compute all of the eigenvalues/vectors of A using this process.

If the Arnoldi process terminates with

$$h_{m+1,m} = 0,$$

then (6.1) reduces to

$$AV_m = V_m H_m.$$

Therefore, $\mathcal{K}_m(A; v_0)$ is an invariant subspace of A , and every eigenvalue of H_m is an exact eigenvalue of A . Similar to the solution of linear systems, this is called a *happy breakdown*.

There is a connection with the power method. It can be seen the vector used by the power method at step m , i.e. $A^{m-1}v_0$ is contained in subspace $\mathcal{K}_m(A; v_0) = \text{span}\{v_0, Av_0, \dots, A^{m-1}v_0\}$. Arnoldi uses all the information in this space instead of using only the latest vector $A^{m-1}v_0$. As a result, several eigenvalues can be approximated simultaneously, and convergence can be substantially faster.

6.3 Lanczos method for symmetric matrices

If A is symmetric, then the Arnoldi matrix $H_m = V_m^T A V_m$ is also symmetric. Since it is upper Hessenberg, it must be tridiagonal. We denote it by

$$T_m = \begin{bmatrix} \alpha_1 & \beta_1 & & & \\ \beta_1 & \alpha_2 & \ddots & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & \beta_{m-1} \\ & & & \beta_{m-1} & \alpha_m \end{bmatrix}.$$

We now solve the symmetric tridiagonal eigenvalue problem

$$T_m z_j = \theta_j z_j.$$

The Ritz vectors are again

$$u_j = V_m z_j.$$

We similarly have

$$Au_j - \theta_j u_j = \beta_m (\tilde{e}_m^T z_j) v_{m+1},$$

and hence

$$\|Au_j - \theta_j u_j\|_2 = |\beta_m| |\tilde{e}_m^T z_j|.$$

The Lanczos eigensolver is therefore inexpensive. Each Lanczos step requires one multiplication with A , a few vector operations, and only the previous two Lanczos vectors are needed to generate the next vector.

Algorithm 18: Lanczos method for computing eigenpairs of a symmetric matrix

Input: Symmetric matrix $A \in \mathbb{R}^{n \times n}$; nonzero starting vector v ; dimension m ; tolerance ε .

Output: Approximate eigenvalues and eigenvectors.

- Apply the symmetric Lanczos algorithm to construct V_{m+1} and the tridiagonal matrix T_m .
 - Compute the eigenpairs $T_m z_j = \theta_j z_j$.
 - Select the desired Ritz values θ_j .
 - Compute $u_j = V_m z_j$.
 - Compute $\rho_j = |\beta_m| |e_m^T z_j|$.
 - Accept (θ_j, u_j) if $\rho_j \leq \varepsilon(|\theta_j| + 1)$.
-

Although only three Lanczos vectors are required to continue the recurrence, the matrix V_m is needed if the Ritz vectors $u_j = V_m z_j$ are required. If only eigenvalues are sought, it is therefore possible to reduce the memory requirements considerably.

6.4 Polynomial interpretation of the Arnoldi method

The Arnoldi process also has a useful interpretation in terms of polynomials. This viewpoint provides another way to understand the role of the Ritz values and also reveals a close connection between Arnoldi and Krylov methods for linear systems such as GMRES.

Recall that the Arnoldi process constructs v_{m+1} by orthogonalizing Av_m against $\mathcal{K}_m(A; v_0)$. Equivalently, since v_m is a polynomial in A applied to v_0 , the new direction can be regarded as the component of $A^m v_0$ orthogonal to $\mathcal{K}_m(A; v_0)$. Thus there exists a vector $w_m \in \mathcal{K}_m(A; v_0)$ such that

$$A^m v_0 - w_m \perp \mathcal{K}_m(A; v_0), \quad (6.2)$$

and, up to normalization,

$$v_{m+1} \propto A^m v_0 - w_m.$$

Since $w_m \in \mathcal{K}_m(A; v_0)$, it can be written as

$$w_m = -a_{m-1}A^{m-1}v_0 - \cdots - a_1Av_0 - a_0v_0.$$

Therefore

$$A^m v_0 - w_m = p_m(A)v_0,$$

where

$$p_m(t) = t^m + a_{m-1}t^{m-1} + \cdots + a_1t + a_0$$

is a monic polynomial of degree m . Hence the Arnoldi orthogonality condition can be written as

$$p_m(A)v_0 \perp \mathcal{K}_m(A; v_0). \quad (6.3)$$

This orthogonality condition has a minimization interpretation. Consider the set of monic polynomials of degree m ,

$$\mathbb{P}_{m+1}^{(1)} = \{p(t) = t^m + a_{m-1}t^{m-1} + \cdots + a_0\}.$$

For any $p \in \mathbb{P}_{m+1}^{(1)}$,

$$p(A)v_0 = A^m v_0 + a_{m-1}A^{m-1}v_0 + \cdots + a_0v_0.$$

Since the last m terms can represent an arbitrary vector in $\mathcal{K}_m(A; v_0)$, we have

$$\min_{p \in \mathbb{P}_{m+1}^{(1)}} \|p(A)v_0\|_2 = \min_{w \in \mathcal{K}_m(A; v_0)} \|A^m v_0 - w\|_2.$$

The right-hand side is an orthogonal projection problem. Its minimizer is therefore characterized precisely by

$$A^m v_0 - w_m \perp \mathcal{K}_m(A; v_0),$$

which is the Arnoldi condition (6.3). Consequently,

$$p_m = \arg \min_{p \in \mathbb{P}_{m+1}^{(1)}} \|p(A)v_0\|_2. \quad (6.4)$$

The following theorem, adapted from [17, 1997], identifies the minimizing polynomial p_m .

Theorem 6.1. If the Arnoldi process has not broken down ($h_{j,j+1} \neq 0$ for $j = 1, \dots, m$), the solution of the minimization problem (6.4) is $p_m = \chi_m$ where χ_m is the characteristic polynomial of Arnoldi matrix $H_m = V_m^T A V_m$.

Proof. Since $v_0 = \|v_0\|_2 v_1$, condition (6.3) is equivalent to

$$V_m^T p_m(A) v_1 = 0.$$

For any polynomial p of degree at most m , the Arnoldi relation implies

$$V_m^T p(A) v_1 = p(H_m) \hat{e}_1. \quad (6.5)$$

Indeed, for $k = 0, \dots, m-1$,

$$A^k v_1 = V_m H_m^k \hat{e}_1,$$

whereas for $k = m$ an additional component in the direction v_{m+1} appears. This component vanishes after multiplication by V_m^T , since $V_m^T v_{m+1} = 0$. It follows that

$$V_m^T A^k v_1 = H_m^k \hat{e}_1, \quad k = 0, \dots, m,$$

which gives (6.5). Applying (6.5) to p_m gives

$$p_m(H_m) \hat{e}_1 = 0. \quad (6.6)$$

Since the Arnoldi process has not broken down, H_m is an unreduced (proper) upper Hessenberg matrix. Consequently,

$$\hat{e}_1, H_m \hat{e}_1, \dots, H_m^{m-1} \hat{e}_1$$

are linearly independent. Thus $\hat{e}_1$ is a cyclic vector for H_m , and no nonzero polynomial of degree less than m can annihilate $\hat{e}_1$. Therefore there is a unique monic polynomial of degree m satisfying (6.6). Now let

$$\chi_m(t) = \det(H_m - tI)$$

be the characteristic polynomial of H_m . It is monic of degree m , and the Cayley–Hamilton theorem gives $\chi_m(H_m) = 0$. In particular,

$$\chi_m(H_m) \hat{e}_1 = 0.$$

By uniqueness, the minimizing polynomial must therefore be the characteristic polynomial. $\square$

This result can also be viewed as a polynomial filtering process. Suppose, for simplicity, that A is normal, with orthonormal eigenvectors x_k , and write

$$v_0 = \sum_{k=1}^n c_k x_k, \quad Ax_k = \lambda_k x_k.$$

Then

$$p(A)v_0 = \sum_{k=1}^n c_k p(\lambda_k) x_k,$$

and hence

$$\|p(A)v_0\|_2^2 = \sum_{k=1}^n |c_k|^2 |p(\lambda_k)|^2.$$

Therefore the Arnoldi polynomial satisfies

$$\chi_m = \arg \min_{p \in \mathbb{P}_{m+1}^{(1)}} \sum_{k=1}^n |c_k|^2 |p(\lambda_k)|^2.$$

In this sense, Arnoldi constructs a polynomial that is small at the eigenvalues of A , with the importance of each eigenvalue weighted by the contribution of its eigenvector to the initial vector v_0 . The roots of this optimal polynomial are exactly the Ritz values.

The polynomial p acts as a filter on the eigendirections. Suppose, for example, that we wish to approximate the eigenvector x_1 associated with λ_1 . After normalizing the polynomial so that $p(\lambda_1) = 1$, a useful polynomial filter should satisfy

$$|p(\lambda_k)| \ll 1, \quad k = 2, \dots, n.$$

Then $p(A)v_0 \approx c_1 x_1$ preserves the component in the direction x_1 while suppressing the unwanted eigendirections. This explains why the distribution of the eigenvalues strongly influences the convergence of Arnoldi. Convergence can be fast when the desired eigenvalue can be separated from the rest of the spectrum by a low-degree polynomial.

For Hermitian matrices, this polynomial picture is clearer because the eigenvalues are real and the eigenvectors are orthogonal. Eigenvalues near the ends of the spectral interval, or eigenvalues well separated from the remaining spectrum, are often easier to isolate by low-degree polynomials than eigenvalues located in the interior. Thus the Ritz values corresponding to extreme or isolated eigenvalues converge first.

For nonnormal matrices, the situation is more subtle. The convergence of Arnoldi is not determined by the eigenvalue distribution alone. Nonorthogonality of the eigenvectors can strongly affect the size of $p(A)v_0$, and eigenvalues on the boundary of the spectrum need not be the first to converge. In this case, polynomial approximation must be considered together with the nonnormality of A and the components of the starting vector v_0 in the eigenvector directions.

Exercise 6.1. Let $\chi_m(t) = \det(H_m - tI)$ be the characteristic polynomial of the Hessenberg matrix H_m generated after m steps of the Arnoldi process starting from $v_0 \neq 0$. Assume that no breakdown occurs.

1. Show that

$$\chi_m(A)v_0 \in \mathcal{K}_{m+1}(A; v_0),$$

and using the minimizing property of χ_m show also that

$$\chi_m(A)v_0 \perp \mathcal{K}_m(A; v_0).$$

Deduce that

$$\chi_m(A)v_0 \propto v_{m+1}.$$

2. Show that the proportionality constant can be determined explicitly as

$$\chi_m(A)v_0 = \|v_0\|_2 \left(\prod_{j=1}^m h_{j+1,j} \right) v_{m+1}.$$

3. Let $\theta_1, \dots, \theta_m$ be the Ritz values generated by Arnoldi. Show that

$$\chi_m(t) = \prod_{j=1}^m (t - \theta_j),$$

and conclude that

$$v_{m+1} \propto \prod_{j=1}^m (A - \theta_j I)v_0.$$

What interpretation does this give for the Ritz values in terms of the optimal monic polynomial that minimizes $\|p(A)v_0\|_2$?

Exercise 6.2. Recall the polynomial characterization of the Arnoldi process,

$$\min_{p \in \mathbb{P}_{m+1}^{(1)}} \|p(A)v_0\|_2,$$

and the polynomial characterization of GMRES given in (3.33),

$$\|r_m\|_2 = \min_{p \in \mathbb{P}_m, p(0)=1} \|p(A)r_0\|_2.$$

Compare these two polynomial minimization problems. In particular:

1. Explain why a normalization condition on p is necessary in each minimization problem.
2. What normalization is imposed on the polynomial in Arnoldi, and what normalization is imposed in GMRES?
3. Explain the different roles played by the optimal polynomials in the two methods. In particular, relate the Arnoldi polynomial to the generation of the next Krylov direction and the GMRES polynomial to the reduction of the residual norm.

6.5 Approximation of extreme eigenvalues by Lanczos

The Lanczos method is particularly effective for computing extreme eigenvalues of a large symmetric matrix. Let the eigenvalues of A be ordered as

$$\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n,$$

and denote the Ritz values of T_m by

$$\theta_1^{(m)} \leq \theta_2^{(m)} \leq \cdots \leq \theta_m^{(m)}.$$

Since

$$T_m = V_m^T A V_m,$$

the Rayleigh quotient characterization gives

$$\lambda_1 \leq \theta_1^{(m)} \leq \theta_m^{(m)} \leq \lambda_n.$$

Moreover, because

$$\mathcal{K}_m(A; v_0) \subset \mathcal{K}_{m+1}(A; v_0),$$

the extreme Ritz values satisfy

$$\theta_1^{(m+1)} \leq \theta_1^{(m)}, \quad \theta_m^{(m)} \leq \theta_{m+1}^{(m+1)}.$$

Thus, the smallest Ritz value approaches the lower end of the spectrum from above, while the largest Ritz value approaches the upper end from below.

6.6 Numerical experiments with Lanczos method

As a first example, consider the one-dimensional Poisson matrix

$$A = \text{tridiag}(-1, 2, -1) \in \mathbb{R}^{n \times n},$$

whose eigenvalues are known explicitly:

$$\lambda_j = 2 - 2 \cos \left(\frac{j\pi}{n+1} \right), \quad j = 1, \dots, n.$$

The matrix is real and symmetric, so the Lanczos method is applicable. We take $n = 500$ and we start with a random vector v_0 .

The left panel of Figure 10 shows the Ritz values generated during the first 50 Lanczos iterations. At iteration m , the projected tridiagonal matrix T_m has m eigenvalues, and hence there are m Ritz values. These are shown along the vertical line corresponding to iteration m . For better visibility, vertical lines are drawn in red at every tenth iteration.

The evolution of the Ritz values can be followed visually from one iteration to the next. In particular, one can consider the largest Ritz value as an approximation to the largest eigenvalue of A , while the smallest Ritz value as an approximation to the smallest eigenvalue. The figure clearly shows an important feature of the Lanczos method. Eigenvalues near the ends of the spectral interval, here interval $(0, 4)$, tend to emerge and converge much

earlier than eigenvalues in the interior of the spectrum. As the iteration proceeds, additional Ritz values gradually move toward the interior eigenvalues. The reason can be understood from the polynomial interpretation of Krylov subspace methods. For a Hermitian matrix, low-degree polynomials can often separate an extreme eigenvalue from the remainder of the spectrum much more effectively than they can isolate an eigenvalue lying in the middle of a densely populated spectral interval. Consequently, good approximations to the extreme eigendirections can appear after relatively few Lanczos iterations, whereas approximating a particular interior eigenvalue may require a much larger Krylov subspace.

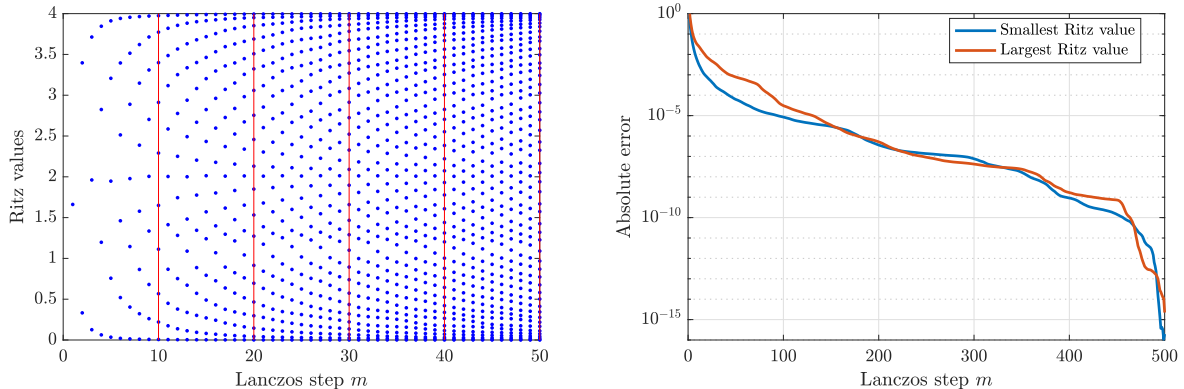


Figure 10: Convergence of Ritz values for the one-dimensional Poisson matrix with $n = 500$. Left: Ritz values during the first 50 Lanczos iterations. Right: convergence errors for selected eigenvalue approximations.

The right panel gives the absolute errors in selected Ritz-value approximations. For the extreme eigenvalues, it is natural to compare the smallest and largest Ritz values with λ_1 and λ_n , respectively. These approximations converge considerably faster than those associated with interior eigenvalues. Nevertheless, convergence is still relatively slow in this example. To achieve an absolute error of about 10^{-10} , approximately 460 Lanczos iterations are required. At this stage, the projected matrix T_m has dimension close to that of the original matrix A . This makes the Lanczos process unattractive if high accuracy is required and motivates techniques like *shift-and-invert* transformation for accelerating convergence. For large-scale problems, *restarting* is also important for limiting the dimension of the Krylov subspace and the associated storage and computational costs.

Next, we illustrate the appearance of *ghost Ritz values*. Consider the diagonal matrix

$$A = \text{diag}(d), \quad d = [0.01 : 0.01 : 1.5, 1.75, 2, 2.5, 3].$$

Thus, A has 150 equally spaced eigenvalues in the interval $[0.01, 1.5]$, together with four additional isolated eigenvalues 1.75, 2, 2.5, and 3. This example is motivated by a similar experiment in [17, 1997]. We choose the starting vector $v_0 = [1, 1, \dots, 1]^T$.

The left panel of Figure 11 shows the Ritz values generated during the first 50 Lanczos iterations. Initially, the behavior is as expected. The isolated eigenvalues are rapidly approximated, with the largest and most isolated eigenvalues converging particularly quickly. However, after sufficiently many iterations, additional Ritz values begin to appear close to

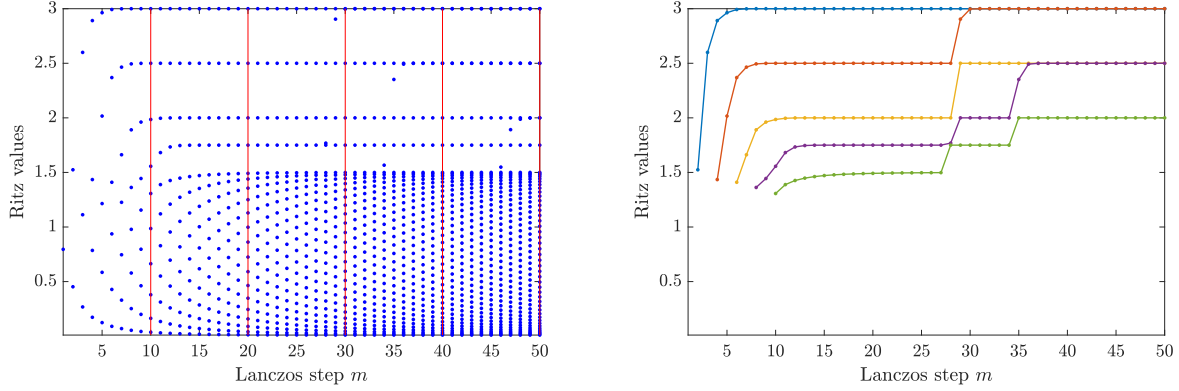


Figure 11: Appearance of ghost Ritz values in Lanczos iteration. Left: Ritz values during the first 50 iterations. Right: convergence paths associated with the five largest eigenvalues, illustrating the effect of loss of orthogonality and the appearance of duplicate Ritz values.

eigenvalues that have already converged. In this experiment, a second Ritz value close to 1.75 appears around iteration 28, and another copy appears around iteration 34. Similar duplication can subsequently be observed near some of the other converged eigenvalues. These spurious duplicate approximations are called *ghost Ritz values*. The right panel of Figure 11 provides another view of the same phenomenon by showing the convergence paths associated with the five largest eigenvalues. Once a ghost value appears, the paths no longer behave as independent approximations to distinct eigenvalues. Instead, some of them jump toward eigenvalues that have already been approximated.

The appearance of ghost Ritz values is closely related to the loss of orthogonality of the Lanczos vectors in finite precision. To see this, suppose that a Ritz pair

$$T_m z = \theta z, \quad u = V_m z,$$

has converged, so that u is a good approximation to an eigenvector x of A associated with an eigenvalue λ . In exact arithmetic, subsequent Lanczos vectors remain orthogonal to the previously generated Krylov subspace and hence to u . Therefore, once this eigendirection has been captured, it is effectively excluded from the subsequent Lanczos process. However, in finite precision this orthogonality is gradually lost. A later Lanczos vector may acquire a small component in the direction of the already converged Ritz vector,

$$v_j = \hat{v}_j + \varepsilon_j u, \quad \hat{v}_j \perp u.$$

Since u is an approximate eigenvector, $Au \approx \lambda u$, and hence

$$Av_j \approx A\hat{v}_j + \varepsilon_j \lambda u.$$

The standard three-term Lanczos recurrence does not explicitly orthogonalize the new vector against all previously computed Lanczos vectors. Consequently, the component in the direction of u is not necessarily removed and may eventually become significant. The later Lanczos vectors can therefore regain a substantial component in an eigendirection that has already converged. In this case, the Lanczos process starts to approximate the same eigenpair

again. Consequently, the projected matrix T_j may acquire another eigenvalue close to the already converged eigenvalue λ which appears as a duplicate or ghost Ritz value.

This reflects the loss of orthogonality in the computed Lanczos basis. In the next section, we discuss reorthogonalization strategies for controlling this loss of orthogonality and suppressing ghost Ritz values.

6.7 Reorthogonalization in the Lanczos method

In exact arithmetic, the Lanczos vectors $v_1, v_2, \dots, v_m$ are orthonormal, even though each new vector is explicitly orthogonalized only against the two preceding vectors. This remarkable property is a consequence of the symmetry of A and leads to the three-term recurrence

$$\beta_{j+1}v_{j+1} = Av_j - \alpha_jv_j - \beta_jv_{j-1}.$$

Thus, unlike the general Arnoldi process, there is no need in exact arithmetic to orthogonalize v_{j+1} against all previously computed Lanczos vectors. In finite precision arithmetic, however, the Lanczos vectors gradually lose orthogonality. This effect becomes noticeable when some Ritz values have converged. The new Lanczos vectors may then develop components in the directions of the corresponding converged Ritz vectors. As we observed in numerical examples of the previous section, an already converged eigenvalue may reappear among the Ritz values and produce the ghost Ritz values.

A direct way of monitoring the loss of orthogonality is to examine

$$\eta_m = \|V_m^T V_m - I\|, \quad V_m = [v_1, \dots, v_m].$$

If η_m becomes large compared with machine precision, the computed Lanczos basis is no longer close to orthonormal. Some reorthogonalization strategies can be used to control this loss of orthogonality [6, 2013].

Full reorthogonalization

The most straightforward approach is to orthogonalize every new Lanczos vector against all previously computed vectors. After computing the candidate vector w from the three-term recurrence, we replace it by

$$w \leftarrow w - V_j(V_j^T w), \quad V_j = [v_1, \dots, v_j],$$

before normalizing it to obtain v_{j+1} . We may apply this procedure twice (applying a second orthogonalization) if orthogonality close to machine precision is desired.

Full reorthogonalization is robust and usually keeps η_m close to machine precision. Its disadvantage is that the computational cost and storage increase as the iteration proceeds. In particular, all previous Lanczos vectors must be stored, and the reorthogonalization at step j requires $\mathcal{O}(nj)$ flops. This partly destroys the main computational advantage of the short Lanczos recurrence.

Selective reorthogonalization

Loss of orthogonality is especially important in directions corresponding to eigenvalues that have already converged. This motivates *selective reorthogonalization*. Rather than orthogonalizing against every previous Lanczos vector, the new vector is reorthogonalized against

converged Ritz vectors. Let

$$T_j z_i = \theta_i z_i$$

be a Ritz pair of the projected tridiagonal matrix. The corresponding Ritz vector is $u_i = V_j z_i$. The residual value

$$\rho_i = \|Au_i - \theta_i u_i\|_2 = |\beta_{j+1} e_j^T z_i|.$$

provides a cheap way to identify converged Ritz pairs. Once a Ritz pair is regarded as converged, subsequent Lanczos vectors can be reorthogonalized against the associated Ritz vector u_i . The purpose is to prevent the iteration from regaining components in directions that have already converged. Selective reorthogonalization can therefore suppress ghost Ritz values at a much lower cost than full reorthogonalization when only a small number of Ritz pairs have converged.

Partial reorthogonalization

A different approach is to allow a moderate loss of orthogonality and reorthogonalize only when it becomes sufficiently large. This is the idea of *partial reorthogonalization*. One may monitor quantities of the form

$$|v_i^T v_j|, \quad i < j,$$

and trigger reorthogonalization when their estimated size exceeds a prescribed threshold. A commonly used value for such threshold is $\sqrt{\epsilon_{\text{mach}}}$. The new Lanczos vector is then reorthogonalized only when necessary, and possibly only against those previous vectors for which the estimated loss of orthogonality is significant.

However, a direct computation of all inner products $v_i^T v_j$ would itself cost $\mathcal{O}(nj)$ flops at step j and would destroy the advantage of partial reorthogonalization. Practical algorithms therefore use recurrence relations to *estimate* the loss of orthogonality from quantities already generated by the Lanczos recurrence. Only when these estimates become too large is explicit reorthogonalization performed. Partial reorthogonalization thus attempts to keep most of the efficiency of the three-term Lanczos recurrence while maintaining enough orthogonality for reliable computations.

In Figure 12, we repeat the previous experiment, this time using the Lanczos method with partial reorthogonalization. The left panel shows that the ghost Ritz values observed in the previous experiment (Figures 11) are suppressed. The right panel shows the loss of orthogonality, measured by $\eta_m = \|V_m^T V_m - I\|_2$, as a function of the Lanczos iteration m . Initially, η_m increases as roundoff errors accumulate. Once the loss of orthogonality reaches the prescribed threshold $\sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}$, reorthogonalization is triggered. As a result, η_m remains controlled at approximately this level throughout the subsequent iterations.

6.8 Computational cost

Assume that A is sparse with $\text{nnz}(A)$ nonzero entries. One Arnoldi step requires one sparse matrix–vector product, costing approximately

$$\mathcal{O}(\text{nnz}(A)),$$

together with orthogonalization against all previous Arnoldi vectors. At step k , this latter part costs $\mathcal{O}(kn)$ flops. After m steps, the total orthogonalization cost is therefore approximately $\mathcal{O}(m^2 n)$, and $\mathcal{O}(mn)$ numbers must be stored for the basis vectors.

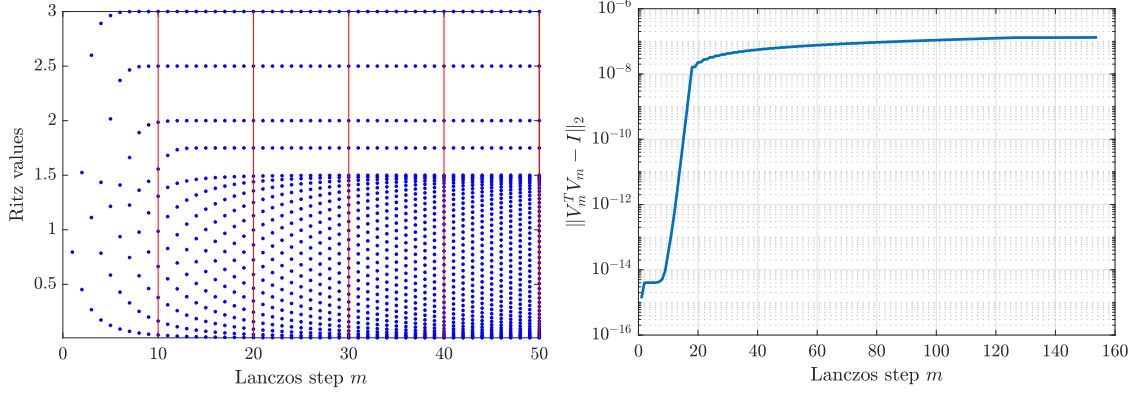


Figure 12: Lanczos iteration with partial reorthogonalization. Left: the ghost Ritz values observed without reorthogonalization are suppressed (compared to Figure 11). Right: the loss of orthogonality $\|V_m^T V_m - I\|_2$ is controlled at approximately the prescribed threshold $\sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}$.

For symmetric matrices, the Lanczos three-term recurrence eliminates most of this cost. Apart from possible reorthogonalization, one step requires

$$\mathcal{O}(\text{nnz}(A)) + \mathcal{O}(n)$$

flops. This makes the Lanczos method particularly attractive for very large sparse symmetric eigenvalue problems.

The projected eigenvalue problem itself has dimension only m . For Arnoldi, a dense eigen-solution of H_m costs $\mathcal{O}(m^3)$ if all eigenpairs are computed, while the symmetric tridiagonal structure of T_m can be exploited in Lanczos. When $m \ll n$ these costs are usually small compared with operations involving the large matrix, but they need not be negligible if a Krylov basis is allowed to become large. This will be briefly addressed in section 6.11.

We observed that there is an important practical distinction between the nice short recurrence of Lanczos and a robust finite precision implementation. If full reorthogonalization is used, the computational cost and storage can again grow like $\mathcal{O}(m^2 n)$ and $\mathcal{O}(mn)$, respectively. Selective or partial reorthogonalization attempts to retain much of the efficiency of Lanczos while preventing serious loss of orthogonality.

6.9 Arnoldi lemniscates

The polynomial interpretation of Arnoldi provides a useful geometric picture of the convergence of Ritz values, particularly for matrices with complex eigenvalues. Following [17, 1997], this picture can be described in terms of *Arnoldi lemniscates*. Recall that after m steps of Arnoldi,

$$p_m(t) = \det(H_m - tI) = \prod_{j=1}^m (t - \theta_j),$$

where $\theta_1, \dots, \theta_m$ are the Ritz values. Moreover, p_m is the monic polynomial of degree m satisfying

$$\|p_m(A)v_0\|_2 = \min_{p \in \mathbb{P}_{m+1}^{(1)}} \|p(A)v_0\|_2.$$

Thus p_m is the polynomial generated implicitly by the Arnoldi process. For an arbitrary constant $c > 0$, the level set $|p_m(t)| = c$ for $t \in \mathbb{C}$ is a polynomial lemniscate. For the Arnoldi process there is a particularly natural choice of the level:

$$\varepsilon_m = \frac{\|p_m(A)v_0\|_2}{\|v_0\|_2} = \prod_{j=1}^m h_{j+1,j}.$$

See also Exercise 6.1. The corresponding curve

$$|p_m(t)| = \varepsilon_m, \quad t \in \mathbb{C}$$

is called the *Arnoldi lemniscate*. Since

$$p_m(t) = \prod_{j=1}^m (t - \theta_j),$$

the Arnoldi lemniscate can equivalently be written as

$$\prod_{j=1}^m |t - \theta_j| = \frac{\|p_m(A)v_0\|_2}{\|v_0\|_2}.$$

Hence its geometry is determined by the Ritz values, while its level is determined by the size of the Arnoldi residual polynomial. To understand this choice, suppose first that A is diagonalizable with orthonormal eigenvectors $x_1, \dots, x_n$, and write $v_0 = c_1x_1 + \dots + c_nx_n$. As we observed previously,

$$\|p_m(A)v_0\|_2^2 = \sum_{i=1}^n |c_i|^2 |p_m(\lambda_i)|^2.$$

After division by $\|v_0\|_2^2 = \sum_k |c_k|^2$, we obtain

$$\varepsilon_m^2 = \frac{\sum_{k=1}^n |c_k|^2 |p_m(\lambda_k)|^2}{\sum_{k=1}^n |c_k|^2}.$$

Thus ε_m is a weighted root mean square value of $|p_m|$ over the eigenvalues of A , with weights determined by the components of the starting vector. Thus an eigenvalue λ_k lies inside the lemniscate if $|p_m(\lambda_k)| < \varepsilon_m$, on the lemniscate if $|p_m(\lambda_k)| = \varepsilon_m$, and outside it if $|p_m(\lambda_k)| > \varepsilon_m$. Suppose that a Ritz value θ_j is approaching an eigenvalue λ_k . Since θ_j is a zero of p_m , $p_m(\lambda_k)$ becomes small when one of the factors $(\lambda_k - \theta_j)$ becomes small. Consequently, λ_k tends to lie inside a component of the Arnoldi lemniscate associated with the nearby Ritz value.

As an example consider the matrix A of size $n = 100$ where the entries are generated from the random normal distribution with mean 0 and standard deviation $n^{-1/2}$ with a change that its top-left entry is set to $a_{11} = 1.5$. Most of the eigenvalues of this matrix are clustered in the unit disk in complex plane. Our modification of a_{11} creates an isolated eigenvalue on the right side. This example is borrowed from [17, 1997]. We apply the Arnoldi iteration

to this matrix starting with a random vector v_0 . The exact eigenvalues (obtained by `eig` command of MATLAB) are shown with black dots in in Figure 13, while the Ritz values computed by the Arnoldi iteration are shown for $m = 6, 7, 8, 9$ with red plus markers. We also plot the Arnoldi lemniscates for each case. For $m = 1$ (not shown here) the lemniscate is nearly a circle. For $n = 6$, a bulge pinches off toward the isolated eigenvalue. For $m = 7$ it starts to separate from the main curve and for $m = 8$ a new component of the lemniscate appears. This small component then proceeds to shrink for $m = 9$.

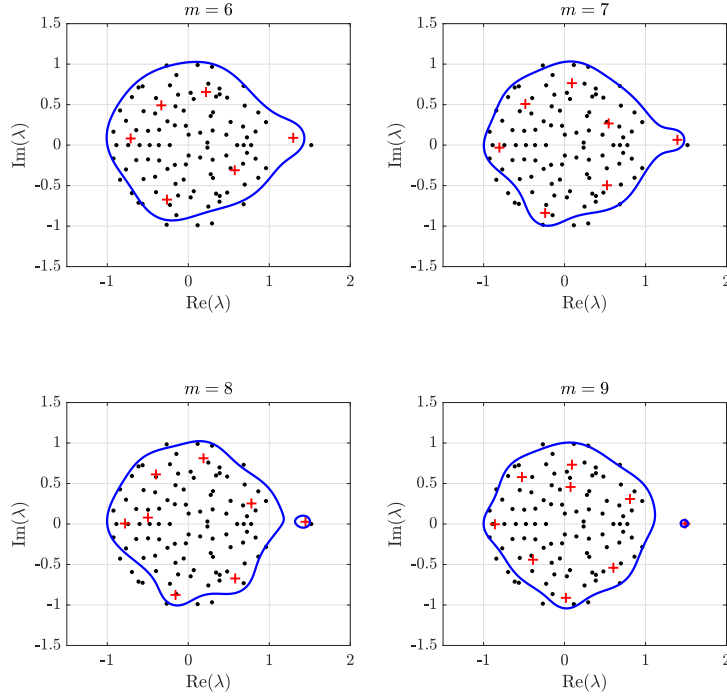


Figure 13: Arnoldi lemniscates at steps $m = 6, 7, 8, 9$. The small black dots are exact eigenvalues, the red plus markers are Ritz values computed by Arnoldi iteration, and the blue curves are Arnoldi lemniscates.

As the iteration proceeds further, components of the lemniscate may separate from one another. A component containing a single eigenvalue indicates that the Arnoldi polynomial has succeeded in separating that part of the spectrum from the rest. If most eigenvalues form a cluster while a few are isolated, components of the Arnoldi lemniscate may separate around the isolated eigenvalues after relatively few iterations. This corresponds to the observation that isolated eigenvalues are usually captured rapidly by Arnoldi. Eigenvalues lying inside a dense cluster are harder to separate by a low degree polynomial and may require more iterations.

6.10 Shift-and-invert transformation

Suppose that we want to compute eigenvalues close to a prescribed number σ . As in shifted inverse iteration, consider the matrix

$$B = (A - \sigma I)^{-1}.$$

If $Av = \lambda v$, then

$$Bv = \frac{1}{\lambda - \sigma}v.$$

Thus, eigenvalues of A close to σ correspond to eigenvalues of B of large magnitude. Arnoldi or Lanczos applied to B will therefore converge preferentially to the eigenvalues of A near σ . If μ is a Ritz value of B , the corresponding approximation to an eigenvalue of A is

$$\lambda = \sigma + \frac{1}{\mu}. \quad (6.7)$$

We should never form $(A - \sigma I)^{-1}$. At every Krylov step, a product

$$w = Bv_k$$

is obtained by solving

$$(A - \sigma I)w = v_k.$$

For a sparse matrix, an incomplete LU (ILU) factorization of $A - \sigma I$ can be computed once and reused at all Krylov iterations. The shift should not be chosen exactly equal to an eigenvalue, since then $A - \sigma I$ is singular.

The transformation strongly changes spectral separation. If two eigenvalues λ_1 and λ_2 are close to σ , their transformed values are

$$\mu_i = \frac{1}{\lambda_i - \sigma}.$$

Hence even eigenvalues located in the interior of the original spectrum can become extreme eigenvalues of the transformed problem. This is the main reason for the effectiveness of shift-and-invert.

For a symmetric matrix, $A - \sigma I$ is symmetric and its inverse is also symmetric. Hence the Lanczos algorithm can still be used. Shift-and-invert Lanczos is one of the standard approaches for computing a few interior eigenvalues of large sparse symmetric matrices.

Remark 6.1. There is an important computational trade-off. Ordinary Arnoldi or Lanczos requires only multiplication by the sparse matrix A , whereas shift-and-invert requires the solution of a linear system with $A - \sigma I$ at every iteration. The number of Krylov iterations may decrease dramatically, but each iteration is more expensive.

Exercise 6.3. For the matrix

$$A = \text{tridiag}(-1, 2, -1) \in \mathbb{R}^{n \times n}.$$

choose a shift value σ in the interior of its spectrum (around 2). Apply shift-and-invert Lanczos method and use (6.7) to recover approximations to the eigenvalues of A nearest to σ . Compare the convergence with ordinary Lanczos.

6.11 Restarting

If A is large and nonsymmetric, the Arnoldi basis

$$V_m = [v_1, \dots, v_m]$$

must be stored, and every new vector must be orthogonalized against all previous vectors. Thus, as in the full GMRES method, both memory and computational cost increase with m .

A simple remedy is to restart the Arnoldi process after a prescribed number of steps. However, an ordinary restart may discard useful information about the desired eigenspace. In practical eigensolvers, more sophisticated restarted strategies are used. The basic idea is to retain information associated with the desired Ritz values while removing components corresponding to unwanted eigenvalues.

One of the most important approaches is the *implicitly restarted Arnoldi method* (IRAM), introduced by Sorensen [16, 1992]. It combines Arnoldi iteration with implicitly shifted QR steps applied to the small Hessenberg matrix H_m . The unwanted Ritz values can be used as shifts, so that the corresponding spectral components are filtered while information associated with the desired eigenvalues is retained. For symmetric matrices, the analogous procedure leads to the *implicitly restarted Lanczos method* (IRLM). Further developments, including deflation strategies for converged Ritz values, are given by Lehoucq and Sorensen [11, 1996]. We do not pursue this idea further and refer the reader to original papers [16, 11] and Chapter 10 of book [6, 2013].

7 Additional Exercises

Exercise 7.1. Consider the system

$$Ax = b, \quad A = \begin{bmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 3 \\ 6 \\ 3 \end{bmatrix}.$$

1. Write down the iteration matrices for the Jacobi and Gauss–Seidel methods.
2. Compute their spectral radii.
3. Starting from $x_0 = 0$, perform three iterations of each method.
4. Compare the observed convergence with the prediction based on the spectral radii.
5. Which method would you expect to converge faster asymptotically? Explain.

Exercise 7.2. Let A be symmetric positive definite and let

$$\phi(x) = \frac{1}{2}x^T Ax - b^T x.$$

1. Show that the unique minimizer of ϕ is the solution of $Ax = b$.

2. Let x_* be the exact solution. Show that

$$\phi(x) - \phi(x_*) = \frac{1}{2} \|x - x_*\|_A^2,$$

where

$$\|z\|_A^2 = z^T A z.$$

3. Use this result to explain why the conjugate gradient iterate satisfies

$$x_m = \arg \min_{x \in x_0 + \mathcal{K}_m(A; r_0)} \|x - x_*\|_A.$$

4. Explain geometrically why CG is particularly appropriate for symmetric positive definite systems.

Exercise 7.3. Suppose that A is symmetric positive definite and has only s distinct eigenvalues.

1. Show that there exists a polynomial q of degree at most $s - 1$ such that

$$A^{-1} = q(A).$$

2. Deduce that if $Ax = b$ and $r_0 = Ax_0 - b$ then

$$x - x_0 \in \mathcal{K}_s(A; r_0).$$

3. Conclude that CG terminates in at most s iterations in exact arithmetic.
4. In particular, what happens if

$$A = \text{diag}(1, 1, 1, 10, 10, 10)?$$

Compare this conclusion with what might be expected by considering only the dimension of the matrix.

Exercise 7.4. Both CG and MINRES can be applied when A is symmetric positive definite, while only MINRES among these two methods is appropriate when A is symmetric indefinite.

1. State the quantity minimized by CG at iteration m .
2. State the quantity minimized by MINRES at iteration m .
3. Explain why the positive definiteness of A is essential for the usual CG method.

4. Consider

$$A = \begin{bmatrix} 2 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 4 \end{bmatrix}.$$

Is CG theoretically appropriate for this problem? Is MINRES? Explain.

5. Both methods are based on Lanczos. Why can MINRES nevertheless be used for a larger class of symmetric matrices?

Exercise 7.5. For GMRES applied to $Ax = b$, the residual can be written as

$$r_m = p_m(A)r_0,$$

where $p_m \in \mathbb{P}_{m+1}$ (polynomials of order $m+1$ or degree m) and $p_m(0) = 1$.

1. Explain why the condition $p_m(0) = 1$ is necessary.
2. Suppose that A is normal,

$$A = Q\Lambda Q^*.$$

Show that

$$\|r_m\|_2 \leq \min_{\substack{p \in \mathbb{P}_{m+1} \\ p(0)=1}} \max_{\lambda \in \lambda(A)} |p(\lambda)| \|r_0\|_2.$$

3. Explain why clustering the eigenvalues away from the origin can lead to fast GMRES convergence.
4. Why can the eigenvalue distribution alone give a misleading picture of GMRES convergence when A is strongly nonnormal?

Exercise 7.6. Consider the preconditioned system

$$M^{-1}Ax = M^{-1}b,$$

where M is an easily invertible approximation to A .

1. Show that if $A = M + E$, then

$$M^{-1}A = I + M^{-1}E.$$

Explain why a small $M^{-1}E$ may be favorable for a Krylov method.

2. Compare the two goals

$$M^{-1}A \approx I \quad \text{and} \quad \text{solving } Mz = r \text{ cheaply.}$$

Why is choosing a good preconditioner a compromise between these goals?

3. Explain why an ILU factorization may be suitable for preconditioned GMRES applied to a nonsymmetric matrix.

4. Why does MINRES require more care in the choice of preconditioner? In particular, explain why a general nonsymmetric ILU preconditioner may destroy the structure required by MINRES.
5. Compare the expected storage requirements of full GMRES, restarted GMRES, and a short-recurrence method such as CG or MINRES.

Exercise 7.7. Let A be symmetric and let (θ, z) be an eigenpair of T_m with $\|z\|_2 = 1$. Using the Lanczos relation prove that the corresponding Ritz vector $u = V_m z$ satisfies

$$\|Au - \theta u\|_2 = |\beta_m| |e_m^T z|.$$

Exercise 7.8. Assume that A is symmetric with eigenvalues

$$\lambda_1 \leq \cdots \leq \lambda_n.$$

Using the minimum and maximum characterization of the Rayleigh quotient, show that

$$\lambda_1 \leq \theta_1^{(m)} \leq \theta_m^{(m)} \leq \lambda_n.$$

Then use

$$\mathcal{K}_m(A; v_0) \subset \mathcal{K}_{m+1}(A; v_0)$$

to show the monotonicity of the smallest and largest Ritz values.

Exercise 7.9. Let $A \in \mathbb{R}^{n \times n}$ and let $V_m \in \mathbb{R}^{n \times m}$ have orthonormal columns spanning a Krylov subspace $\mathcal{K}_m(A; v_0)$. Suppose that

$$H_m = V_m^T A V_m$$

and that (θ, z) is an eigenpair of H_m , with $\|z\|_2 = 1$. Define

$$u = V_m z.$$

1. Show that $\|u\|_2 = 1$.
2. Show that the residual

$$r = Au - \theta u$$

satisfies

$$V_m^T r = 0.$$

3. Explain why this means that the Ritz pair (θ, u) satisfies the Galerkin condition.
4. If A is symmetric, show that

$$\theta = u^T A u.$$

Exercise 7.10. Suppose that the Arnoldi process starting from v_0 terminates at step m because

$$h_{m+1,m} = 0.$$

Using the Arnoldi relation

$$AV_m = V_m H_m + h_{m+1,m} v_{m+1} e_m^T,$$

answer the following questions.

1. Show that

$$A\mathcal{K}_m(A; v_0) \subseteq \mathcal{K}_m(A; v_0).$$

Thus, $\mathcal{K}_m(A; v_0)$ is an invariant subspace of A .

2. Let $H_m z = \theta z$. Show that

$$A(V_m z) = \theta(V_m z).$$

Hence every Ritz value of H_m is an exact eigenvalue of A .

3. Explain why this situation is called a *happy breakdown*.

Exercise 7.11. Let A be a normal matrix with orthonormal eigenvectors $x_1, \dots, x_n$ and corresponding eigenvalues $\lambda_1, \dots, \lambda_n$. Suppose that

$$v_0 = \sum_{j=1}^n c_j x_j.$$

For a polynomial p , show that

$$p(A)v_0 = \sum_{j=1}^n c_j p(\lambda_j) x_j$$

and consequently

$$\|p(A)v_0\|_2^2 = \sum_{j=1}^n |c_j|^2 |p(\lambda_j)|^2.$$

Now suppose that λ_1 is an isolated eigenvalue and that a polynomial q satisfies

$$q(\lambda_1) = 1, \quad |q(\lambda_j)| \leq \delta, \quad j = 2, \dots, n,$$

where $\delta \ll 1$. Show that

$$q(A)v_0 = c_1 x_1 + w,$$

where

$$\|w\|_2 \leq \delta \left(\sum_{j=2}^n |c_j|^2 \right)^{1/2}.$$

Explain how this result gives a polynomial-filtering interpretation of the rapid convergence of Arnoldi toward isolated eigenvalues.

Exercise 7.12. Consider an m -step Arnoldi process for a large nonsymmetric matrix A . Suppose that p Ritz values

$$\mu_1, \dots, \mu_p$$

have been identified as unwanted and define the polynomial

$$\psi(t) = \prod_{j=1}^p (t - \mu_j).$$

1. Suppose, for simplicity, that A is diagonalizable,

$$Ax_k = \lambda_k x_k, \quad v = \sum_k c_k x_k.$$

Show that

$$\psi(A)v = \sum_k c_k \psi(\lambda_k) x_k.$$

2. Explain why choosing μ_j close to unwanted eigenvalues tends to suppress the corresponding eigendirections.
3. Why would simply restarting Arnoldi with an arbitrary new vector generally lose useful spectral information?
4. Explain, from the polynomial viewpoint, the main idea of implicitly restarted Arnoldi: why is it preferable to restart with filtered Krylov information rather than starting the Arnoldi process completely from scratch?

Exercise 7.13. Let $A \in \mathbb{R}^{n \times n}$ and let $\mathcal{V} \subseteq \mathbb{R}^n$ be a subspace. We say that $\mathcal{V}$ is an *invariant subspace* of A (or under A) if

$$A\mathcal{V} \subseteq \mathcal{V}.$$

1. Let x be an eigenvector of A . Show that

$$\mathcal{V} = \text{span}\{x\}$$

is an invariant subspace of A . More generally, if $x_1, \dots, x_k$ are eigenvectors of A , then

$$\mathcal{V} = \text{span}\{x_1, \dots, x_k\}$$

is an invariant subspace of A .

2. Suppose that the Arnoldi process terminates at step m with

$$h_{m+1,m} = 0.$$

Using the Arnoldi relation

$$AV_m = V_m H_m + h_{m+1,m} v_{m+1} e_m^T,$$

show that the Krylov subspace

$$\mathcal{K}_m(A; v_0) = \text{span}\{v_1, \dots, v_m\}$$

is an invariant subspace of A .

3. Let (θ, z) be an eigenpair of H_m . Show that $u = V_m z$ is an eigenvector of A corresponding to the eigenvalue θ . Then Explain why the situation $h_{m+1,m} = 0$ is called a *happy breakdown* in the Arnoldi method for eigenvalue computation.

A Appendix

A.1 Implementation of Lanczos-Galerkin (LanGal) method

As discussed in Section 3.4, when A is symmetric, the Arnoldi–Galerkin method reduces to the Lanczos–Galerkin process with the reduced system

$$T_m y_m = \|r_0\|_2 \hat{e}_1, \quad (\text{A.1})$$

where T_m is the $m \times m$ symmetric tridiagonal matrix generated by the Lanczos process. If A is further assumed to be positive definite, then T_m is also symmetric positive definite. In this case, as shown in Exercise 3.4, an LL^T factorization of T_m leads to recursive formulas for computing the iterates x_m from x_{m-1} without storing the Lanczos matrices T_m and V_m . However, the CG algorithm is a more efficient implementation in this case.

If A is symmetric but indefinite then T_m is no longer guaranteed to be positive definite, and the LL^T factorization may fail. Although one may use an LDL^T factorization, this approach can become unstable in practice. An alternative is the SYMMLQ method [12, 1975], which is based on an LQ factorization

$$T_m = LQ,$$

where L is lower triangular and Q is orthogonal. Then (A.1) becomes

$$LQy = \|r_0\|_2 \hat{e}_1$$

and

$$x_m = x_0 + V_m y = x_0 + V_m Q^T z$$

where z is obtained by forward substitution for $Lz = \|r_0\|_2 \hat{e}_1$. In the SYMMLQ algorithm, the product $V_m Q^T$ is updated recursively, leading again to a short-recurrence method that avoids storing the entire Lanczos basis. Note that, since T_m is symmetric tridiagonal, its LQ factorization can equivalently be obtained from the QR factorization of T_m .

Here we present an alternative algorithm, analogous to the MINRES formulation based on search directions as basis vectors, but now applied to the Galerkin approach. We may call it LanGal algorithm.

We compute the QR factorization of T_m based on Givens rotations. The process is similar (but not exactly the same) to computation of QR factorization of $\tilde{T}_m$ in the MINRES

algorithm detailed in section 3.8. There the matrix $\tilde{T}_m$ has size $(m+1) \times m$, so the last Givens rotation G_m is applied at step m to eliminate the entry β_m . For the square matrix T_m , for $m = 1$ we have $T_1 = [\alpha_1]$ so no rotation requires, and for $m \geq 2$, only the rotations $G_1, \dots, G_{m-1}$ (and not G_m) are applied to eliminate the subdiagonals $\beta_1, \dots, \beta_{m-1}$ in T_m . Let

$$R_m = G_{m-1} \circ \dots \circ G_1 \circ T_m$$

be the upper triangular factor obtained by applying Givens rotations from the left. Since T_m is tridiagonal, R_m is upper tridiagonal. We write

$$R_m = \begin{bmatrix} \alpha_1 & \beta_1 & \gamma_1 & & & \\ & \alpha_2 & \beta_2 & \gamma_2 & & \\ & & & \ddots & \ddots & \\ & & & \ddots & \beta_{m-2} & \gamma_{m-2} \\ & & & & \alpha_{m-1} & \beta_{m-1} \\ & & & & & \hat{\alpha}_m \end{bmatrix}.$$

Here the entries α_j , β_j , and γ_j denote the updated entries after the Givens rotations have been applied; they should not be confused with the original Lanczos coefficients. Since only the rotations $G_1, \dots, G_{m-1}$ are applied here (and not G_m), the last diagonal entry of R_m has been updated only once. For this reason, we denote it by $\hat{\alpha}_m$. This will make the main difference between implementations of two methods. The transformed right-hand side is

$$g_m = G_{m-1} \circ \dots \circ G_1 \circ \hat{e}_1.$$

Note that, unlike the MINRES case where g_m is a vector in $\mathbb{R}^{m+1}$, here $g_m \in \mathbb{R}^m$. Thus the reduced system becomes

$$R_m y_m = \|r_0\|_2 g_m, \quad (\text{A.2})$$

A direct approach would be to solve this upper tridiagonal system by backward substitution and then compute $x_m = V_m y_m$. However, this requires storing all Lanczos vectors in V_m . To avoid this, we introduce a new basis

$$P_m = [p_0, p_1, \dots, p_{m-2}, \hat{p}_{m-1}]$$

defined by

$$P_m = V_m R_m^{-1}, \quad \text{or equivalently} \quad P_m R_m = V_m.$$

The last search direction has a hat notation because it is connected to $\hat{\alpha}_m$. Since R_m is nonsingular unless an exact solution has already been obtained, the columns of P_m form a basis for the same Krylov subspace as the columns of V_m . Comparing the columns in $P_m R_m = V_m$ gives the same three-term recursive formula as (3.32) for vectors p_k for $k = 1, \dots, m-2$. However, for the $k = m-1$ the recursion is

$$\hat{p}_{m-1} = \frac{1}{\hat{\alpha}_m} (v_m - \beta_{m-1} p_{m-2} - \gamma_{m-2} p_{m-3}). \quad (\text{A.3})$$

When we pass to the next $m + 1$, the new rotation G_m is applied on $[\hat{\alpha}_m, \beta_m]^T$ in order to eliminate the new subdiagonal entry β_m . This rotation also changes the previous last diagonal entry $\hat{\alpha}_m$ to

$$\alpha_m = c_m \hat{\alpha}_m + s_m \beta_m,$$

where $c_m = \hat{\alpha}_m / \sqrt{\hat{\alpha}_m^2 + \beta_m^2}$ and $s_m = \beta_m / \sqrt{\hat{\alpha}_m^2 + \beta_m^2}$. Here α_m is the update of the original Lanczos parameter α_m after operation of two Givens rotations. This relations show that $\hat{\alpha}_m / \alpha_m = c_m$, thus from (A.3) we conclude that $p_{m-1} = (\hat{\alpha}_m / \alpha_m) \hat{p}_{m-1}$. This equality is true for all search vectors, i.e.,

$$p_j = c_{j+1} \hat{p}_j, \quad j = 0, 1, \dots \quad (\text{A.4})$$

We now derive a recursive formula for the approximate solution. From (A.2) and $P_m R_m = V_m$, we have

$$x_m = x_0 + V_m y_m = x_0 + P_m R_m y_m = x_0 + \|r_0\|_2 P_m g_m.$$

The vector g_m is updated by applying the Givens rotation G_{m-1} to the previous vector g_{m-1} appended by a zero. For $m \geq 2$ and assuming that $\tau_0 = 1$ we can write

$$g_{m-1} = \begin{bmatrix} \tilde{g}_{m-1} \\ \tau_{m-2} \end{bmatrix},$$

and after appending a zero and applying the new Givens rotation G_{m-1} , we get

$$g_m = \begin{bmatrix} c_{m-1} & s_{m-1} \\ -s_{m-1} & c_{m-1} \end{bmatrix} \circ \begin{bmatrix} \tilde{g}_{m-1} \\ \tau_{m-2} \\ 0 \end{bmatrix} = \begin{bmatrix} \tilde{g}_{m-1} \\ c_{m-1} \tau_{m-2} \\ -s_{m-1} \tau_{m-2} \end{bmatrix}.$$

Therefore,

$$\tau_{m-1} = -s_{m-1} \tau_{m-2}, \quad m \geq 2, \quad \tau_0 = 1.$$

Now we try to write the vector g_m in terms of the previous vector g_{m-1} by adding and subtracting τ_{m-2} in its second last component:

$$g_m = \begin{bmatrix} \tilde{g}_{m-1} \\ \tau_{m-2} + c_{m-1} \tau_{m-2} - \tau_{m-2} \\ -s_{m-1} \tau_{m-2} \end{bmatrix} = \begin{bmatrix} \tilde{g}_{m-1} \\ \tau_{m-2} \\ \tau_{m-1} \end{bmatrix} + \begin{bmatrix} 0 \\ (c_{m-1} - 1) \tau_{m-2} \\ 0 \end{bmatrix} = \begin{bmatrix} g_{m-1} \\ \tau_{m-1} \end{bmatrix} + \begin{bmatrix} 0 \\ (c_{m-1} - 1) \tau_{m-2} \\ 0 \end{bmatrix}.$$

Now we write P_m in terms of P_{m-1} . We have

$$P_m = [p_0, \dots, p_{m-2}, \hat{p}_{m-1}] = [p_0, \dots, p_{m-3}, c_{m-1} \hat{p}_{m-2}, \hat{p}_{m-1}] = \begin{bmatrix} P_{m-1} & \begin{bmatrix} I & 0 \\ 0 & c_{m-1} \end{bmatrix} \end{bmatrix}, \hat{p}_{m-1}.$$

Then we can write

$$\begin{aligned} x_m &= x_0 + \|r_0\|_2 P_m g_m \\ &= x_0 + \|r_0\|_2 \begin{bmatrix} P_{m-1} & \begin{bmatrix} I & 0 \\ 0 & c_{m-1} \end{bmatrix} \end{bmatrix} \begin{bmatrix} g_{m-1} \\ \tau_{m-1} \end{bmatrix} + \|r_0\|_2 (c_{m-1} - 1) \tau_{m-2} p_{m-2} \\ &= x_0 + \|r_0\|_2 P_{m-1} \begin{bmatrix} I & 0 \\ 0 & c_{m-1} \end{bmatrix} g_{m-1} + \|r_0\|_2 \tau_{m-1} \hat{p}_{m-1} + \|r_0\|_2 (c_{m-1} - 1) \tau_{m-2} p_{m-2}. \end{aligned}$$

To extract x_{m-1} from the right side, we write the second term as follows

$$\begin{aligned}
P_{m-1} \begin{bmatrix} I & 0 \\ 0 & c_{m-1} \end{bmatrix} g_{m-1} &= P_{m-1} g_{m-1} - P_{m-1} g_{m-1} + P_{m-1} \begin{bmatrix} I & 0 \\ 0 & c_{m-1} \end{bmatrix} g_{m-1} \\
&= P_{m-1} g_{m-1} - P_{m-1} \begin{bmatrix} 0 \cdot I & 0 \\ 0 & 1 - c_{m-1} \end{bmatrix} \begin{bmatrix} \tilde{g}_{m-1} \\ \tau_{m-2} \end{bmatrix} \\
&= P_{m-1} g_{m-1} - P_{m-1} \begin{bmatrix} 0_{(m-2) \times 1} \\ (1 - c_{m-1}) \tau_{m-2} \end{bmatrix} \\
&= P_{m-1} g_{m-1} + (c_{m-1} - 1) \tau_{m-2} \hat{p}_{m-2}.
\end{aligned}$$

Substituting back to the x_m formula above we get

$$x_m = x_{m-1} + \|r_0\|_2 \tau_{m-1} \hat{p}_{m-1} + \|r_0\|_2 \tau_{m-2} [(c_{m-1} - 1) c_{m-1} + c_{m-1} - 1] \hat{p}_{m-2}.$$

In summary, the update equation is

$$x_m = x_{m-1} + \mu_{m-1} \hat{p}_{m-1} + \nu_{m-2} \hat{p}_{m-2}, \quad m = 2, 3, \dots \quad (\text{A.5})$$

where

$$\begin{aligned}
\mu_{m-1} &= \|r_0\|_2 \tau_{m-1} \\
\nu_{m-2} &= \|r_0\|_2 \tau_{m-2} (c_{m-1}^2 - 1) = \|r_0\|_2 \tau_{m-1} s_{m-1} = \mu_{m-1} s_{m-1}.
\end{aligned} \quad (\text{A.6})$$

Finally the residual is computed according to (3.6) by

$$r_m = -\beta_{m+1} y_m^{(m)} v_{m+1} = -\|r_0\|_2 \beta_{m+1} \frac{\tau_{m-1}}{\hat{\alpha}_m} v_{m+1} = -\|r_0\|_2 \frac{s_m}{c_m} \tau_{m-1} v_{m+1} = \|r_0\|_2 \frac{\tau_m}{c_m} v_{m+1}$$

where the last component of y_m is simply obtained as $\|r_0\|_2 \tau_{m-1} / \hat{\alpha}_m$ from triangular equation (A.2), and then $\beta_{m+1} / \hat{\alpha}_m$ is replaced by s_m / c_m according to definition of Givens parameters. Finally $-s_m \tau_{m-1}$ is replaced by τ_m . Now the norm of residual is

$$\|r_m\|_2 = \|r_0\|_2 \frac{|\tau_m|}{c_m}.$$

Compare this with the residual of MIREs method given in (3.28) to observe that

$$\rho_m^{\text{LanGal}} = \frac{1}{c_m} \rho_m^{\text{MINRES}}.$$

where $\rho_m = \|r_m\|_2$. We have the same relation for residuals of ArGal and GMRES methods. See Exercise 3.11.

For implementation, the process is initiated by $\hat{p}_0 = v_1 / \alpha_1$, $x_1 = x_0 + \|r_0\| \hat{p}_0$ in the first step $m = 1$. In the second step $m = 2$ we compute

$$\hat{p}_1 = \frac{1}{\hat{\alpha}_2} (v_2 - \beta_1 c_1 \hat{p}_0), \quad \mu_1 = \|r_0\|_2 \tau_1 = -\|r_0\|_2 s_1, \quad \nu_0 = \mu_1 s_1$$

and update the solution via (A.5). For iterations $m \geq 2$ we compute the next direction according to (A.3) and (A.4) as

$$\hat{p}_{m-1} = \frac{1}{\hat{\alpha}_m} (v_m - \beta_{m-1} c_{m-1} \hat{p}_{m-2} - \gamma_{m-2} c_{m-2} \hat{p}_{m-3}), \quad m = 3, 4, \dots,$$

and coefficients μ_{m-1} and ν_{m-2} from (A.6) and update the solution by (A.5). Thus, the entire formulation was recast in vectors with hat notations.

Exercise A.1. According to derivation above, write down the algorithm of LanGal method.

The computational cost of this method for solving the symmetric system $Ax = b$ of size $n \times n$ is about

$$\text{nnz}(A) + 16n,$$

the same as the MINRES method detailed in Table 2. Here $\text{nnz}(A)$ is the number of nonzero entries in A .

A.2 Householder and Givens transformations

Suppose that we are given a nonzero vector $x \in \mathbb{C}^n$ and wish to construct a unitary transformation that annihilates all entries of x below the first. Two classical techniques for accomplishing this are Householder reflections and Givens rotations.

Householder reflections. Householder's idea is to construct a reflector that maps the vector x to a multiple of the first coordinate vector. To this end, define

$$u = x + \alpha \hat{e}_1, \quad \alpha = e^{i\theta} \|x\|_2, \quad \theta = \text{angle}(x_1),$$

where $\hat{e}_1 = [1, 0, \dots, 0]^T$ is the first coordinate vector. The corresponding Householder reflector is

$$P = I - \frac{2}{u^H u} u u^H.$$

The matrix P is both Hermitian and unitary,

$$P^H = P, \quad P^H P = I,$$

and satisfies

$$Px = -\alpha \hat{e}_1.$$

Consequently, a single Householder reflection annihilates every entry of x except the first.

Givens rotations. An alternative approach is based on Givens rotations, which eliminate one entry at a time. Consider first the vector

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \neq 0.$$

We seek a 2×2 unitary matrix of the form

$$G = \begin{bmatrix} c & s \\ -\bar{s} & c \end{bmatrix},$$

where $c \in \mathbb{R}$, $s \in \mathbb{C}$, and

$$c^2 + |s|^2 = 1,$$

such that

$$G \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \alpha \\ 0 \end{bmatrix}.$$

If $x_2 = 0$, we simply choose $c = 1$, $s = 0$, $\alpha = x_1$. If $x_1 = 0$, we take $c = 0$, $s = 1$, and $\alpha = x_2$. Otherwise, let

$$\rho = \sqrt{|x_1|^2 + |x_2|^2}, \quad \beta = \frac{x_1}{|x_1|},$$

and define

$$c = \frac{|x_1|}{\rho}, \quad s = \frac{\beta \bar{x}_2}{\rho}, \quad \alpha = \beta \rho.$$

Then c is real and nonnegative,

$$c^2 + |s|^2 = 1,$$

and a straightforward computation verifies that

$$G \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \alpha \\ 0 \end{bmatrix}.$$

For a vector $x \in \mathbb{C}^n$, a sequence of Givens rotations is applied successively. Starting with the last two components, we construct a rotation that annihilates the last entry:

$$G_{n-1} \begin{bmatrix} x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} \alpha_{n-1} \\ 0 \end{bmatrix}.$$

Next, we eliminate the $(n-1)$ st entry by constructing a rotation satisfying

$$G_{n-2} \begin{bmatrix} x_{n-2} \\ \alpha_{n-1} \end{bmatrix} = \begin{bmatrix} \alpha_{n-2} \\ 0 \end{bmatrix}.$$

Proceeding in this manner, we continue upward until the first component is reached. After $n-1$ rotations, all entries below the first have been eliminated.

If G_k° denotes the rotation acting on rows k and $k+1$ only, then the resulting transformation is

$$G = G_1 \circ G_2 \circ \cdots \circ G_{n-1} \circ I,$$

and

$$Gx = \begin{bmatrix} \alpha_1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Householder reflections and Givens rotations each have their advantages. A Householder reflector annihilates an entire subvector in a single transformation and requires approximately $4n$ flops. In contrast, eliminating the same subvector with Givens rotations requires $n-1$ successive rotations and approximately $6n$ flops. Consequently, Householder reflections are generally preferred for reducing dense matrices. On the other hand, Givens rotations affect only two rows (or columns) at a time. This locality makes them particularly well suited for sparse matrices and for applications in which individual nonzero entries must be eliminated without significantly altering the remaining matrix structure.

References

- [1] W. E. Arnoldi. The principle of minimized iterations in the solution of the matrix eigenvalue problem. *Quart. Appl. Math.*, 9(1):17–29, 1951.
- [2] Å Björck. *Numerical Methods in Matrix Computations*. Springer, 2013.
- [3] T. A. Davis and Y. Hu. The university of florida sparse matrix collection. *ACM Trans. Math. Software*, 38(1):1–25, 2011.
- [4] R. Fletcher. Conjugate gradient methods for indefinite systems. In G. A. Watson, editor, *Numerical Analysis*, volume 506 of *Lecture Notes in Mathematics*, pages 73–89. Springer, Berlin, 1976.
- [5] R. W. Freund and N. M. Nachtigal. QMR: A quasi-minimal residual method for non-Hermitian linear systems. *Numer. Math.*, 60(1):315–339, 1991.
- [6] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, 4 edition, 2013.
- [7] A. Greenbaum, V. Pták, and Z. Strakoš. Any nonincreasing convergence curve is possible for GMRES. *SIAM J. Matrix Anal. Appl.*, 17(3):465–469, 1996.
- [8] M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *J. Res. Natl. Bur. Stand.*, 49(6):409–436, 1952.
- [9] C. Lanczos. An iteration method for the solution of the eigenvalue problem of linear differential and integral operators. *J. Res. Natl. Bur. Stand.*, 45(4):255–282, 1950.
- [10] C. Lanczos. Solution of systems of linear equations by minimized iterations. *J. Res. Natl. Bur. Stand.*, 49(1):33–53, 1952.
- [11] R. B. Lehoucq and D. C. Sorensen. Deflation techniques for an implicitly restarted Arnoldi iteration. *SIAM Journal on Matrix Analysis and Applications*, 17(4):789–821, 1996.
- [12] C. C. Paige and M. A. Saunders. Solution of sparse indefinite systems of linear equations. *SIAM J. Numer. Anal.*, 12(4):617–629, 1975.
- [13] Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2 edition, 2003.
- [14] Y. Saad and M. H. Schultz. Gmres: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM J. Sci. Stat. Comput.*, 7(3):856–869, 1986.
- [15] P. Sonneveld. CGS, a fast Lanczos-type solver for nonsymmetric linear systems. *SIAM J. Sci. Stat. Comput.*, 10(1):36–52, 1989.
- [16] D. C. Sorensen. Implicit application of polynomial filters in a k -step Arnoldi method. *SIAM Journal on Matrix Analysis and Applications*, 13(1):357–385, 1992.

- [17] L. N. Trefethen and David Bau III. *Numerical Linear Algebra*. SIAM, Philadelphia, 1997.
- [18] H. A. van der Vorst. Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems. *SIAM J. Sci. Stat. Comput.*, 13(2):631–644, 1992.
- [19] H. F. Walker. Implementation of the gmres method using householder transformations. *SIAM J. Sci. Stat. Comput.*, 9(1):152–163, 1988.
- [20] D. S. Watkins. *Fundamentals of Matrix Computations*. John Wiley & Sons, Hoboken, NJ, 2nd edition 2002, 3rd edition, 2010.
- [21] D. M. Young. Iterative methods for solving partial differential equations of elliptic type. *Trans. Amer. Math. Soc.*, 76(1):92–111, 1954.